



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

TESI DI LAUREA

Crittografia visuale e calcolo sicuro: implementazione in Python del protocollo V2PC

RELATORE

Prof. Roberto De Prisco

Università degli Studi di Salerno

CANDIDATO

Gennaro Carmine Tozza

Matricola: 0512120382

Anno Accademico 2025-2026

“Tutti nascono come originali, ma molti muoiono come fotocopie”.

- San Carlo Acutis

Abstract

La Crittografia Visuale, introdotta da Naor e Shamir nel 1994, consente di suddividere un'immagine segreta in un insieme di share, immagini che appaiono come una collezione casuale di pixel bianchi e neri. Ciascuna share, presa singolarmente, non rivela alcuna informazione sul segreto. La fase di ricostruzione non richiede alcun calcolo: è sufficiente sovrapporre fisicamente le trasparenze su cui le share sono stampate perché l'immagine originale diventi visibile. È il sistema visivo umano a svolgere il lavoro: nessuno strumento crittografico è necessario, e chiunque può partecipare alla ricostruzione senza alcuna conoscenza tecnica. La possibilità di realizzare protocolli crittografici senza l'ausilio di un computer apre scenari di applicazione in contesti in cui i computer non possono essere impiegati, o in cui le persone non si sentono a proprio agio nell'interagire con un computer o nel fidarsi di esso. In questa direzione, D'Arco e De Prisco hanno proposto un protocollo per il calcolo sicuro tra due parti (V2PC) che unisce la costruzione del garbled circuit di Yao con gli schemi di crittografia visuale. Il risultato è un processo interamente fisico: Alice e Bob possono calcolare il valore di una funzione booleana sui rispettivi input privati usando soltanto forbici, trasparenze da sovrapporre e il sistema visivo umano, senza mai ricorrere a chiavi crittografiche o algoritmi di cifratura. L'obiettivo principale di questa tesi è l'implementazione in Python del protocollo V2PC, finalizzata a fornire una realizzazione software dello schema che sia accessibile e documentata. Il lavoro punta a trasformare un risultato teorico elegante in uno strumento concreto, contribuendo a colmare il divario tra la formulazione matematica del protocollo e la sua possibile applicazione pratica o dimostrativa.

Indice

Elenco delle Figure	iv
Elenco delle Tabelle	viii
Introduzione	1
1 Crittografia Visuale	5
1.1 Secret Sharing	5
1.2 Visual Secret Sharing	6
1.3 Il modello di Naor e Shamir	8
1.3.1 (2,2)-VCS	11
1.4 Estensioni e varianti	14
1.4.1 Modello Random Grid	15
1.4.2 Modello Probabilistico	17
1.4.3 Modelli equivalenti	18
1.5 Limiti e compromessi	19
1.5.1 Contrasto	20
1.5.2 Espansione dei pixel	20
1.5.3 Riduzione della casualità	21
1.6 Verso il calcolo sicuro visuale	21
2 Il protocollo V2PC	22
2.1 Il calcolo sicuro a due parti	22
2.2 I circuiti garbled di Yao	24

2.3	La costruzione visuale	25
2.3.1	Un multi-secret sharing visuale	26
2.3.2	Dalla porta cifrata alla porta visuale	29
2.3.3	Il pointer bit	33
2.3.4	Un oblivious transfer fisico	37
2.3.5	Le due fasi del protocollo	40
2.4	Dimensione delle share e qualità della ricostruzione	44
2.5	Un esempio completo	47
2.6	Sicurezza del protocollo	49
2.7	Proprietà e limiti	51
3	Implementazione in Python	53
3.1	Obiettivi e ambito dell'implementazione	53
3.2	Architettura del software	54
3.3	Rappresentazione della funzione booleana	55
3.4	Primitive visuali	56
3.4.1	Random grid e Scheme-2-MVCS	56
3.4.2	Costruzione di una porta	57
3.4.3	Rappresentazione e propagazione del pointer bit	58
3.5	Le fasi dell'implementazione	59
3.5.1	Costruzione di tutte le alternative	59
3.5.2	Distribuzione delle share	60
3.5.3	Ricostruzione	60
3.6	Persistenza ed esportazione delle share	61
3.7	Interfaccia a riga di comando	61
3.8	Demo web	62
3.9	Misure	67
3.9.1	Crescita delle share	67
3.9.2	Probabilità di falso positivo	68
3.10	Limiti	71
4	Conclusioni	72

A Implementazione dello schema (2,2)	75
A.1 Implementazione dello schema (2,2) con Random Grid	77
B Riproduzione del software V2PC	78
B.1 La primitiva Scheme-2-MVCS	78
B.2 Preparazione dell'ambiente	80
B.3 Riproduzione dell'esempio completo	80
B.4 Demo web ed esportazione fisica	82
B.5 Riproduzione delle misure	83
B.6 Riproducibilità e limiti dell'esecuzione	85
B.7 Tavole di taglio delle share	86
Bibliografia	89

Elenco delle figure

1.1	OR logico effettuato dall'occhio umano	8
1.2	Esempio di pixel expansion con $m = 4$. Un singolo pixel logico dell'immagine originale viene suddiviso in un blocco di 2×2 sotto-pixel (sp) su ciascuna <i>share</i>	9
1.3	Le quattro configurazioni dello schema $(2, 2)$. Per un pixel bianco le due <i>share</i> ricevono lo stesso blocco e la sovrapposizione conserva un sotto-pixel trasparente (peso 1, percepito come grigio); per un pixel nero le <i>share</i> ricevono blocchi complementari e la sovrapposizione è interamente nera (peso 2). In entrambi i casi una singola <i>share</i> è una coppia con un sotto-pixel nero e uno bianco, e perciò non rivela il colore del pixel.	12
1.4	Lo schema $(2, 2)$ applicato a un'immagine intera. Il segreto, il logo dell'Università di Salerno ridotto in bianco e nero, viene codificato in due <i>share</i> dall'aspetto casuale; ciascuna, da sola, non rivela nulla. Solo la loro sovrapposizione fa riapparire il segreto: le aree nere come nero pieno, quelle bianche come grigio.	14
1.5	Schema random grid $(2, 2)$ applicato alla scritta UNISA. Immagini generate con il codice riportato nell' Appendice A.1	17

1.6	Spettro dei modelli di crittografia visuale. Ai due estremi si trovano i modelli senza espansione dei pixel ($m = 1$) e con espansione dei pixel grande (m grande), che corrispondono rispettivamente al modello random grid e a quello deterministico di Naor e Shamir. In mezzo si colloca il modello probabilistico generalizzato, che permette di scambiare espansione dei pixel con probabilità di errore.	19
2.1	Scheme-2-MVCS su due segreti 48×48 . La stessa share sh_1 ricostruisce due immagini diverse a seconda di come viene accoppiata: sovrapposta a sh_2 mostra il cerchio, sovrapposta a sh_3 mostra la croce. Le tre share, prese singolarmente, sono indistinguibili dal rumore. Il nero si ricostruisce esatto, mentre le aree bianche appaiono grigie perché metà dei loro pixel viene resa nera. Le immagini sono state prodotte con l'implementazione riportata nell'Appendice B.1.	29
2.2	Le quattro share prodotte dal VGESS di una porta AND con bit di permutazione $b = 0$. Nelle share del filo sinistro il pointer bit è il blocco 1×2 che sporge in basso a sinistra; quelle del filo destro sono larghe il doppio perché concatenano le parti delle due istanze A e B . Il reticolo evidenzia i singoli pixel logici senza riempire lo spazio trasparente sopra il pointer. Prese singolarmente, le share principali sono indistinguibili dal rumore.	32
2.3	Le quattro valutazioni della stessa porta AND. Il cerchio è l'immagine I_0 che codifica il valore 0, la croce è I_1 che codifica il valore 1. Solo la combinazione $(1, 1)$ ricostruisce la croce, come deve fare una porta AND. Il nero riappare esatto, le aree bianche restano grigie per via del modello random grid.	33

2.4	Condivisione in stile GESS del pointer bit di una porta AND, nei due casi $b = 0$ e $b = 1$, dove b è il bit di permutazione della porta soprastante. Ogni riquadro è una share (2,2)-NS da due sotto-pixel, e le due share del filo sinistro coincidono. Sovrapponendo sh_1 alla share destra della stessa istanza si ottiene un blocco con un solo pixel nero, cioè pointer bit 0, quando le due share sono uguali, e un blocco interamente nero, cioè pointer bit 1, quando sono complementari. Per $b = 0$ solo la combinazione (1,1) produce il pointer bit 1; per $b = 1$ i valori del pointer si invertono, poiché esso codifica l'uscita della porta XOR b . Ridisegnata a partire da [1], Fig. 5.	36
2.5	Lo stesso schema per una porta OR. Per $b = 0$ solo la combinazione (0,0) produce il pointer bit 0; per $b = 1$ i valori del pointer si invertono. A parità di b , rispetto alla porta AND cambiano due delle quattro share del filo destro, quelle associate agli ingressi (0,1) e (1,0). Ridisegnata a partire da [1], Fig. 6.	37
2.6	I tre passi dell'oblivious transfer fisico. Il foglietto viene staccato dalla busta scartata prima della distruzione, così che Alice possa verificare che la busta sia stata eliminata senza poter dedurre quale delle due Bob abbia tenuto.	40
2.7	Il circuito della funzione dell'esempio. Il figlio sinistro di G_1 è G_2 , quello destro è G_3 ; per G_3 i figli sono G_6 a sinistra e G_7 a destra. In ciascuna coppia di foglie il filo di Alice è quello sinistro.	47
2.8	Le sei trasparenze in possesso di Bob dopo la consegna e i tre oblivious transfer, disposte sotto la porta a cui appartengono. L'input è $x = (0,1,1)$, $y = (1,1,0)$. Sono riprodotte tutte nella stessa scala: si vede crescere la larghezza da sinistra a destra, dai 12 pixel di x_1 ai 64 di y_3 . I blocchi 1×2 in basso rappresentano i pointer bit delle porte correnti e le share dei pointer bit, protetti mediante lo schema deterministico (2,2), che saranno necessari nelle valutazioni successive.	48

2.9	La trasparenza che Bob ottiene a ciascuna porta, accanto alla porta che la produce. Ogni sovrapposizione consuma il pointer della porta corrente e propaga quelli ancora necessari; la larghezza delle cinque uscite è rispettivamente 10, 18, 32, 16 e 8 pixel. Sul filo di uscita resta la sola immagine 8×8 , qui interamente nera perché $f(x, y) = 1$	49
3.1	Il modulo di configurazione. La formula si scrive con la sintassi della Sezione 3.3, e per ogni variabile riconosciuta compare un selettore binario. I due pulsanti tengono separata la costruzione, che ignora i valori scelti, dalla distribuzione simulata.	63
3.2	Le sei share selezionate per l'assegnamento $x = (0, 1, 1)$, $y = (1, 1, 0)$. Accanto a ciascuna compaiono ingranditi il pointer bit in chiaro della porta corrente e le share dei pointer destinati alle porte superiori. Il filo y_3 , che non fa da ingresso sinistro a nessuna porta, non porta alcun pointer.	64
3.3	La ricostruzione dell'esempio del paper, porta per porta. Le immagini si restringono salendo da G_2 verso G_1 , e le prime quattro porte producono uscite lette come 0: solo la radice arriva al nero pieno. . .	65
3.4	Il pannello del risultato. L'immagine di uscita è interamente nera e viene letta come 1, in accordo con la valutazione booleana della formula sull'assegnamento scelto. Il riquadro a destra riassume che cosa ciascuna delle tre fasi ha visto.	65
3.5	Il circuito disegnato dalla demo. Ogni porta riporta l'uscita letta e la share che la attraversa; alle foglie compaiono la share di ingresso, il pointer in chiaro p e le share dei pointer associati alle porte superiori p_i . Le larghezze crescono scendendo di livello, come previsto dalla concatenazione sul filo destro.	66

Elenco delle tabelle

2.1	La costruzione GenGESS. Il bit di permutazione b fissa l'ordine delle due metà sul filo destro; il pointer bit premesso alla share sinistra vale b per il valore 0 e $\bar{b}$ per il valore 1, cioè $p = v_1 \oplus b$	31
2.2	Lato minimo t dell'unità di share in funzione della profondità d , con probabilità di errore tollerata $\varepsilon = 0,0002$. L'ultima colonna riporta, in pixel, la larghezza della share del filo destro al livello più basso, pari a $2^{d+1}t$. Le due crescite si sommano: l'area dell'unità va all'incirca come $2^{d+1} \ln(1/\varepsilon)$, e la concatenazione aggiunge un altro fattore 2^{d+1}	46
3.1	Organizzazione dei moduli dell'implementazione Python.	55
3.2	Esponente k ricavato dall'errore misurato su quattromila costruzioni per riga, con lati s scelti piccoli perché l'errore resti misurabile. Ogni riga viene dal secondo script della Sezione B.5, eseguito con semi consecutivi a partire da zero: la colonna dell'errore è la frequenza osservata, quella di k si ottiene invertendo la formula come indicato qui sopra.	69
3.3	Lato minimo dell'unità di share per un albero binario completo, con la probabilità di errore tollerata della Sezione 2.4, secondo i due criteri messi a confronto. In un albero completo con L livelli le porte sono $2^L - 1$, quindi lo scarto fra le due colonne si allarga a ogni livello. . .	70
B.1	Materiale prodotto dalla demo web per l'esperimento fisico.	82

Introduzione

Buona parte della crittografia moderna ruota attorno a una domanda: come far collaborare due parti che non si fidano l'una dell'altra. Il calcolo sicuro a due parti ne è un esempio diretto. Alice e Bob possiedono ciascuno un dato privato e vogliono calcolare insieme il valore di una funzione che dipende da entrambi, senza però rivelarsi nulla oltre al risultato. Il problema, posto da Andrew Yao nei primi anni ottanta con l'esempio dei due milionari che vogliono sapere chi è più ricco senza dichiarare il proprio patrimonio [2], ha ricevuto soluzioni generali ed eleganti [3, 4]. Tutte, però, condividono un presupposto: che le due parti dispongano di un computer e abbiano almeno una nozione di che cosa siano chiavi, cifrature e protocolli.

Esistono situazioni in cui questo presupposto non regge. Ci sono contesti in cui un computer non può essere impiegato, e persone che non si sentono a proprio agio nell'affidare i propri dati a una macchina o nel fidarsi di un procedimento che non possono vedere. È qui che si inserisce una linea di ricerca particolare: realizzare protocolli crittografici che funzionino senza l'ausilio di un computer. Lo strumento che rende possibile questa idea è la crittografia visuale, introdotta da Naor e Shamir nel 1994 [5]. Un'immagine segreta viene divisa in più *share* che, prese singolarmente, sembrano rumore privo di struttura; sovrapponendo le trasparenze su cui sono stampate, l'immagine riappare. La ricostruzione non richiede alcun calcolo, la svolge il sistema visivo umano, e la sicurezza è incondizionata, la stessa dell'esempio canonico di cifrario a sicurezza perfetta, il One-Time Pad.

Che la crittografia visuale sappia proteggere un segreto è noto. Meno ovvio è che sappia anche calcolare. D'Arco e De Prisco hanno mostrato che, combinando la costruzione del *garbled circuit* di Yao con gli schemi di crittografia visuale, si può ottenere

un protocollo di calcolo sicuro a due parti la cui fase di valutazione è interamente fisica, chiamato V2PC [1, 6]. Alice e Bob possono valutare una qualsiasi funzione booleana dei propri input privati usando soltanto forbici, trasparenze da sovrapporre e l'occhio, senza mai ricorrere a chiavi crittografiche o algoritmi di cifratura nella fase di calcolo. Un computer serve una sola volta, ad Alice, per preparare le trasparenze; da quel momento in poi il calcolo è pura sovrapposizione di fogli.

Obiettivi e risultati

L'obiettivo di questa tesi è studiare il protocollo V2PC e portarlo dalla descrizione teorica a un'implementazione funzionante, che si possa eseguire e mostrare. La dimostrazione degli autori stabilisce che il protocollo è corretto e sicuro, ma lascia aperte alcune domande quantitative che si pongono soltanto quando si prova a eseguirlo: di quanto crescono le share lungo un circuito concreto, quanto devono essere grandi le immagini perché la ricostruzione di uno zero non venga scambiata per un uno, che aspetto ha il materiale che le due parti dovrebbero ritagliare e sovrapporre. Sono domande a cui si risponde scrivendo il codice ed eseguendolo, più che rileggendo il paper.

Il risultato è una libreria Python scritta da zero sulla descrizione di D'Arco e De Prisco [1, 6], senza riusare il codice del software C++ che gli autori citano [7], il quale ha però fatto da riferimento comportamentale per la numerazione delle porte e la disposizione delle share. Le tre operazioni del protocollo restano separate: la costruzione genera entrambe le alternative di ogni ingresso senza ricevere alcun assegnamento, la distribuzione ne sceglie una sola, la ricostruzione vede soltanto il materiale selezionato. L'esempio del paper usa soltanto porte AND e OR, mentre qui il linguaggio delle formule accetta anche lo XOR e la negazione, e da questi si ottengono NAND, NOR e XNOR per composizione. Gli operatori supportati costituiscono una base funzionalmente completa. Sono ammesse inoltre le formule in cui una variabile compare più volte: ogni occorrenza diventa una foglia con share proprie, al prezzo di più porte e di share complessivamente più grandi. Due interfacce, una a riga di comando e una demo web, permettono di usare il protocollo senza scrivere codice, e la demo prepara anche le trasparenze da stampare per eseguire il calcolo davvero,

con forbici e lucidi.

Eseguire il protocollo ha reso misurabile un punto che la teoria enuncia e lascia in secondo piano, cioè il legame fra correttezza e dimensione delle immagini. Poiché la ricostruzione delle aree bianche è probabilistica, un'uscita che vale zero può essere letta come uno quando le immagini sono troppo piccole, e la stima degli autori lega quel rischio alla profondità del circuito. Sui circuiti ramificati esaminati, però, la frequenza di errore misurata è più alta di quella prevista: sull'esempio del paper, con il lato che gli autori propongono, si osserva il 13,0% di letture sbagliate contro lo 0,02% atteso. Ogni porta, in ricostruzione, sovrappone una griglia casuale nuova, e le griglie che salgono dai due sottoalberi si accumulano entrambe sulla stessa area: le misure sono compatibili con un modello in cui l'esponente è il numero di porte anziché la profondità, e le due quantità coincidono sui circuiti a catena. Il risultato non tocca la sicurezza del protocollo, ma sposta la soglia oltre la quale è praticabile. Va detto subito che cosa il software non fa. L'oblivious transfer fisico basato sulle buste non è implementato, e nessun protocollo crittografico di rete lo sostituisce: la selezione di Bob avviene in locale, nello stesso processo che ha già letto l'intero assegnamento. Il programma riproduce il flusso del protocollo e l'informazione disponibile alla fine di ogni fase, non le sue garanzie di sicurezza.

Organizzazione della tesi

La tesi segue lo stesso percorso con cui il protocollo è costruito, dai mattoni della crittografia visuale fino al codice che li mette insieme.

Capitolo 1

Il primo capitolo introduce la crittografia visuale. Parte dal problema più generale del secret sharing, con le soluzioni di Shamir e Blakley, che ricostruiscono il segreto ma richiedono un calcolatore per farlo, e arriva alla variante proposta da Naor e Shamir, in cui il segreto è un'immagine e la ricostruzione avviene sovrapponendo trasparenze. Dopo il modello formale a matrici di base e il caso più semplice, lo schema (2,2), il capitolo esamina le due varianti principali, il random grid, su cui poggia il protocollo, e il modello probabilistico, e la loro equivalenza. Si chiude sui

compromessi che questi schemi impongono, il contrasto, l'espansione dei pixel e la quantità di casualità richiesta, e su un'osservazione che apre il capitolo successivo: la sovrapposizione non si limita a nascondere un segreto, calcola un OR.

Capitolo 2

Il secondo capitolo ricostruisce il protocollo V2PC di D'Arco e De Prisco. Comincia dai due ingredienti che lo compongono, il calcolo sicuro a due parti nel modello semi-onesto e i circuiti garbled di Yao, e mostra poi come la crittografia visuale possa prendere il posto delle cifrature: lo Scheme-2-MVCS, uno schema di multi-secret sharing su tre share, una realizzazione visuale della singola porta logica, il pointer bit che indica quale metà della share destra ritagliare, e un oblivious transfer eseguito con due buste e un tritacarte. Le due fasi del protocollo, la costruzione delle trasparenze da parte di Alice e la valutazione fisica da parte di Bob, vengono descritte per intero e applicate a un esempio a sei ingressi. Chiudono il capitolo la dimostrazione di sicurezza contro avversari semi-onesti e statici e i limiti della costruzione, primo fra tutti la crescita esponenziale delle share nella profondità del circuito.

Capitolo 3

Il terzo capitolo descrive l'implementazione. Dopo aver dichiarato che cosa il software riproduce e che cosa no, presenta l'organizzazione dei moduli, il modo in cui una funzione booleana si scrive come formula e si traduce in albero, e le primitive visuali su cui poggia tutto il resto: il random grid, lo Scheme-2-MVCS e lo schema deterministico che protegge i pointer bit. Seguono le tre fasi in cui il protocollo è stato spezzato, costruzione, distribuzione e ricostruzione, con i pacchetti su disco che le tengono separate, e i due modi per usarle, la riga di comando e la demo web che disegna il circuito e prepara il kit di stampa. La sezione delle misure confronta poi il comportamento osservato con le previsioni del Capitolo 2: la crescita delle share si comporta come atteso, la probabilità di leggere uno zero come un uno no, e da lì si ricava un criterio di dimensionamento più prudente. Chiudono il capitolo i limiti: l'oblivious transfer che manca, e quelli che discendono dall'ipotesi di fan-out uno su cui il protocollo poggia.

Capitolo 1

Crittografia Visuale

Questo capitolo introduce la crittografia visuale, lo strumento su cui poggia tutto il resto della tesi. Si parte dal problema del secret sharing, cioè dal ripartire un segreto tra più parti in modo che solo alcune coalizioni possano ricostruirlo, e si passa alla sua versione visuale, in cui il segreto è un'immagine e la ricostruzione avviene sovrapponendo trasparenze, senza alcun calcolo. Dopo il modello di base di Naor e Shamir e il caso più semplice, lo schema $(2, 2)$, si vedono le varianti principali, il random grid e il modello probabilistico, e la loro equivalenza. Il capitolo si chiude con i limiti e i compromessi degli schemi visuali e con un'osservazione che apre al capitolo successivo: la sovrapposizione non solo nasconde un segreto, ma calcola un OR, e su questa idea si costruisce il calcolo sicuro visuale.

1.1 Secret Sharing

Supponiamo che undici scienziati stiano lavorando a un progetto segreto. Desiderano chiudere i documenti in un armadietto in modo tale che questo possa essere aperto se e solo se sono presenti almeno sei scienziati. Qual è il numero minimo di lucchetti necessari? Qual è il numero minimo di chiavi che ogni scienziato deve portare con sé?

Questo è il problema proposto da Liu in [8] e che Adi Shamir [9], nel 1979, ha generalizzato considerando i limiti fisici e combinatori di tale approccio. L'obiettivo di Shamir era quello di suddividere un dato segreto D in n frammenti $D_1, \dots, D_n$, in

modo tale che la conoscenza di almeno k frammenti renda D facilmente calcolabile, mentre la conoscenza di $k - 1$ o meno frammenti lo lasci completamente indeterminato. Questo modello è oggi noto come (k, n) -threshold scheme, e costituisce il fondamento di ciò che viene più in generale chiamato secret sharing: la pratica di distribuire un segreto tra più parti in modo controllato, garantendo che solo le coalizioni autorizzate siano in grado di ricostruirlo.

Nello stesso anno, e in modo del tutto indipendente, George Robert Blakley propose una soluzione al medesimo problema partendo da un'intuizione geometrica [10]. Egli descrisse il segreto come un punto di intersezione di vari iperpiani in uno spazio vettoriale, dove ogni "ombra" (frammento) del segreto rappresenta un iperpiano specifico.

Tuttavia, sia l'approccio polinomiale di Shamir che quello geometrico di Blakley condividono un presupposto fondamentale: la necessità di un apparato computazionale per eseguire le operazioni matematiche necessarie alla ricostruzione. In contesti in cui non è disponibile un computer, o dove si desidera una decodifica immediata e intuitiva, tali metodi risultano impraticabili.

1.2 Visual Secret Sharing

Proprio per rispondere a questa esigenza, nel 1994, lo stesso Shamir insieme a Moni Naor ha introdotto in [5] una variante nota come Visual Secret Sharing (VSS). In un tipico schema di crittografia visuale, un'immagine segreta, originariamente concepita in bianco e nero, viene codificata in n immagini all'apparenza casuali, denominate share. Ciascuna share è composta da un insieme di pixel bianchi¹ e neri disposti in modo che, presa singolarmente, essa appaia come puro rumore visivo, privo di qualsiasi struttura riconoscibile. Le share vengono stampate su supporti trasparenti, come dei lucidi. La decrittazione avviene in modo puramente fisico: sovrapponendo k o più lucidi in modo allineato, l'immagine segreta diventa immediatamente visibile all'occhio umano. Qualsiasi sottoinsieme di $k - 1$ o meno share, invece, non rivela alcuna informazione sul segreto.

¹Nelle implementazioni fisiche, i pixel "bianchi" corrispondono ad aree trasparenti. Per convenzione, continueremo a riferirci ad essi come bianchi.

Questa proprietà di sicurezza fa sì che la crittografia visuale offra la cosiddetta sicurezza perfetta: anche un avversario con risorse computazionali illimitate non potrebbe ricavare alcuna informazione dal segreto. Si parla, in termini formali, di sicurezza incondizionata. Da questo punto di vista, lo schema di crittografia visuale condivide le stesse garanzie del One-Time Pad (OTP), noto anche come cifrario di Vernam. Il One-Time Pad, ideato da Gilbert Vernam e la cui sicurezza perfetta fu formalmente dimostrata da Claude Shannon nel 1949 in [11], rappresenta l'esempio canonico di cifrario a sicurezza perfetta, la cui riservatezza è garantita in senso teorico-informativo quando sono rispettate le condizioni previste dallo schema. La proprietà che gli conferisce tale garanzia risiede nell'uso di una chiave generata in modo completamente casuale e lunga quanto il messaggio da cifrare. La chiave, inoltre, deve essere utilizzata per crittografare e decrittografare un solo messaggio, ed essere poi scartata. Poiché il testo cifrato rispecchia interamente la casualità della chiave, esso non fornisce alcuna informazione utile a un attaccante, indipendentemente dalle risorse a sua disposizione.

L'analogia con la crittografia visuale è diretta. L'immagine segreta assume il ruolo del testo in chiaro, mentre l'insieme delle share generate svolge una funzione analoga a quella della chiave casuale: ciascuna share, presa isolatamente, è indipendente dall'immagine originale, esattamente come nell'OTP il testo cifrato, senza la chiave, non fornisce alcuna informazione sul messaggio. Questa corrispondenza strutturale con il più noto esempio di cifrario a sicurezza perfetta costituisce un vantaggio significativo della crittografia visuale rispetto ad altri metodi crittografici applicabili alle immagini, i quali fondano la propria sicurezza su ipotesi di complessità computazionale, offrendo dunque una sicurezza di tipo condizionato. Conviene però tenere distinte due proprietà che l'analogia rischia di confondere. La sicurezza perfetta riguarda la riservatezza: una singola share, o più in generale una coalizione vietata, è indipendente dal segreto, esattamente come il testo cifrato dell'OTP senza la chiave. Non riguarda invece la ricostruzione, cioè la fedeltà con cui l'immagine riappare quando si sovrappongono le share. La ricostruzione è esatta nel modello deterministico, ma solo statistica nei modelli senza espansione dei pixel, come il random grid su cui poggia il protocollo V2PC, dove le aree bianche riappaiono grigie.

1.3 Il modello di Naor e Shamir

Per descrivere in modo rigoroso il funzionamento della crittografia visuale, ci affidiamo alla formalizzazione proposta in letteratura, come illustrato da D'Arco e De Prisco in [12].

Sia I un'immagine segreta che deve essere condivisa visivamente tra un insieme $\mathcal{P} = \{1, 2, \dots, n\}$ di n partecipanti. Una terza parte fidata, chiamata *dealer*, genera n immagini stampate su supporti trasparenti (le *share*), e ne distribuisce privatamente una a ciascun partecipante. Alcuni sottoinsiemi di partecipanti, definiti qualificati (*qualified*) [13], sono in grado di ricostruire il segreto sovrapponendo le proprie *share*. Tutti gli altri sottoinsiemi, definiti vietati (*forbidden*), non possono dedurre alcuna informazione sull'immagine segreta, né sovrapponendo le *share* in loro possesso, né effettuando alcun tipo di calcolo computazionale su di esse.

A differenza della crittografia digitale, in cui è possibile "sottrarre" il rumore, nel mondo fisico se un'area è stampata in nero su un lucido, sovrapporre un secondo lucido non potrà mai renderla trasparente. Questa proprietà, definita *monotonicità*, implica che la sovrapposizione si comporti esattamente come l'operatore logico OR.

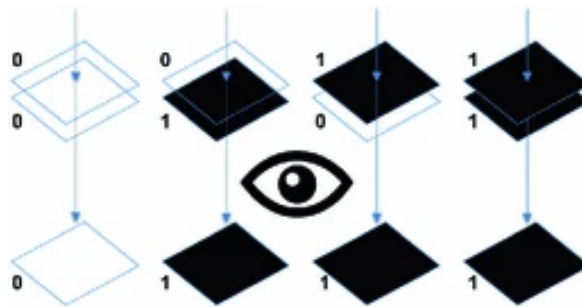


Figura 1.1: OR logico effettuato dall'occhio umano

A causa di questa limitazione, non è possibile mappare direttamente un pixel bianco in un punto trasparente e un nero in un punto opaco sulle singole *share*. Si rende perciò necessario il concetto di *pixel expansion* (espansione dei pixel), denotata con m : ogni singolo pixel dell'immagine segreta viene scomposto in un blocco di m sotto-pixel su ciascuna *share*. Questi sotto-pixel vengono stampati in stretta prossimità l'uno all'altro, in modo che l'occhio umano, operando una naturale media spaziale, ne percepisca soltanto un livello di grigio complessivo.

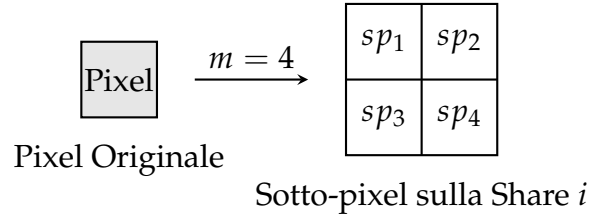


Figura 1.2: Esempio di pixel expansion con $m = 4$. Un singolo pixel logico dell'immagine originale viene suddiviso in un blocco di 2×2 sotto-pixel (sp) su ciascuna *share*.

Resta da precisare *come* il *dealer* sceglie la configurazione dei sotto-pixel su ciascuna *share*. È qui che interviene la formalizzazione di Naor e Shamir [5], ripresa da D'Arco e De Prisco [12]. L'idea è descrivere lo schema non assegnando direttamente i sotto-pixel, bensì attraverso due *collezioni* di matrici booleane, dalle quali il *dealer* estrae casualmente di volta in volta.

Conveniamo di rappresentare un sotto-pixel *nero* con il valore 1 e un sotto-pixel *bianco* (trasparente) con il valore 0. Una *share* relativa a un singolo pixel è dunque un vettore booleano di lunghezza m , e l'insieme delle n *share* si descrive con una matrice booleana $n \times m$, in cui la riga i contiene gli m sotto-pixel assegnati al partecipante i . Poiché la sovrapposizione delle trasparenze coincide con l'operatore OR , il blocco percepito quando i partecipanti di un insieme $X = \{i_1, \dots, i_q\}$ sovrappongono le proprie *share* si ottiene calcolando l' OR delle righe corrispondenti della matrice. Il livello di grigio risultante dipende soltanto dal numero di sotto-pixel neri di tale OR , ovvero dal suo *peso di Hamming* $w(\cdot)$: più sotto-pixel sono neri, più scuro appare il blocco.

Uno schema di crittografia visuale è allora definito da una coppia di collezioni di matrici booleane $n \times m$, $\mathcal{C}^0$ e $\mathcal{C}^1$. Per cifrare un pixel *bianco* il *dealer* sceglie in modo casuale e uniforme una matrice da $\mathcal{C}^0$, per un pixel *nero* una matrice da $\mathcal{C}^1$; in entrambi i casi distribuisce poi la riga i della matrice estratta alla *share* i . Affinché lo schema sia corretto e sicuro, le due collezioni devono soddisfare due condizioni complementari.

- **Condizione di contrasto (leggibilità).** Indichiamo con $w(V)$ il *peso di Hamming* di un blocco di sotto-pixel V , ovvero il numero dei suoi sotto-pixel neri: maggiore è $w(V)$, più scuro appare il blocco all'occhio. La condizione richiede che esistano un intero d (con $1 \leq d \leq m$), che funge da livello di soglia, e un valore

$\alpha > 0$, detto *contrasto relativo*, tali che, quando i partecipanti di un insieme qualificato X sovrappongono le proprie *share*, il blocco risultante, cioè l'OR delle righe di indice X , che denotiamo $\bigvee_{i \in X} S_i$, sia:

- *chiaro* se il pixel è bianco (matrice estratta da $\mathcal{C}^0$): $w(\bigvee_{i \in X} S_i) \leq d - \alpha m$;
- *scuro* se il pixel è nero (matrice estratta da $\mathcal{C}^1$): $w(\bigvee_{i \in X} S_i) \geq d$.

In altre parole, il blocco di un pixel nero contiene almeno αm sotto-pixel neri *in più* rispetto a quello di un pixel bianco. È esattamente questo divario garantito a rendere le due regioni visivamente distinguibili, e il valore α ne misura la qualità: più è grande, più l'immagine ricostruita è leggibile.

- **Condizione di sicurezza (riservatezza).** Questa condizione assicura che un gruppo non autorizzato non possa scoprire il colore del pixel, neanche in parte. Consideriamo un insieme vietato X , cioè un gruppo non in grado di ricostruire (nel caso soglia (k, n) , un gruppo di meno di k partecipanti), e chiediamoci cosa questi partecipanti possono osservare: soltanto le proprie *share*, ossia le righe di indice X della matrice estratta dal *dealer*. L'insieme delle configurazioni che X può vedere, con le rispettive probabilità, deve essere lo stesso sia che il pixel sia bianco, sia che sia nero. Formalmente, restringendo alle righe di X tutte le matrici di $\mathcal{C}^0$ e, separatamente, tutte quelle di $\mathcal{C}^1$, si ottengono due famiglie di matrici $|X| \times m$ che devono coincidere *come multinsiemi*: le stesse configurazioni, ciascuna con la stessa frequenza.

L'effetto è che le *share* in possesso di X hanno un aspetto statisticamente identico nei due casi: osservandole, il gruppo non può in alcun modo capire se il *dealer* stesse codificando un pixel bianco o nero, perché ciò che vede sarebbe potuto comparire, con la stessa probabilità, in entrambe le situazioni. È esattamente il meccanismo del One-Time Pad, dove un testo cifrato può corrispondere a qualunque messaggio in chiaro: l'osservazione non restringe le possibilità, e dunque non rivela nulla.

Nella pratica le collezioni $\mathcal{C}^0$ e $\mathcal{C}^1$ non vengono elencate per esteso, ma generate a partire da due sole *matrici di base* (*basis matrices*) S^0 ed S^1 : la collezione $\mathcal{C}^0$ (rispettivamente $\mathcal{C}^1$) è costituita da tutte le matrici ottenute permutando in tutti i modi possibili

le colonne di S^0 (rispettivamente S^1). La permutazione delle colonne redistribuisce i sotto-pixel senza alterare il peso di Hamming delle righe, ma da sola non basta a garantire la sicurezza: le matrici di base devono essere scelte affinché le collezioni risultanti, ristrette a ogni coalizione vietata, coincidano come multinsiemi, secondo la condizione appena definita. Determinare buone matrici di base, con contrasto α elevato ed espansione m ridotta, è il problema centrale nella progettazione di uno schema di crittografia visuale. Poiché la ricostruzione di un insieme qualificato è sempre corretta, questo schema prende il nome di *modello deterministico*: lo chiameremo così da qui in avanti, per distinguerlo dai modelli in cui la ricostruzione non è sempre esatta. Il caso più semplice, che ne illustra concretamente il meccanismo, è lo schema (2,2).

1.3.1 (2,2)-VCS

Per comprendere appieno il meccanismo formale appena introdotto, si consideri lo schema di crittografia visuale (2,2), ovvero il caso più semplice in cui un'immagine segreta viene divisa in due *share* ($n = 2$) ed entrambe sono necessarie per la ricostruzione ($k = 2$). In questo schema l'espansione dei pixel minima necessaria è pari a due ($m = 2$): ogni pixel dell'immagine segreta viene cioè rappresentato, su ciascuna *share*, da una coppia di sotto-pixel di cui uno bianco e uno nero.

Lo schema è interamente determinato dalle due matrici di base S^0 ed S^1 , di dimensione $n \times m = 2 \times 2$, in cui la riga i descrive il blocco assegnato alla *share* i (1 indica un sotto-pixel nero, 0 uno trasparente). Naor e Shamir pongono:

$$S^0 = \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix} \quad S^1 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix},$$

dove S^0 codifica un pixel *bianco* ed S^1 un pixel *nero*. Come stabilito in generale, il *dealer* non impiega direttamente queste matrici, ma le due collezioni $\mathcal{C}^0$ e $\mathcal{C}^1$ ottenute permutandone le colonne. Con $m = 2$ le permutazioni delle colonne sono soltanto due per cui le collezioni contano due matrici ciascuna:

$$\mathcal{C}^0 = \left\{ \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 1 \\ 0 & 1 \end{bmatrix} \right\}, \quad \mathcal{C}^1 = \left\{ \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \right\}.$$

Per ogni pixel del segreto, il *dealer* estrae a caso e in modo uniforme una matrice dalla collezione corrispondente al colore del pixel, e ne assegna la riga i alla *share* i .

Osserviamo ora l'effetto della sovrapposizione, che coincide con l'OR. Se il pixel è *bianco*, le due righe della matrice estratta da C^0 sono identiche: le due *share* portano lo stesso blocco e, sovrapposte, lasciano un sotto-pixel nero e uno trasparente. Il blocco ricostruito ha peso di Hamming 1 e viene percepito come *grigio*. Se il pixel è *nero*, le due righe (estratte da C^1) sono complementari: la sovrapposizione annerisce entrambi i sotto-pixel, producendo un blocco di peso 2, ossia *nero pieno*.

Le due situazioni si leggono bene osservando i quattro casi possibili, riportati in Figura 1.3. In entrambi i casi una singola *share* mostra, per ogni pixel, una coppia di sotto-pixel con esattamente un elemento nero e uno bianco, scelta in modo uniforme tra le due disposizioni possibili: presa da sola, dunque, non lascia trasparire nulla sul colore del pixel, in accordo con la condizione di sicurezza. È solo la sovrapposizione a fare la differenza. Per un pixel bianco le due *share* coincidono e il blocco ricostruito conserva un solo sotto-pixel nero (peso 1); per un pixel nero le *share* sono complementari e il blocco diventa interamente nero (peso 2). La distanza tra i due livelli è di un sotto-pixel su due, cioè il contrasto $\alpha = 1/2$: è questo divario che l'occhio interpreta come la differenza tra grigio e nero.

	Pixel segreto	Share 1	Share 2	Sovrapposizione	
bianco					peso 1 (grigio)
nero					peso 2 (nero)

Figura 1.3: Le quattro configurazioni dello schema (2,2). Per un pixel bianco le due *share* ricevono lo stesso blocco e la sovrapposizione conserva un sotto-pixel trasparente (peso 1, percepito come grigio); per un pixel nero le *share* ricevono blocchi complementari e la sovrapposizione è interamente nera (peso 2). In entrambi i casi una singola *share* è una coppia con un sotto-pixel nero e uno bianco, e perciò non rivela il colore del pixel.

L'intera procedura si riassume in due algoritmi, riportati nell'Algoritmo 1: la cifratura SHR, che genera le due *share*, e la ricostruzione REC, che si limita a sovrapporle.

Indichiamo con $I(i, j) \in \{0, 1\}$ il colore del pixel in posizione (i, j) (0 per il bianco, 1 per il nero), con C_r^c la r -esima matrice della collezione C^c (con $m = 2$ ogni collezione contiene due matrici, perciò $r \in \{0, 1\}$ ne seleziona una) e con Sup la sovrapposizione delle *share*, che coincide con l'OR logico. La cifratura non fa che ripetere, per ogni pixel, l'estrazione casuale di una matrice e l'assegnazione delle sue righe alle due *share*; la ricostruzione, come anticipato, non richiede alcun calcolo.

Algorithm 1 Schema di crittografia visuale (2,2)

```
1: procedure SHR( $I$ )                                ▷ cifratura: genera le due share
2:   for ogni pixel  $(i, j)$  dell'immagine segreta  $I$  do
3:     scegli  $r_{i,j} \in \{0, 1\}$  in modo casuale e uniforme
4:     for  $k = 1, 2$  do
5:        $sh_k(i, j) \leftarrow$  riga  $k$  della matrice  $C_{r_{i,j}}^{I(i,j)}$ 
6:     end for
7:   end for
8:   return  $(sh_1, sh_2)$ 
9: end procedure

10: procedure REC( $sh_1, sh_2$ )                        ▷ ricostruzione: solo sovrapposizione
11:   return  $I = \text{Sup}(sh_1, sh_2)$ 
12: end procedure
```

Applicando la procedura pixel per pixel a un'immagine intera si ottiene il risultato di Figura 1.4, generato con una semplice implementazione Python dello schema, il cui codice è riportato per intero nell'Appendice A. Il segreto è il logo dell'Università di Salerno, ridotto in bianco e nero; le due *share* appaiono come rumore privo di struttura e, prese separatamente, non rivelano nulla. Solo sovrapponendole il logo riemerge: le aree nere tornano nere piene, mentre quelle bianche restano grigie (circa metà sotto-pixel neri), con la perdita di contrasto tipica della ricostruzione visuale.

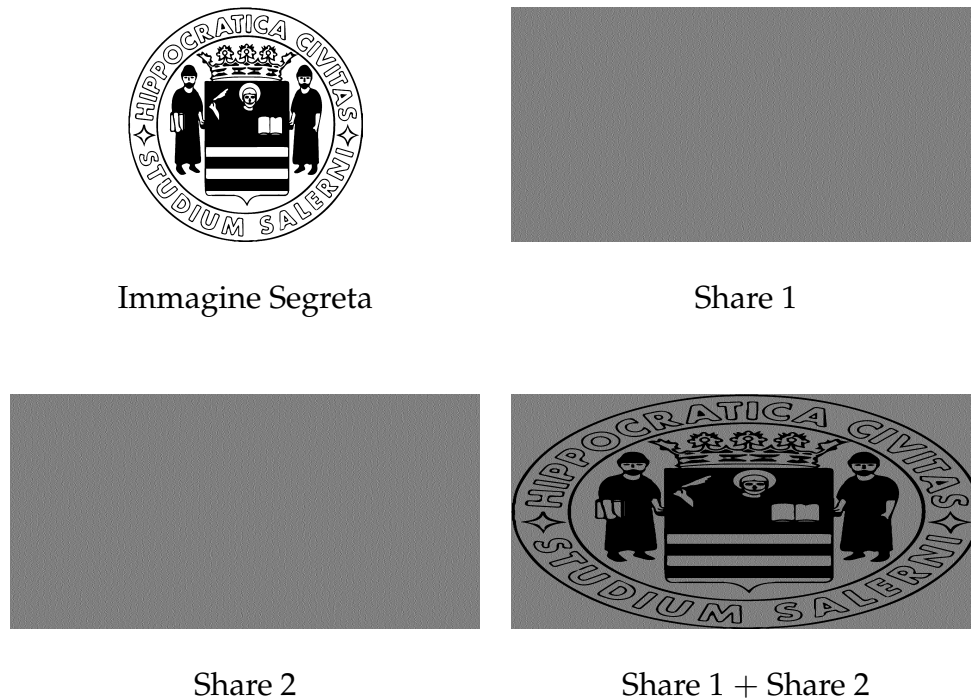


Figura 1.4: Lo schema $(2,2)$ applicato a un'immagine intera. Il segreto, il logo dell'Università di Salerno ridotto in bianco e nero, viene codificato in due *share* dall'aspetto casuale; ciascuna, da sola, non rivela nulla. Solo la loro sovrapposizione fa riapparire il segreto: le aree nere come nero pieno, quelle bianche come grigio.

Lo schema $(2,2)$ è soltanto il caso più semplice. La stessa idea, cioè matrici di base, collezioni ottenute permutando le colonne e ricostruzione tramite OR, si estende ai più generali schemi soglia (k, n) , in cui il segreto è diviso tra n partecipanti e ne bastano k qualsiasi per ricostruirlo, e perfino a strutture di accesso arbitrarie. Il prezzo da pagare è un'espansione dei pixel che cresce con k e n , con un conseguente calo del contrasto: i compromessi tra questi parametri sono discussi nella sezione 1.5.

1.4 Estensioni e varianti

Lo schema deterministico di Naor e Shamir costituisce il punto di partenza della crittografia visuale, ma non ne è l'unica formulazione. Il prezzo da pagare per la ricostruzione perfetta, quella in cui ogni pixel nero del segreto viene ricostruito come un blocco interamente nero, è l'espansione dei pixel m , che ingrandisce le *share* e ne distorce le proporzioni.

Per attenuare questo costo, o semplicemente per descrivere la crittografia visuale da una prospettiva diversa, in letteratura sono stati proposti altri modelli. I principali sono il modello *random grid* e il modello *probabilistico*; come vedremo, lungi dall'essere alternative scollegate, essi risultano strettamente imparentati con il modello deterministico.

1.4.1 Modello Random Grid

Il modello Random Grid è stato introdotto da Kafri e Keren [14]. Una *random grid* è un'immagine delle stesse dimensioni del segreto, in cui ogni pixel è colorato di bianco o nero in modo casuale e indipendente. La caratteristica distintiva di questo modello è l'assenza di espansione dei pixel: ogni pixel del segreto corrisponde a un singolo pixel su ciascuna *share*, ossia $m = 1$. Le *share* hanno perciò la stessa dimensione dell'immagine segreta, senza alcuna distorsione delle proporzioni: è questo il principale vantaggio rispetto al modello deterministico di Naor e Shamir. Consideriamo il caso $(2,2)$, l'unico trattato esplicitamente da Kafri e Keren. La prima *share* R_1 è generata come una random grid: ciascun pixel è bianco o nero con probabilità $1/2$, indipendentemente dagli altri. La seconda *share* R_2 viene poi costruita pixel per pixel in funzione del segreto:

- dove il pixel segreto è *bianco*, R_2 copia il corrispondente pixel di R_1 ;
- dove è *nero*, R_2 ne assume il colore complementare.

Poiché la sovrapposizione delle trasparenze coincide ancora con l'operatore OR , nelle regioni bianche del segreto si sovrappongono due pixel identici e il risultato riproduce R_1 , ossia circa metà pixel neri; nelle regioni nere si sovrappongono pixel complementari e ogni blocco risulta nero. L'occhio percepisce così le aree bianche come un grigio uniforme (circa il 50% di pixel neri) e quelle nere come nero pieno. Non si può più affermare che un'area bianca sia priva di pixel neri; si può solo dire che, in media, contiene meno pixel neri di un'area nera.

La qualità della ricostruzione si misura allora attraverso la *trasmissione luminosa media* (*average light transmission*) λ , definita come la frazione di pixel bianchi in una regione. Indicate con $\lambda_\circ$ e $\lambda_\bullet$ le trasmissioni medie delle aree bianche e nere ricostruite, il

contrasto del modello random grid è dato da $\gamma_{RG} = \frac{\lambda_o - \lambda_\bullet}{1 + \lambda_\bullet}$ [15]. Kafri e Keren si limitarono allo schema $(2, 2)$ e ne suggerirono un'estensione ai livelli di grigio; solo in anni più recenti il modello è stato generalizzato a schemi (k, n) , a immagini a colori e a strutture di accesso arbitrarie.

La costruzione del caso $(2, 2)$ vista sopra è quella che garantisce il contrasto migliore; Kafri e Keren ne proposero in realtà tre varianti, che differiscono nel modo in cui si genera la seconda *share* [14]. La procedura, nella variante qui considerata, è riassunta nell'Algoritmo 2.

Algorithm 2 Random grid $(2, 2)$ (algoritmo di Kafri e Keren)

```

1: procedure SHRRG( $I$ )                                ▷ cifratura: nessuna espansione dei pixel
2:   for ogni pixel  $(i, j)$  dell'immagine segreta  $I$  do
3:      $R_1(i, j) \leftarrow$  bit casuale in  $\{0, 1\}$  scelto in modo uniforme
4:     if  $I(i, j) = 0$  then                                ▷ pixel bianco
5:        $R_2(i, j) \leftarrow R_1(i, j)$ 
6:     else                                                ▷ pixel nero
7:        $R_2(i, j) \leftarrow 1 - R_1(i, j)$ 
8:     end if
9:   end for
10:  return  $(R_1, R_2)$ 
11: end procedure

12: procedure REC( $R_1, R_2$ )                                ▷ ricostruzione: solo sovrapposizione
13:  return  $I = \text{Sup}(R_1, R_2)$ 
14: end procedure

```

La sicurezza si legge bene in questo schema. Presa singolarmente, ogni *share* è una random grid uniforme: i suoi pixel sono bianchi o neri con probabilità $1/2$ a prescindere dal segreto, perciò la trasmittanza media di una sola *share* è identica nei due casi e da essa non si ricava alcuna informazione. La formalizzazione ripresa da D'Arco e De Prisco [12] richiede esattamente due condizioni: una di *contrasto*, per cui la sovrapposizione di un insieme qualificato deve avere trasmittanza media più alta sulle aree bianche (λ_o) che su quelle nere ($\lambda_\bullet$), con $\lambda_o > \lambda_\bullet$; e una di *sicurezza*, per

cui le *share* di un insieme vietato hanno la stessa trasmittanza media nei due casi, e dunque non permettono di distinguere un'area bianca da una nera.

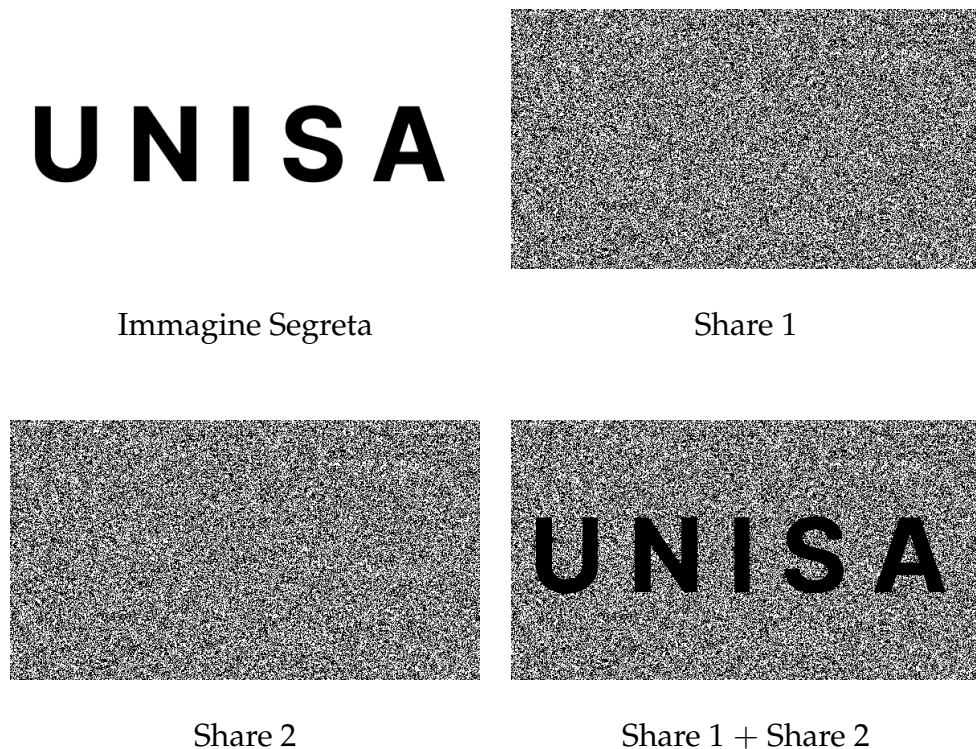


Figura 1.5: Schema random grid (2,2) applicato alla scritta UNISA. Immagini generate con il codice riportato nell' Appendice A.1

Conviene confrontare questa figura con quella dello schema (2,2) deterministico: là la ricostruzione usciva larga il doppio del segreto, qui ha la stessa dimensione. È il vantaggio del random grid, pagato però con un contrasto inferiore e una ricostruzione soltanto statistica. I due modelli, del resto, non sono così distanti: De Prisco e De Santis hanno dimostrato che sono di fatto equivalenti, come vedremo più avanti in questa sezione.

1.4.2 Modello Probabilistico

Il modello *probabilistico* fu proposto da Yang nel 2004 [16]. L'idea è conservare l'impianto di Naor e Shamir, con le sue matrici di base e le collezioni, ma rinunciare all'espansione dei pixel. Ogni pixel del segreto viene allora codificato con un solo sotto-pixel per *share* ($m = 1$), scelto a caso secondo una probabilità ricavata dalle colonne delle matrici di base. Il prezzo, come nel random grid, è che la ricostruzione

non è più esatta: un pixel bianco risulta bianco con probabilità p_0 , un pixel nero risulta bianco con probabilità p_1 , e si ha $p_1 < p_0$. La differenza $\beta = p_0 - p_1$, detta *fattore probabilistico*, misura la leggibilità dello schema, così come α misura il contrasto nel caso deterministico.

Su una singola immagine la correttezza vale solo in media. Qualche pixel viene ricostruito con il colore sbagliato, ma gli errori sono pochi e il segreto resta riconoscibile. È la stessa ricostruzione del random grid, scritta però nel linguaggio delle matrici di base.

Cimato, De Prisco e De Santis [17] hanno poi generalizzato lo schema, rimettendo l'espansione dei pixel tra i parametri liberi ($m \geq 1$). Qui β e m si possono scambiare: più si aumenta m , meno la ricostruzione è incerta, finché per m abbastanza grande si torna alla correttezza esatta del modello deterministico. I due casi limite ricadono così sotto un'unica descrizione: per $m = 1$ si ritrova Yang, per m grande Naor e Shamir, e in mezzo si baratta espansione dei pixel con probabilità di errore.

1.4.3 Modelli equivalenti

I tre modelli visti finora, deterministico, probabilistico e random grid, sembrano risposte diverse allo stesso problema, ma descrivono lo stesso fenomeno. De Prisco e De Santis lo hanno reso preciso [15]: a ogni schema random grid corrisponde uno schema deterministico, e viceversa. Le due tecniche, formulate a sette anni di distanza e con linguaggi diversi, si traducono l'una nell'altra, e per accorgersene sono serviti altri vent'anni. Ne segue subito un'altra identità: il modello random grid coincide con quello probabilistico privo di espansione, cioè con il caso $m = 1$.

Messi insieme, questi risultati danno un quadro unico, in cui tutti i modelli sono governati da un solo parametro, l'espansione dei pixel m (Figura 1.6) [12]. Agli estremi di questo spettro ritroviamo i casi già visti. Per $m = 1$ siamo nel caso senza espansione dei pixel, con ricostruzione non esatta. Per m abbastanza grande si ricade invece nel modello deterministico di Naor e Shamir, che ricostruisce il segreto in modo esatto, al prezzo di *share* più grandi dell'originale. In mezzo si colloca il modello probabilistico generalizzato: qui il compromesso tra espansione dei pixel e probabilità di errore si sposta con continuità, e all'aumentare di m la ricostruzione diventa più

fedele mentre le *share* crescono.

Questa equivalenza ha un risvolto pratico. Un risultato dimostrato in un modello, per esempio un limite sul contrasto o una costruzione efficiente, può essere riportato negli altri. La scelta del modello diventa così una questione di convenienza, da decidere in base a quale compromesso serve all'applicazione che si ha davanti.

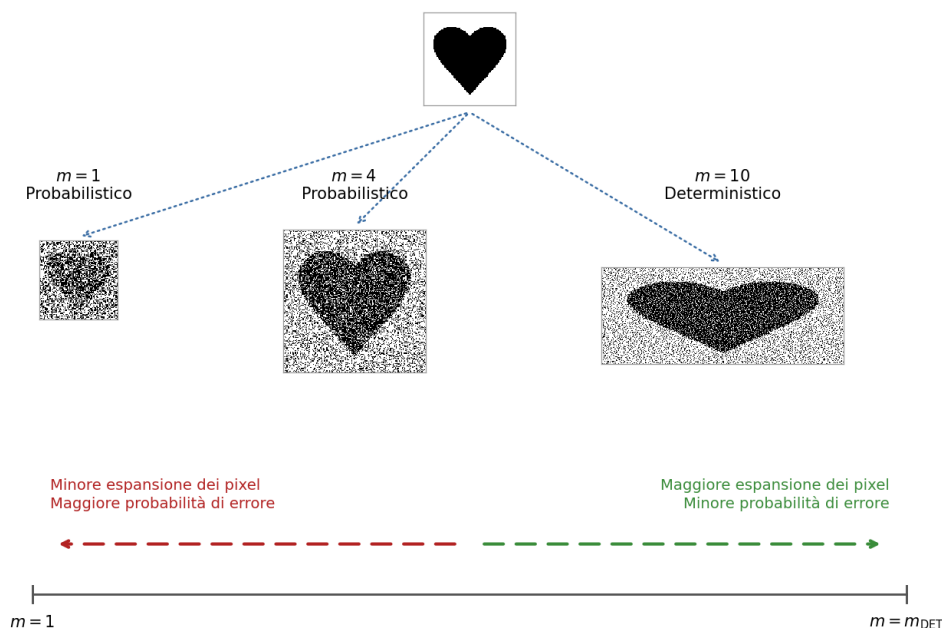


Figura 1.6: Spettro dei modelli di crittografia visuale. Ai due estremi si trovano i modelli senza espansione dei pixel ($m = 1$) e con espansione dei pixel grande (m grande), che corrispondono rispettivamente al modello random grid e a quello deterministico di Naor e Shamir. In mezzo si colloca il modello probabilistico generalizzato, che permette di scambiare espansione dei pixel con probabilità di errore.

1.5 Limiti e compromessi

La qualità e la praticità di uno schema di crittografia visuale dipendono da tre grandezze legate tra loro: il contrasto, l'espansione dei pixel e la quantità di casualità richiesta. Sono in tensione, perché migliorarne una di solito peggiora le altre, e buona parte della ricerca sulla crittografia visuale consiste nello studiare questi compromessi.

1.5.1 Contrasto

Il contrasto α misura quanto le aree bianche e nere restino distinguibili nella ricostruzione: più è grande, più l'immagine è leggibile. Non lo si può però aumentare a piacere, e il limite dipende sia da k sia da n . Hofmeister, Krause e Simon hanno dato un metodo per calcolare il contrasto ottimo di un qualunque schema (k, n) risolvendo un programma lineare, e ne hanno ricavato due conseguenze [18]. La prima riguarda il caso $k = n$, in cui gli schemi (k, k) di Naor e Shamir risultano ottimi con contrasto $2^{-(k-1)}$: per $k = 2$ si ritrova il valore $\alpha = 1/2$ dello schema $(2, 2)$ visto sopra. La seconda riguarda gli schemi (k, n) con n qualsiasi, per i quali, con $k = 2, 3, 4$, il contrasto ottenibile è almeno $1/4$, $1/16$ e $1/64$, cioè $4^{-(k-1)}$: chiedere che uno schema serva anche coalizioni scelte in un insieme grande costa contrasto. In entrambi i casi il valore cala in modo esponenziale in k . Determinare il contrasto migliore possibile, e le matrici di base che lo raggiungono, è uno dei problemi più studiati: Blundo, De Santis e Stinson hanno caratterizzato completamente gli schemi $(2, n)$ a contrasto ottimo e dato limiti superiori e inferiori per $k \geq 3$ [19], mentre Blundo, De Bonis e De Santis hanno costruito schemi che migliorano il contrasto a parità di altri parametri [20].

1.5.2 Espansione dei pixel

L'espansione dei pixel m è il costo più diretto del modello deterministico: ogni pixel diventa un blocco di m sotto-pixel, quindi le *share* e la ricostruzione sono m volte più grandi del segreto, con maggiore consumo di supporti e più difficoltà di allineamento. Contrasto ed espansione sono legati: in genere un contrasto migliore si paga con un'espansione maggiore, e viceversa. Negli schemi (k, n) l'espansione cresce con il numero di partecipanti, perciò ridurla al minimo mantenendo un contrasto accettabile è un obiettivo centrale della progettazione [19]. È anche per evitare del tutto questo costo che i modelli random grid e probabilistico rinunciano all'espansione ($m = 1$), accettando in cambio una ricostruzione non esatta.

1.5.3 Riduzione della casualità

Un costo meno evidente ma reale è la casualità. Per cifrare ogni pixel il *dealer* compie una scelta casuale, cioè quale matrice estrarre dalla collezione, e ogni scelta consuma bit casuali. Su immagini grandi questo si traduce in una notevole quantità di casualità richiesta, che incide sull'efficienza della generazione delle *share*. Quanta casualità serva davvero, e come ridurla senza peggiorare contrasto ed espansione, è una questione studiata da De Bonis e De Santis [21].

1.6 Verso il calcolo sicuro visuale

Fin qui la crittografia visuale è stata presentata per ciò per cui è nata: nascondere e condividere un'immagine. Vale però la pena di osservare il meccanismo da un'angolazione diversa. La sovrapposizione delle trasparenze calcola gratuitamente un OR logico; le *share* codificano informazione con sicurezza perfetta, analoga a quella del One-Time Pad; e tutto questo avviene senza alcun calcolo elettronico, affidandosi soltanto al sistema visivo umano. Se la sovrapposizione è in grado di eseguire un'operazione logica, è naturale chiedersi se gli stessi strumenti, cioè trasparenze, sotto-pixel e sovrapposizione, possano essere impiegati per fare qualcosa di più che proteggere un segreto: per *calcolare* su dati segreti.

È esattamente questa la direzione che esploreremo nel prossimo capitolo. D'Arco e De Prisco hanno mostrato come combinare gli schemi di crittografia visuale con la costruzione del *garbled circuit* di Yao per ottenere un protocollo di calcolo sicuro a due parti interamente fisico, denominato V2PC. Le proprietà della crittografia visuale discusse in questo capitolo, cioè la sicurezza incondizionata, la decrittazione senza calcolo e la natura puramente ottica della ricostruzione, sono precisamente i mattoni su cui quel protocollo è costruito.

Capitolo 2

Il protocollo V2PC

Nel capitolo precedente la crittografia visuale è stata usata per proteggere un segreto: un'immagine viene divisa in *share* che, prese singolarmente, non rivelano nulla, e che ricompongono il segreto quando vengono sovrapposte. L'ultima sezione ha però lasciato intravedere un uso diverso dello stesso strumento. La sovrapposizione di due trasparenze calcola un OR logico, e lo fa senza alcun apparato di calcolo, affidandosi soltanto all'occhio. Se una semplice operazione booleana è già alla portata di questo meccanismo, è lecito chiedersi fino a che punto ci si possa spingere: si può usare la crittografia visuale non solo per nascondere dati, ma per *calcolare* su dati segreti?

La risposta, dovuta a D'Arco e De Prisco [1, 6], è affermativa. Combinando la costruzione del *garbled circuit* di Yao con gli schemi di crittografia visuale, i due autori hanno ottenuto un protocollo di calcolo sicuro a due parti in cui tutta la fase di valutazione è fisica: Alice e Bob calcolano il valore di una funzione booleana sui propri input privati usando soltanto forbici, trasparenze da sovrapporre e il sistema visivo umano. Il protocollo prende il nome di V2PC (*Visual Two-Party Computation*). Questo capitolo lo ricostruisce a partire dai suoi due ingredienti, il calcolo sicuro a due parti e i circuiti garbled, per poi descriverne la costruzione, la sicurezza e i limiti.

2.1 Il calcolo sicuro a due parti

Il problema del calcolo sicuro a due parti fu posto da Andrew Yao nel 1982 con un esempio diventato celebre, quello dei milionari [2]. Due milionari vogliono sapere

chi dei due è più ricco, ma nessuno dei due è disposto a rivelare all'altro la propria ricchezza. Esiste un modo per stabilire chi vince il confronto senza che nessuno dei due apprenda la cifra dell'altro? Il caso dei milionari è solo l'istanza più nota di una domanda generale: due parti, che chiameremo Alice e Bob, dispongono ciascuna di un input privato, x e y , e vogliono calcolare il valore di una funzione concordata $f(x, y)$ in modo che al termine entrambe conoscano il risultato e nient'altro. In particolare, Alice non deve apprendere nulla di y oltre a ciò che il risultato stesso lascia dedurre, e simmetricamente per Bob.

Un modo semplice per raggiungere l'obiettivo consiste nell'affidarsi a una terza parte fidata: Alice e Bob le comunicano privatamente i propri input, essa calcola $f(x, y)$ e restituisce a entrambi il risultato, dimenticando poi tutto. Questa soluzione ideale definisce esattamente ciò che si vorrebbe ottenere, ma richiede un intermediario di cui fidarsi ciecamente. Il calcolo sicuro a due parti punta a fare a meno di lui: si vuole che il protocollo, eseguito direttamente da Alice e Bob, offra le stesse garanzie della terza parte fidata [4, 22].

Per rendere precisa questa idea occorre fissare il modello di avversario. In questo capitolo, come nel protocollo che descriveremo, si considera un avversario *statico* e *semi-onesto* (*semi-honest*, onesto ma curioso). Un avversario di questo tipo controlla una delle due parti fin dall'inizio e ne segue fedelmente le istruzioni, senza deviare dal protocollo; tenta però di ricavare informazioni in più esaminando tutto ciò che la parte corrotta vede durante l'esecuzione, cioè il proprio input, la propria casualità e i messaggi ricevuti. È un modello meno pessimistico di quello *malicious*, in cui la parte corrotta può anche comportarsi in modo arbitrario, ma è sufficiente in molte situazioni reali e rende la trattazione più pulita.

La sicurezza si formalizza con il paradigma della simulazione. L'insieme di tutto ciò che una parte osserva durante l'esecuzione prende il nome di *view*. Un protocollo è sicuro se, per ciascuna delle due parti, esiste un algoritmo efficiente, detto *simulatore*, che a partire dal solo input di quella parte e dal solo risultato $f(x, y)$ produce una *view* distribuita esattamente come quella reale. Se una *view* simulabile a partire da input e risultato è indistinguibile da quella vera, allora la parte non può aver appreso nulla che non fosse già deducibile da ciò che le spetta di diritto: tutto il resto, in linea di principio, se lo sarebbe potuto costruire da sola. La definizione,

nella sua versione incondizionata, richiede l'uguaglianza delle due distribuzioni; nella versione computazionale ci si accontenta della loro indistinguibilità per un avversario con risorse limitate [23].

2.2 I circuiti garbled di Yao

La costruzione con cui Yao risolse il problema generale è quella dei circuiti garbled [3]. Il punto di partenza è la rappresentazione della funzione f come circuito booleano C : un insieme di fili collegati da porte logiche a due ingressi, in cui $2n$ fili portano i bit di input di Alice e Bob e un filo finale porta il bit di uscita. Ogni funzione booleana può essere descritta in questa forma. I circuiti che consideriamo sono formule, cioè alberi: ogni filo alimenta una sola porta (fan-out uno) e ogni porta ha un unico filo di uscita. È l'ipotesi su cui poggia la costruzione visuale, e su cui, come vedremo, la negazione diventa gratuita; il caso di fan-out maggiore di uno è discusso nel Capitolo 3.

L'idea di Yao è cifrare il circuito. A ciascun filo ω_i si associano due chiavi crittografiche indipendenti, $k_{\omega_i}^0$ e $k_{\omega_i}^1$, che rappresentano in forma segreta i due possibili valori del filo, 0 e 1: chi possiede una chiave può usarla per proseguire la valutazione, senza che la chiave riveli di per sé quale bit rappresenti. A ogni porta G , con fili di ingresso ω_i, ω_j e filo di uscita ω_k , si associa poi una *tabella* composta da quattro doppie cifrature, una per ciascuna combinazione dei valori di ingresso. La riga relativa alla coppia (a, b) cifra la chiave di uscita $k_{\omega_k}^{G(a,b)}$ sotto le due chiavi di ingresso $k_{\omega_i}^a$ e $k_{\omega_j}^b$:

$$\text{Enc}_{k_{\omega_i}^a} \left(\text{Enc}_{k_{\omega_j}^b} \left(k_{\omega_k}^{G(a,b)} \right) \right), \quad (a, b) \in \{0, 1\}^2.$$

Le quattro righe vengono disposte in ordine casuale, così che la loro posizione non riveli nulla, e la cifratura è congegnata in modo che solo la coppia di chiavi corretta permetta di decifrare una riga valida. Chi possiede $k_{\omega_i}^a$ e $k_{\omega_j}^b$ riesce a recuperare, e solo, la chiave $k_{\omega_k}^{G(a,b)}$: valuta cioè la porta senza mai conoscere i valori in chiaro dei fili. L'intero circuito cifrato, che chiamiamo $\tilde{C}$, è la collezione di tutte queste tabelle insieme alla corrispondenza esplicita tra le chiavi del filo di uscita e i bit 0 e 1.

Il protocollo si svolge allora in due fasi. Nella prima, che si può eseguire in anticipo, Alice costruisce $\tilde{C}$ e lo invia a Bob. Nella seconda, Alice consegna a Bob le chiavi corrispondenti al proprio input e, per ciascun bit di Bob, i due eseguono un protocollo

di *oblivious transfer* 1-su-2 [24]: Bob riceve la chiave del proprio bit senza apprendere l'altra, mentre Alice non apprende quale bit Bob abbia scelto. A questo punto Bob possiede una chiave per ogni filo di ingresso e può valutare il circuito tabella per tabella, fino a ottenere la chiave del filo di uscita, che la corrispondenza esplicita traduce nel bit $f(x, y)$. Le porte NOT non richiedono una tabella propria: poiché ogni filo alimenta una sola porta, negarlo equivale a scambiare le due chiavi che lo etichettano, invertendo il significato di 0 e 1. La costruzione è di Yao, che però non ne pubblicò mai la dimostrazione: la prima prova completa di sicurezza nel modello semi-onesto, sotto ipotesi standard sullo schema di cifratura e sull'*oblivious transfer*, è di Lindell e Pinkas [23].

Il meccanismo che rende sicuro il circuito garbled è tutto nelle chiavi e nelle cifrature: sono queste a nascondere i valori dei fili e a lasciar trapelare, di ogni porta, soltanto la riga giusta. È esattamente questo il punto in cui la crittografia visuale entra in gioco.

2.3 La costruzione visuale

L'intuizione di D'Arco e De Prisco è sostituire le cifrature del circuito garbled con share visuali, e la decifrazione con la sovrapposizione delle trasparenze. I valori booleani dei fili non sono più rappresentati da chiavi, ma da due immagini: un'immagine tutta bianca codifica il valore 0, un'immagine tutta nera il valore 1. Le tabelle delle porte diventano collezioni di share visuali costruite in modo che, sovrapponendo le share giuste, riappaia l'immagine del valore di uscita corretto. La valutazione del circuito, che nella costruzione di Yao richiedeva un algoritmo di decifrazione, si riduce così a sovrapporre fogli trasparenti. Dove Yao valutava una porta decifrando la riga giusta della tabella, qui la si valuta scegliendo e sovrapponendo la metà giusta della share destra; l'informazione che dice quale metà è il pointer bit, che introdurremo tra poco. Come nella costruzione di Yao, anche la negazione resta gratuita: negare un filo equivale a scambiare le due immagini che lo codificano, quella del valore 0 e quella del valore 1, senza costruire alcuna porta. Ricostruiamo la costruzione dai suoi mattoni.

2.3.1 Un multi-secret sharing visuale

Il primo mattone della costruzione è uno schema di crittografia visuale che condivide due immagini invece di una. D'Arco e De Prisco lo chiamano Scheme-2-MVCS, dove MVCS sta per *multi-secret visual cryptography scheme*, e lo definiscono su un insieme di tre partecipanti $\mathcal{P} = \{1, 2, 3\}$. La scelta di tre partecipanti non è arbitraria: è esattamente ciò che serve alla costruzione delle porte, come sarà chiaro nella sottosezione successiva. Lo schema prende in ingresso due immagini segrete I_0 e I_1 e produce tre share sh_1, sh_2, sh_3 , con due strutture di accesso distinte,

$$\mathcal{A}_0 = \{\{1, 2\}\}, \quad \mathcal{A}_1 = \{\{1, 3\}\}, \quad \mathcal{F}_0 = \mathcal{F}_1 = \{\{1\}, \{2\}, \{3\}\} :$$

la coppia (sh_1, sh_2) ricostruisce I_0 , la coppia (sh_1, sh_3) ricostruisce I_1 , e nessuna share presa da sola rivela alcunché. La share sh_1 compare in entrambe le coalizioni qualificate ed è quindi il perno dello schema: la stessa trasparenza, sovrapposta a due share diverse, mostra due immagini diverse.

Della coppia (sh_2, sh_3) la definizione non dice nulla. Non si richiede che ricostruisca qualcosa, e neppure che non riveli nulla. È un'omissione voluta, non una svista: nel protocollo la coppia (sh_2, sh_3) non viene impiegata per la ricostruzione e non viene consegnata congiuntamente a Bob. Chiedere garanzie su una configurazione che non viene utilizzata avrebbe soltanto irrigidito lo schema.

Le due proprietà richieste sono quelle consuete, riformulate nel modello random grid introdotto nella Sezione 1.4.1. Per $h \in \{0, 1\}$ e per ogni posizione di pixel (i, j) :

- **correttezza:** se $I_h(i, j)$ è nero, allora $\text{Sup}(sh_1, sh_{2+h})(i, j)$ è nero; se $I_h(i, j)$ è bianco, allora $\Pr [\text{Sup}(sh_1, sh_{2+h})(i, j) \text{ bianco}] \geq \varepsilon$ per una costante $\varepsilon > 0$;
- **riservatezza:** per ogni colore σ ,

$$\Pr[sh_1(i, j) = \sigma] = \Pr[sh_2(i, j) = \sigma] = \Pr[sh_3(i, j) = \sigma],$$

indipendentemente dal valore dei pixel segreti.

Il nero si ricostruisce quindi in modo esatto, il bianco solo con probabilità non trascurabile. È l'asimmetria tipica del random grid, già discussa nel capitolo precedente, e sarà quella che nel protocollo distinguerà il valore 1, riconoscibile con certezza, dal

valore 0, riconoscibile per contrasto statistico.

La costruzione che realizza queste proprietà è essenziale, e riusa quasi per intero lo schema random grid $(2,2)$ di Kafri e Keren visto nel Capitolo 1. Siano I_0 e I_1 due immagini $t \times t$ in bianco e nero. Per ogni posizione (i, j) si estrae un bit casuale uniforme $r_{i,j}$ e lo si usa per scegliere una delle due matrici di distribuzione della collezione $\mathcal{C}_{I_0(i,j)}$ dello schema $(2,2)$ -KK, che fissa insieme $sh_1(i, j)$ e $sh_2(i, j)$. La coppia (sh_1, sh_2) è dunque, né più né meno, una condivisione $(2,2)$ -KK di I_0 . La terza share nasce invece per confronto: si guarda il pixel di I_1 e si copia sh_1 dove I_1 è bianca, la si complementa dove I_1 è nera. Il risultato è che anche (sh_1, sh_3) è una condivisione $(2,2)$ -KK, questa volta di I_1 , costruita però attorno a una share sh_1 che esisteva già. La procedura è riportata nell'Algoritmo 3.

Letta con l' $\oplus$ la costruzione si riassume in due righe: $sh_2 = sh_1 \oplus I_0$ e $sh_3 = sh_1 \oplus I_1$, con sh_1 estratta a caso. È la stessa struttura del One-Time Pad, con sh_1 nel ruolo della chiave e due messaggi cifrati sotto la medesima chiave. Nel caso del pad questo sarebbe un errore grave, perché lo $\oplus$ dei due cifrati rivela lo $\oplus$ dei messaggi. Anche in questa costruzione vale $sh_2 \oplus sh_3 = I_0 \oplus I_1$; tuttavia, la coppia $\{2, 3\}$ non appartiene né alle coalizioni qualificate né a quelle vietate definite dallo Scheme-2-MVCS e non viene utilizzata nella ricostruzione prevista dal protocollo.

Algorithm 3 Scheme-2-MVCS: condivisione di due immagini su tre share

```

1: procedure SHR( $I_0, I_1$ )
2:   for  $i, j = 1, \dots, t$  do
3:     scegli  $r_{i,j} \in \{0, 1\}$  uniformemente a caso
4:     usa la matrice  $r_{i,j}$  di  $\mathcal{C}_{I_0(i,j)}$  per  $sh_1(i, j)$  e  $sh_2(i, j) \triangleright (sh_1, sh_2)$ : condivisione
       KK di  $I_0$ 
5:     if  $I_1(i, j)$  è nero then
6:        $sh_3(i, j) \leftarrow$  complemento di  $sh_1(i, j)$ 
7:     else
8:        $sh_3(i, j) \leftarrow sh_1(i, j)$ 
9:     end if
10:  end for
11:  return  $(sh_1, sh_2, sh_3)$ 
12: end procedure

13: procedure REC'( $sh_i, sh_j$ )
14:  return Sup( $sh_i, sh_j$ )
15: end procedure

```

Che la costruzione soddisfi entrambe le proprietà si verifica pixel per pixel, con $\varepsilon = 1/2$; la dimostrazione è in [1].

Una precisazione sulle figure. Nel protocollo le immagini associate al filo di uscita sono quelle piene descritte all'inizio della sezione, una tutta bianca e una tutta nera. Negli esempi che illustrano i singoli schemi useremo invece due forme riconoscibili, un cerchio e una croce, perché così si vede a colpo d'occhio quale dei due segreti è stato ricostruito; le due immagini piene renderebbero le figure illeggibili. Nell'esempio completo della Sezione 2.5, dove il protocollo viene eseguito per intero, torneremo alle immagini piene.

Trattandosi di uno schema random grid non c'è espansione dei pixel: le tre share hanno la stessa dimensione dei segreti. È una proprietà che qui conta più che altrove, perché nel protocollo le share vengono condivise di nuovo a ogni livello del circuito, e un fattore di espansione anche piccolo diventerebbe insostenibile dopo poche porte.

Il prezzo è la ricostruzione approssimata delle aree bianche, visibile nella Figura 2.1: metà dei pixel bianchi riappare nera, e il bianco si legge come un grigio a scacchiera. Il nero, invece, è sempre esatto.

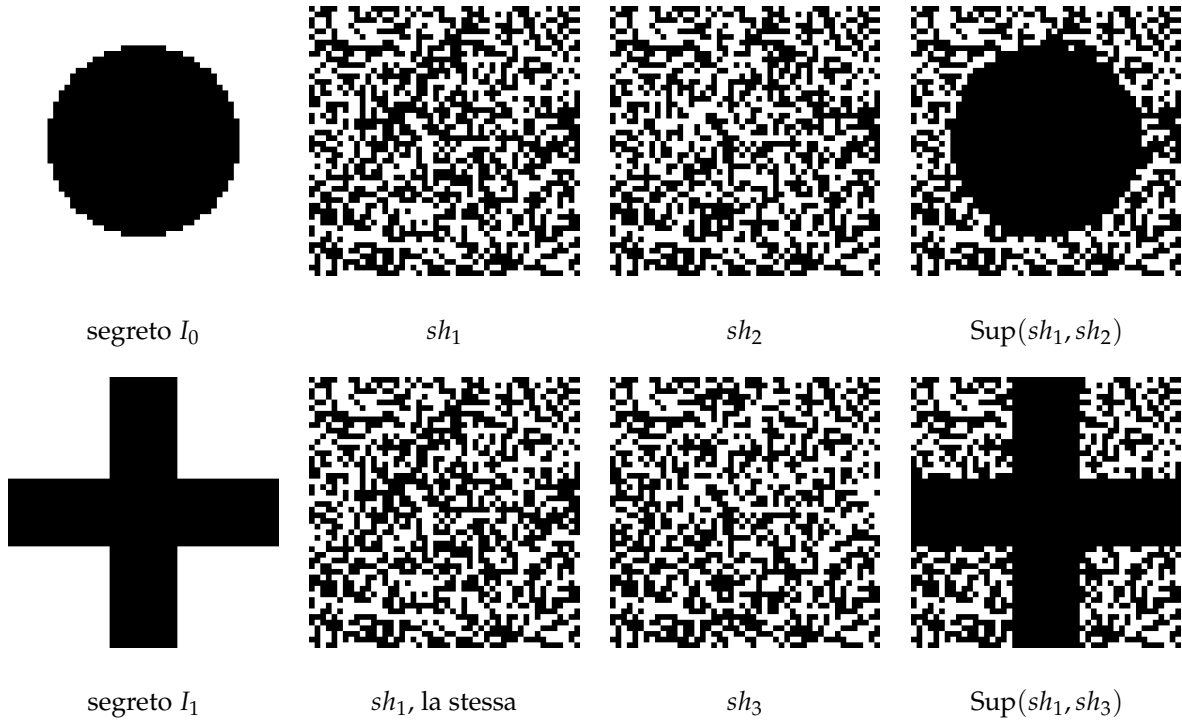


Figura 2.1: Scheme-2-MVCS su due segreti 48×48 . La stessa share sh_1 ricostruisce due immagini diverse a seconda di come viene accoppiata: sovrapposta a sh_2 mostra il cerchio, sovrapposta a sh_3 mostra la croce. Le tre share, prese singolarmente, sono indistinguibili dal rumore. Il nero si ricostruisce esatto, mentre le aree bianche appaiono grigie perché metà dei loro pixel viene resa nera. Le immagini sono state prodotte con l’implementazione riportata nell’Appendice B.1.

2.3.2 Dalla porta cifrata alla porta visuale

Il secondo mattone è l’analogo visuale della tabella cifrata di Yao: un oggetto che, date le share dei due fili di ingresso, lascia ricostruire l’immagine del filo di uscita e nient’altro. La nozione di riferimento è il *gate evaluation secret sharing* (GESS), introdotto da Kolesnikov nel 2005 [25]. Un GESS per una porta G condivide due segreti s_0 e s_1 in due coppie di share, una coppia per filo di ingresso. Chi valuta riceve un *selettore* $v = (v_1, v_2)$, cioè i valori dei due fili, e tiene la share v_1 -esima del filo sinistro e la v_2 -esima del filo destro. Da quelle due sole share deve poter ricostruire

$s_{G(v)}$, il segreto associato all'uscita della porta. La riservatezza si chiede in forma simulativa: esiste un simulatore che, ricevuto il solo $s_{G(v)}$, produce una coppia di share distribuita come quella vera. Chi valuta impara il valore di uscita, e sulle altre tre combinazioni di ingresso non impara nulla.

D'Arco e De Prisco osservano che un GESS si può costruire a partire da un qualunque schema di multi-secret sharing su tre parti, e ne danno una meta-costruzione che chiamano GenGESS. Servono due istanze indipendenti dello stesso schema, indicate con A e B , usate con casualità fresca. L'istanza A condivide la coppia $(s_{G(0,0)}, s_{G(0,1)})$, cioè le due uscite della porta quando il filo sinistro vale 0; l'istanza B condivide $(s_{G(1,0)}, s_{G(1,1)})$, le due uscite quando il filo sinistro vale 1. Ciascuna produce tre share, e per la proprietà del multi-secret sharing vale

$$\begin{aligned} \text{Rec}'(sh_1^A, sh_2^A) &= s_{G(0,0)}, & \text{Rec}'(sh_1^A, sh_3^A) &= s_{G(0,1)}, \\ \text{Rec}'(sh_1^B, sh_2^B) &= s_{G(1,0)}, & \text{Rec}'(sh_1^B, sh_3^B) &= s_{G(1,1)}, \end{aligned}$$

dove Rec' è la ricostruzione dello schema di multi-secret sharing sottostante, da non confondere con quella a livello di porta che vedremo fra poco.

La divisione del lavoro è quindi netta: il filo sinistro sceglie l'istanza, il filo destro sceglie quale dei due segreti di quell'istanza ricostruire. Le sei share prodotte vengono allora ridistribuite sui due fili. Al filo sinistro vanno le due share sh_1 , una per valore. Al filo destro va, per ciascun valore, la *concatenazione* delle share omologhe delle due istanze. Resta però un problema: se le due parti fossero sempre concatenate nello stesso ordine, la posizione della metà usata tradirebbe il valore del filo sinistro. Per questo si estrae un *bit di permutazione* $b \in \{0, 1\}$ uniforme, che decide l'ordine, e lo si combina con il valore del filo sinistro in un *pointer bit* premesso alla share sinistra:

$$p = v_1 \oplus b.$$

Il risultato è riassunto nella Tabella 2.1, in cui $\parallel$ denota la concatenazione.

filo sinistro	filo destro, se $b = 0$	filo destro, se $b = 1$
$sh_{1,0} = b \parallel sh_1^A$	$sh_{2,0} = sh_2^A \parallel sh_2^B$	$sh_{2,0} = sh_2^B \parallel sh_2^A$
$sh_{1,1} = \bar{b} \parallel sh_1^B$	$sh_{2,1} = sh_3^A \parallel sh_3^B$	$sh_{2,1} = sh_3^B \parallel sh_3^A$

Tabella 2.1: La costruzione GenGESS. Il bit di permutazione b fissa l'ordine delle due metà sul filo destro; il pointer bit premesso alla share sinistra vale b per il valore 0 e $\bar{b}$ per il valore 1, cioè $p = v_1 \oplus b$.

La ricostruzione legge il pointer bit p dalla share sinistra e lo usa come indice: se $p = 0$ prende la prima metà della share destra, se $p = 1$ la seconda, e ricostruisce con la share sinistra. Che il risultato sia sempre $s_{G(v_1, v_2)}$ si controlla sui quattro casi. Con $b = 0$ e $v_1 = 0$ il pointer bit vale 0, la prima metà è quella dell'istanza A , e la share sinistra è sh_1^A : le due si accoppiano correttamente. Con $v_1 = 1$ il pointer bit vale 1, la seconda metà è dell'istanza B e la share sinistra è sh_1^B . Con $b = 1$ le due metà sono scambiate e il pointer bit è complementato, quindi punta di nuovo all'istanza giusta. Il filo destro, dal canto suo, decide con v_2 se usare le share di indice 2 o quelle di indice 3, cioè quale dei due segreti dell'istanza selezionata far riapparire.

La costruzione di Kolesnikov è il caso particolare in cui il multi-secret sharing è realizzato con lo xor : le share sono $sh_1^A = R_0$, $sh_2^A = s_{G(0,0)} \oplus R_0$, $sh_3^A = s_{G(0,1)} \oplus R_0$ per stringhe casuali R_0 , e analogamente per B . Proprio questo è il motivo per cui la costruzione originale non si può rendere visuale: la sovrapposizione di trasparenze calcola un OR , e nessuno schema di crittografia visuale in cui la ricostruzione sia la sovrapposizione realizza lo xor .¹ GenGESS aggira l'ostacolo perché non nomina mai l'operazione di ricostruzione: chiede soltanto uno schema di multi-secret sharing su tre parti con le due strutture di accesso viste sopra. Lo Scheme-2-MVCS è esattamente questo, e istanziando GenGESS con esso si ottiene un *visual gate evaluation secret sharing* (VGESS), la versione visuale di una porta. I segreti diventano immagini e Rec' diventa Sup . Correttezza e riservatezza si trasferiscono senza modifiche: la prima

¹Esistono modelli visuali che ottengono lo xor ricorrendo a meccanismi diversi dalla sovrapposizione, per esempio la complementazione delle trasparenze o l'uso della polarizzazione della luce. Non li consideriamo qui, perché rinunciano proprio alla decodifica per semplice sovrapposizione su cui poggia tutto il protocollo.

nella versione probabilistica del random grid, con il nero esatto e il bianco corretto con probabilità almeno ε , la seconda nella stessa forma simulativa del GESS, perché il simulatore di GenGESS non sfrutta alcuna proprietà dello schema sottostante oltre a quelle richieste dalla definizione [1].

Vale la pena vedere il caso di una porta AND, che è anche l'esempio usato dagli autori. Qui $G(0,0) = G(0,1) = G(1,0) = 0$ e $G(1,1) = 1$, quindi l'istanza A condivide la coppia (I_0, I_0) e l'istanza B la coppia (I_0, I_1) . Tre delle quattro combinazioni ricostruiscono l'immagine del valore 0 e una sola quella del valore 1, ma le sei share restano indistinguibili fra loro: la tabella di verità della porta è interamente nascosta nella scelta di quali segreti dare in pasto alle due istanze. La Figura 2.2 mostra le share che ne risultano e la Figura 2.3 le quattro valutazioni corrispondenti.



Figura 2.2: Le quattro share prodotte dal VGESS di una porta AND con bit di permutazione $b = 0$. Nelle share del filo sinistro il pointer bit è il blocco 1×2 che sporge in basso a sinistra; quelle del filo destro sono larghe il doppio perché concatenano le parti delle due istanze A e B . Il reticolo evidenzia i singoli pixel logici senza riempire lo spazio trasparente sopra il pointer. Prese singolarmente, le share principali sono indistinguibili dal rumore.

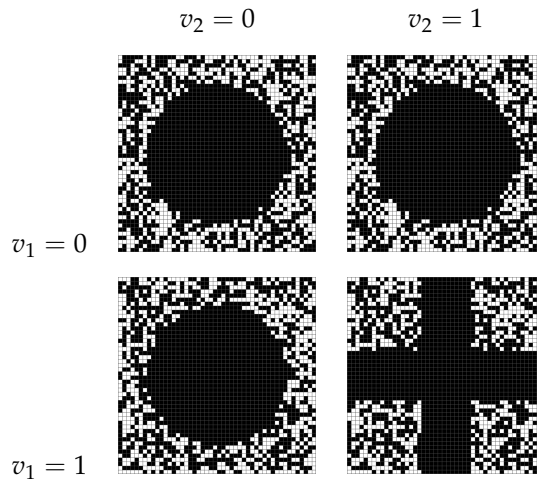


Figura 2.3: Le quattro valutazioni della stessa porta AND. Il cerchio è l'immagine I_0 che codifica il valore 0, la croce è I_1 che codifica il valore 1. Solo la combinazione $(1, 1)$ ricostruisce la croce, come deve fare una porta AND. Il nero riappare esatto, le aree bianche restano grigie per via del modello random grid.

2.3.3 Il pointer bit

Il pointer bit è l'unica informazione che, oltre alle immagini, serve durante la valutazione: dice quale delle due metà della share destra va ritagliata e sovrapposta. Deve quindi arrivare a chi valuta, ma senza tradire i valori dei fili. Sulla porta a cui si riferisce il problema non si pone, perché $p = v_1 \oplus b$ è mascherato dal bit di permutazione: osservare p senza conoscere b equivale a non osservare nulla. Chi valuta, leggendo $p = 0$, non sa se sta tenendo la prima o la seconda share del filo sinistro, e questo è quanto basta. Il pointer bit può perciò viaggiare in chiaro, premesso alla share sinistra della sua porta.

La difficoltà nasce un livello più su. Le share di un filo interno non sono oggetti finali: diventano a loro volta i segreti che la porta soprastante condivide. Se il pointer bit di quel filo restasse in chiaro anche lassù, chi valuta lo leggerebbe prima di avere entrambi gli ingressi della porta superiore, e da lì risalirebbe all'ordine delle sottoshare. Va quindi protetto. D'Arco e De Prisco lo fanno con uno schema separato da quello delle immagini: il pointer bit è un singolo pixel, bianco per 0 e nero per 1, e viene condiviso con lo schema deterministico $(2, 2)$ -NS visto nel Capitolo 1. La scelta del deterministico non è casuale. Sulle immagini l'errore probabilistico del random grid

è tollerabile, perché un'area si legge nel suo complesso; su un bit solo non lo sarebbe affatto, e la (2,2)-NS ricostruisce senza errore. Nella forma ricostruita il pointer bit occupa due sotto-pixel: uno solo nero per il valore 0, entrambi neri per il valore 1. È il peso a portare l'informazione, non la posizione del pixel nero. Sono i due pixel che compaiono all'estrema sinistra delle share del filo sinistro nella Figura 2.2.

Poiché una share attraversa più porte lungo la salita verso l'uscita, i pointer bit si accumulano. Ogni filo sinistro ne porta con sé una lista: quello della porta corrente, in chiaro, e quelli delle porte successive, presenti soltanto in forma di share. Il punto importante è che questi ultimi non vengono ricondivisi a ogni livello. Vengono soltanto ricopiati accanto alle share delle immagini, e accodati da destra verso sinistra man mano che si sale. Il numero di pointer bit su un filo cresce quindi di uno per livello, e ciascuno costa due pixel: la crescita che i pointer bit aggiungono di proprio è lineare. Il loro ingombro segue comunque il raddoppio delle immagini sui fili destri, perché ogni metà deve portarne una copia per poter essere ritagliata da sola.

Qui la costruzione si discosta da quella di Kolesnikov. Nell'originale i pointer bit del livello i fanno parte dei segreti condivisi per produrre le share dei livelli superiori, e seguono quindi la stessa sorte delle immagini. Gli autori avrebbero potuto fare lo stesso, e osservano che la scelta è stata dettata da una ragione pratica: tenendo i pointer bit fuori dai segreti si semplifica il taglio delle trasparenze che chi valuta deve ritagliare e sovrapporre. La variante resta sicura. Per ricostruire il pointer bit di una porta servono le share visuali di entrambi i fili di ingresso, e questa condizione si verifica soltanto in fase di valutazione, quando una delle due parti possiede già entrambi gli ingressi di quella porta e la sta valutando. Prima di quel momento il bit non è leggibile da nessuno.

Resta da dire come il pointer bit venga ricostruito, e non solo trasportato, quando appartiene a una porta interna. Il meccanismo è lo stesso GenGESS della Sezione 2.3.2, istanziato però con la (2,2)-NS invece che con lo Scheme-2-MVCS. Il filo di uscita di una porta ammette due pointer bit possibili, s_0 e s_1 , uno per ciascun valore dell'uscita, scritti nella forma ricostruita a due sotto-pixel. Due istanze A e B li condividono seguendo la tabella di verità della porta: A si occupa delle uscite con filo sinistro a 0, B di quelle con filo sinistro a 1. Le share sh_1 vanno al filo sinistro, le altre quattro si concatenano sul filo destro nello stesso ordine delle immagini principali, così che il

taglio della trasparenza sia uno solo e valga per entrambe.

Con una differenza rispetto alle immagini, che è la chiave di tutto il meccanismo: qui la share sh_1 è unica, la stessa per entrambi i valori del filo, come si vede nelle Figure 2.4 e 2.5, dove i due riquadri in alto a sinistra coincidono. È questa coincidenza a rendere possibile il trasporto descritto sopra. Se le due share sinistre fossero diverse, un pointer bit già condiviso non potrebbe essere ricopiato tale e quale sui livelli inferiori senza tradire il valore del filo che lo porta, e andrebbe ricondiviso a ogni passo, perdendo per strada proprio il pattern esatto che serve a ricostruirlo.

Due dettagli distinguono infine questo caso da quello delle immagini. Il primo è che la $(2,2)$ -NS è deterministica, quindi la corrispondenza è secca: la share sh_2 ripete sh_1 quando il segreto è bianco e la complementa quando è nero, e la sovrapposizione dà un blocco di peso uno oppure di peso due senza margine di errore. Il secondo è che s_0 e s_1 non rappresentano i valori 0 e 1 dell'uscita, ma i pointer bit corrispondenti, cioè $v \oplus b$ dove b è il bit di permutazione della porta *soprastante*, quella a cui il filo fa da ingresso sinistro. Cambiando b le due immagini si scambiano, e con esse tutte le share. Le due figure riportano i due casi per una porta AND e per una porta OR: fra i due tipi di porta cambiano due posizioni su quattro, quelle corrispondenti agli ingressi $(0,1)$ e $(1,0)$, per i quali differiscono le rispettive tabelle di verità.

Un'ultima osservazione riguarda i fili di ingresso del circuito. I loro pointer bit non vengono mai condivisi, perché sotto di essi non c'è nessuna porta, e basterebbe un pixel solo. Gli autori usano comunque la rappresentazione a due sotto-pixel, per non avere due convenzioni diverse nella stessa costruzione, e noi facciamo altrettanto.

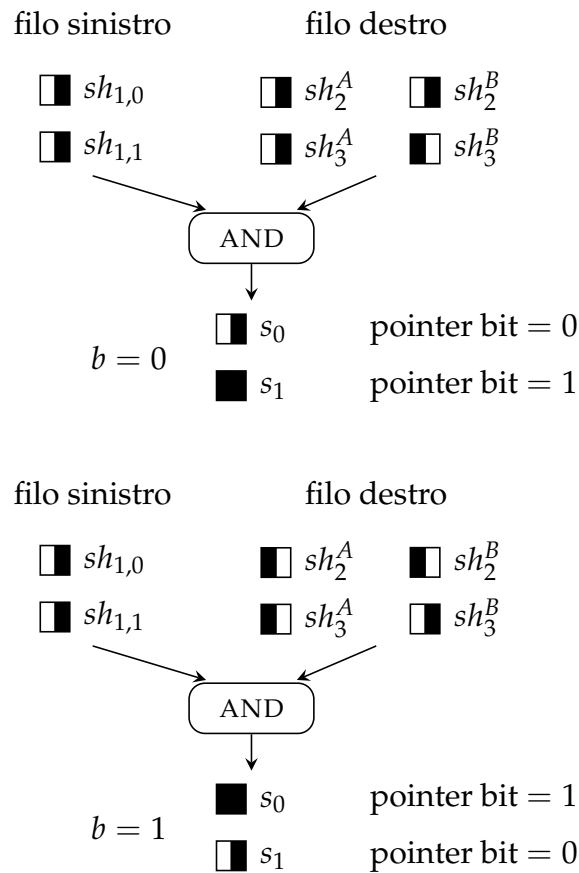


Figura 2.4: Condivisione in stile GESS del pointer bit di una porta AND, nei due casi $b = 0$ e $b = 1$, dove b è il bit di permutazione della porta soprastante. Ogni riquadro è una share (2,2)-NS da due sotto-pixel, e le due share del filo sinistro coincidono. Sovrapponendo sh_1 alla share destra della stessa istanza si ottiene un blocco con un solo pixel nero, cioè pointer bit 0, quando le due share sono uguali, e un blocco interamente nero, cioè pointer bit 1, quando sono complementari. Per $b = 0$ solo la combinazione (1,1) produce il pointer bit 1; per $b = 1$ i valori del pointer si invertono, poiché esso codifica l'uscita della porta XOR b . Ridisegnata a partire da [1], Fig. 5.

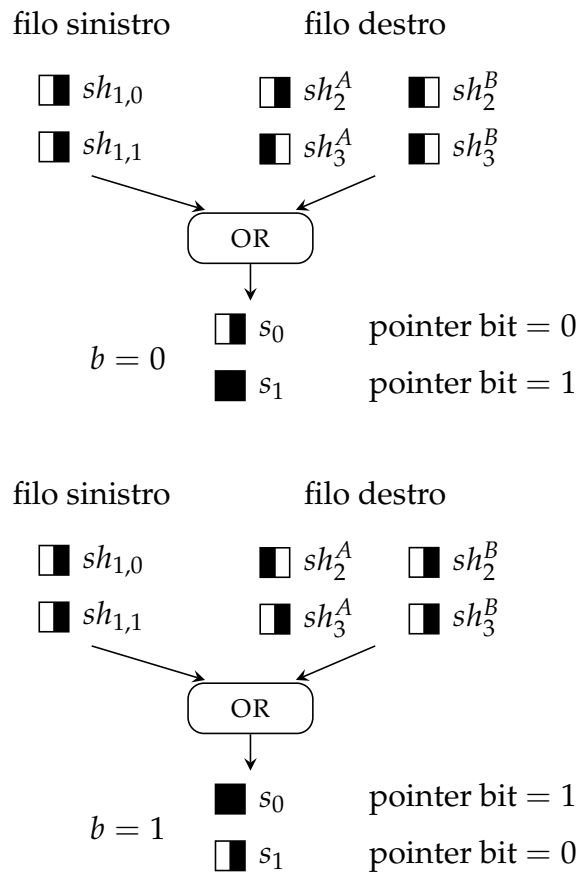


Figura 2.5: Lo stesso schema per una porta OR. Per $b = 0$ solo la combinazione $(0, 0)$ produce il pointer bit 0; per $b = 1$ i valori del pointer si invertono. A parità di b , rispetto alla porta AND cambiano due delle quattro share del filo destro, quelle associate agli ingressi $(0, 1)$ e $(1, 0)$. Ridisegnata a partire da [1], Fig. 6.

2.3.4 Un oblivious transfer fisico

Nella costruzione di Yao l’oblivious transfer è un protocollo crittografico, e la sua sicurezza poggia su ipotesi di difficoltà computazionale. Se il V2PC lo ereditasse così com’è, l’intero edificio resterebbe condizionato: la fase di valutazione sarebbe pure fisica, ma la consegna delle share a Bob richiederebbe di nuovo un computer e un’ipotesi non dimostrata. D’Arco e De Prisco chiudono la falla sostituendo anche questo pezzo con una procedura manuale.

Il compito da svolgere è quello di sempre. Alice possiede due trasparenze, z_0 e z_1 , che sono le due share del filo di ingresso di Bob, una per il valore 0 e una per il valore 1. Bob ha un bit σ e vuole ricevere z_σ . La funzionalità richiede due cose in una: Bob

non deve apprendere nulla di $z_{1-\sigma}$, e Alice non deve apprendere σ . In notazione, $f_{OT} : (z_0, z_1, \sigma) \mapsto (\perp, z_\sigma)$, dove $\perp$ segnala che Alice non riceve alcun output.

La realizzazione usa due buste e un tritacarte. L'idea è ripresa da una procedura di voto verificabile proposta da Chaum [26], dove l'elettore, uscendo dalla cabina, conserva una parte della ricevuta e distrugge l'altra davanti a un testimone: la parte trattenuta prova che il voto è stato registrato, quella distrutta impedisce di dimostrare a terzi come si è votato. Qui Bob compra due buste opache identiche, capaci di nascondere del tutto una trasparenza, e le consegna ad Alice. È lui a comprarle, non lei, così che non possano essere state marcate. Alice dà le spalle a Bob, infila una trasparenza in ciascuna busta, le chiude e vi appunta due foglietti adesivi, con il numero 0 sulla busta che contiene z_0 e il numero 1 sull'altra. Restituisce poi entrambe le buste a Bob. A questo punto Bob volta a sua volta le spalle ad Alice, tiene la busta con il foglietto che corrisponde al proprio bit, e dall'altra stacca il foglietto. Si gira di nuovo verso Alice e, sotto i suoi occhi, infila la busta rimasta in un tritacarte che riduce carta e contenuto in polvere. La procedura è riassunta nell'Algoritmo 4 e illustrata nella Figura 2.6.

Il gesto di staccare il foglietto è la parte meno ovvia e la più importante. Alice deve assistere alla distruzione, altrimenti nulla impedirebbe a Bob di conservare entrambe le buste e di leggere anche la share che non gli spetta. Ma se la busta che finisce nel tritacarte portasse ancora il suo numero, Alice leggerebbe quel numero e saprebbe quale busta Bob ha scartato, e quindi quale ha tenuto. Senza foglietto la busta eliminata è anonima, e Alice vede una distruzione di cui non può dire nulla. Le due esigenze, che sembrano opposte, si conciliano così.

Algorithm 4 Oblivious transfer 1-su-2 fisico

- 1: Bob compra due buste opache indistinguibili e le consegna ad Alice
 - 2: Alice, di spalle a Bob, inserisce z_0 e z_1 in una busta ciascuna e le chiude
 - 3: Alice appunta su ogni busta un foglietto con il numero della share contenuta, poi restituisce entrambe le buste
 - 4: Bob, di spalle ad Alice, trattiene la busta con il foglietto σ e stacca il foglietto dall'altra
 - 5: Bob distrugge la busta rimasta nel tritacarte, davanti ad Alice
-

Sotto l'ipotesi che le buste siano davvero indistinguibili e davvero opache, il protocollo realizza un oblivious transfer 1-su-2 perfettamente sicuro. La correttezza è immediata. Per la riservatezza servono due simulatori. Quello di Alice è banale, perché Alice non riceve nulla da Bob e la sua view si esaurisce nel proprio input. Quello di Bob si costruisce così: ricevuta z_σ dalla funzionalità ideale, il simulatore compra due buste e due foglietti, scrive 0 e 1 e li associa alle buste a caso, infila z_σ nella busta con il numero σ e una trasparenza vuota nell'altra. La view che ne risulta ha la stessa distribuzione di quella reale, perché in quella reale l'unica cosa che Bob osserva è una busta chiusa, e una busta chiusa opaca non dice nulla del proprio contenuto.

Vale la pena essere espliciti su cosa regge questa garanzia. La sicurezza è perfetta nel senso teorico-informativo, cioè vale anche contro un avversario con potenza di calcolo illimitata, ma è appoggiata a due ipotesi sul mondo fisico invece che a un problema difficile. La prima è che una busta chiusa non lasci trapelare il proprio contenuto, nemmeno in controluce. La seconda è che la distruzione sia irreversibile, che dalla polvere non si possa ricostruire nulla, neppure parzialmente. Gli autori osservano che il tritacarte si può sostituire con il fuoco, purché la proprietà resti quella. Sono ipotesi diverse in natura da quelle crittografiche, non necessariamente più deboli, e sono comunque verificabili a occhio dalle parti stesse, che è poi lo spirito dell'intera costruzione.

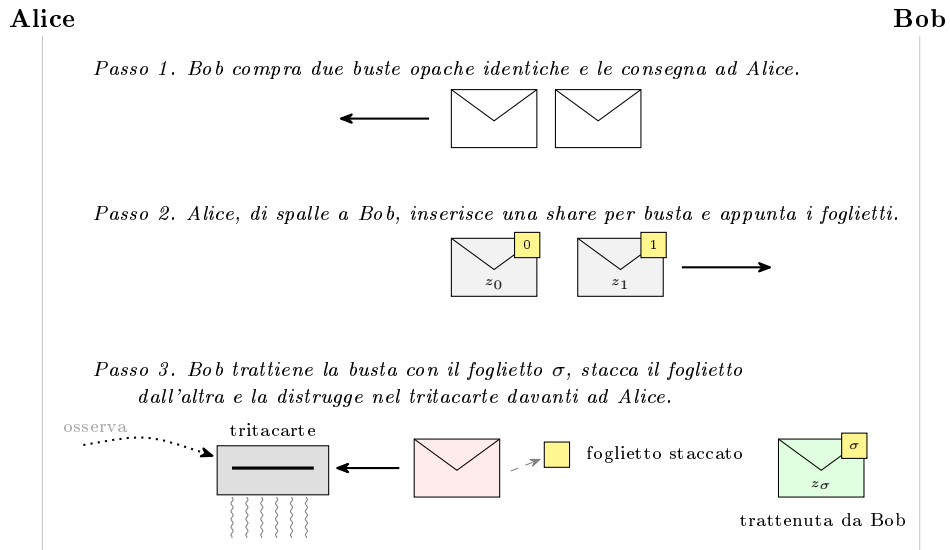


Figura 2.6: I tre passi dell'oblivious transfer fisico. Il foglietto viene staccato dalla busta scartata prima della distruzione, così che Alice possa verificare che la busta sia stata eliminata senza poter dedurre quale delle due Bob abbia tenuto.

2.3.5 Le due fasi del protocollo

Con i mattoni a disposizione il protocollo si monta in fretta, e mantiene la struttura in due tempi della costruzione di Yao. Fissiamo la notazione. La funzione da calcolare è $f : \{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}$, e C è un circuito booleano che la calcola. I fili di ingresso sono $2n$, etichettati $\omega_1, \dots, \omega_{2n}$: da ω_1 a ω_n portano i bit di Alice, da ω_{n+1} a ω_{2n} quelli di Bob. Le porte sono ℓ . Come già detto, ogni porta ha un solo filo di uscita e ogni filo alimenta al più una porta; il filo di uscita del circuito, in particolare, non entra in nessuna porta.

La prima fase costruisce le trasparenze e la svolge Alice da sola. È l'unico momento in cui serve un calcolatore. Alice parte dall'alto: associa al filo di uscita del circuito le due immagini I_0 e I_1 , quella tutta bianca e quella tutta nera, e poi scende. Per ogni porta il cui filo di uscita ha già ricevuto una coppia di immagini, esegue l'algoritmo Shr del VGESS che realizza quella porta, e ottiene quattro share: due per il filo di ingresso sinistro, una per valore, e due per il filo destro. Queste quattro share diventano le immagini associate ai due fili di ingresso, e il procedimento riparte da lì. Il punto da tenere a mente è che le immagini di un filo interno non sono figure disegnate da

qualcuno: sono share prodotte dalla porta che sta sopra. Soltanto il filo di uscita del circuito porta le due immagini piene.

La direzione conta. La costruzione scende dall'uscita verso gli ingressi, mentre la valutazione risalirà dagli ingressi verso l'uscita. È lo stesso rovesciamento che si ha nei circuiti garbled, e ha una conseguenza pratica: Alice deve conoscere l'intero circuito prima di cominciare, e non può costruire una porta senza aver costruito tutte quelle che le stanno sopra.

Ai pointer bit si pensa nello stesso giro. Se la porta in esame ne ha, cioè in tutti i casi tranne quello della porta di uscita, i pointer bit delle due immagini vengono codificati con le share dello schema $(2,2)$ -NS nel modo descritto nella Sezione 2.3.3, e le share risultanti si premettono a quelle delle immagini sui quattro fili. Quelle del livello corrente si accodano a quelle che arrivano dai livelli precedenti, senza essere ricondivise. Alla fine della discesa Alice possiede, per ciascuno dei $2n$ fili di ingresso, due trasparenze: una per il valore 0 e una per il valore 1.

La seconda fase è il calcolo vero e proprio, e qui il computer sparisce. Alice consegna a Bob, per ciascuno dei propri n fili, la sola trasparenza che corrisponde al bit che ha scelto: se $x_j = 1$ gli dà la share del valore 1 e tiene per sé l'altra. Per i fili di Bob si procede invece con l'oblivious transfer fisico della Sezione 2.3.4, ripetuto n volte, una per bit. Al termine Bob ha esattamente una trasparenza per ogni filo di ingresso, e non sa nulla delle altre; Alice, dal canto suo, non sa quali abbia ricevuto.

Bob valuta il circuito livello per livello, dal basso verso l'alto. Per ogni porta legge il pointer bit sulla share del filo sinistro, ritaglia con le forbici la metà indicata della share del filo destro e la sovrappone alla share sinistra. L'altra metà si può gettare via, e gli autori la presentano come una comodità pratica: sull'immagine principale le due metà vengono da istanze indipendenti dello schema, quindi conservarla non aggiungerebbe niente a chi la tenesse. Quello che ottiene è la share del filo di uscita di quella porta, che diventa l'ingresso della porta successiva. Salendo di livello in livello arriva alla porta di uscita, dove la sovrapposizione produce un'immagine tutta nera se $f(x, y) = 1$, oppure un'immagine con dei pixel bianchi sparsi se $f(x, y) = 0$. La procedura completa è nell'Algoritmo 5.

Due cautele accompagnano quest'ultima parte, e sono di nuovo fisiche più che matematiche. La prima è che Alice non deve assistere alla ricostruzione: le immagini

che compaiono lungo la salita dipendono dalle share che Bob ha ricevuto, e quindi dal suo input. La seconda riguarda il finale. Bob comunica ad Alice il risultato, cioè un solo bit, e non le mostra l'immagine. Consegnare l'immagine sarebbe diverso dal dire il risultato: la disposizione precisa dei pixel neri e bianchi dipende da quali share sono state sovrapposte lungo il cammino, e Alice, che le conosce tutte, potrebbe leggerci dentro il percorso seguito nel circuito e da lì risalire ai bit di Bob.

Algorithm 5 Il protocollo V2PC

```

1: procedure COSTRUZIONESHARE( $C$ )           ▷ svolta da Alice, con un calcolatore
2:   associa al filo di uscita del circuito le immagini  $I_0$  e  $I_1$ 
3:   for ogni porta  $G$ , in un ordine tale che ogni porta segua la propria porta
     soprastante do
4:     siano  $\omega_i, \omega_j$  i fili di ingresso di  $G$  e  $I_0, I_1$  le immagini del suo filo di uscita
5:     esegui  $\text{Shr}(I_0, I_1)$  del VGESS di  $G$ 
6:     assegna le due share ottenute a  $\omega_i$  e le altre due a  $\omega_j$ 
7:     if i pointer bit sono presenti then
8:       condividili con lo schema  $(2, 2)$ -NS in stile GESS
9:       premetti le share ottenute a quelle delle immagini
10:      accodale a quelle dei livelli precedenti
11:    end if
12:  end for
13:  return le due share di ciascun filo  $\omega_1, \dots, \omega_{2n}$ 
14: end procedure

15: procedure CALCOLO( $x, y$ )                 ▷ svolta da Alice e Bob, senza calcolatore
16:   for  $j = 1, \dots, n$  do
17:     Alice consegna a Bob la share di  $\omega_j$  relativa a  $x_j$ 
18:   end for
19:   for  $j = 1, \dots, n$  do
20:     Alice e Bob eseguono l'OT fisico sulle due share di  $\omega_{n+j}$ , con bit di scelta
        $y_j$ 
21:   end for
22:   for ogni porta  $G$ , in un ordine tale che ogni porta segua i propri figli do
23:      $p \leftarrow$  pointer bit letto sulla share del filo sinistro
24:     ritaglia la metà  $p$  della share del filo destro
25:     sovrapponi le due trasparenze
26:   end for
27:   Bob legge  $I_{f(x,y)}$  e comunica ad Alice il solo risultato
28: end procedure

```

2.4 Dimensione delle share e qualità della ricostruzione

Prima di vedere il protocollo all'opera conviene affrontare la domanda che il lettore si sta ponendo da qualche pagina: quanto diventano grandi queste trasparenze? La risposta non è incoraggiante, e gli autori la discutono apertamente.

Il problema nasce dalla concatenazione. A ogni porta la share del filo destro è formata da due parti, quindi è larga il doppio delle immagini che riceve in ingresso. Salendo di livello il fattore due si moltiplica, e la dimensione delle share dei fili di ingresso cresce in modo esponenziale nella profondità del circuito. Non è un difetto di questa particolare costruzione: Kolesnikov ha mostrato che, riducendo la valutazione di funzione all'oblivious transfer con sicurezza perfetta e schemi GESS, non si può fare sostanzialmente meglio [25].² Il V2PC è quindi utilizzabile solo su circuiti poco profondi, e questo è un limite strutturale più che un dettaglio implementativo.

Fissiamo la notazione per la profondità, perché si presta a equivoci. Indichiamo con d , come fanno gli autori, il numero di passi da porta a porta lungo il cammino più lungo del circuito, cioè uno in meno del numero di livelli di porte. Una share che parte da un filo di ingresso viene quindi sovrapposta $d + 1$ volte prima di arrivare all'uscita. Nel circuito dell'esempio della Sezione 2.5, che ha tre livelli di porte, si ha $d = 2$.

Si capisce ora perché la costruzione usa il random grid e non uno schema deterministico. Con un deterministico i valori 0 e 1 si potrebbero rappresentare con un pixel solo, il che sembrerebbe un vantaggio. Ma l'espansione dei pixel moltiplicherebbe la dimensione delle share per un fattore m a ogni livello, e per non deformare l'immagine occorrerebbe espandere in un quadrato $m \times m$. Con lo schema (2,2)-NS le share dei fili di ingresso della porta di uscita sarebbero 2×2 , quelle del livello sotto 4×4 , poi 8×8 , e così via: esponenziale nella profondità anche questa, ma in aggiunta al raddoppio della concatenazione. C'è poi un problema pratico che gli autori sottolineano e che si apprezza solo pensando alle trasparenze vere: immagini di dimensione diversa a ogni livello vanno ritagliate e impacchettate in modi diversi, mentre un'unità di share di dimensione fissa rende tutti i tagli uguali. Il random grid

²Con un'eccezione che gli stessi autori segnalano: una terza costruzione di [25] scende sotto quel limite sfruttando l'informazione comune fra i segreti.

dà esattamente questo, perché non espande. Il raddoppio sul filo destro resta, ma è l'unica crescita in gioco.

Resta la contropartita, cioè l'errore. La correttezza del VGESS chiede che il nero si ricostruisca esatto e che l'area bianca conservi, con buona probabilità, almeno un pixel bianco. Serve almeno un pixel bianco perché è quello che permette di distinguere a occhio l'uscita 0 dall'uscita 1: il nero pieno significa 1, qualunque traccia di bianco significa 0. Il punto delicato è che ogni sovrapposizione può soltanto aggiungere pixel neri, mai toglierne. Più il circuito è profondo, più sovrapposizioni si accumulano, e più è probabile che l'area bianca venga coperta per intero, facendo leggere 1 dove il risultato è 0.

Il conto degli autori è immediato. Ogni pixel di una share è nero o bianco con probabilità $1/2$, e le sovrapposizioni lungo il cammino sono $d + 1$, quindi un dato pixel dell'area bianca sopravvive bianco con probabilità $(1/2)^{d+1}$. Trattando i t^2 pixel di un'immagine $t \times t$ come indipendenti,³

$$\Pr[\text{almeno un pixel bianco}] = 1 - \left(1 - \frac{1}{2^{d+1}}\right)^{t^2}.$$

Fissata una probabilità di errore tollerata ε , la condizione da imporre è dunque $\alpha^{t^2} < \varepsilon$ con $\alpha = 1 - 2^{-(d+1)}$, cioè

$$t \geq \sqrt{\log_{\alpha} \varepsilon}.$$

Nell'esempio della sezione successiva si ha $d = 2$, e scegliendo un margine largo, $\varepsilon = 0,0002$, la disuguaglianza dà $t \geq 7,99$: da qui la scelta $t = 8$ fatta dagli autori [6, Sez. 4.4]. La Tabella 2.2 mostra come cresce t al crescere della profondità, a parità di ε .

³L'indipendenza fra i t^2 pixel è un'ipotesi semplificativa adottata dagli autori, che manteniamo qui.

d	t minimo	t^2	larghezza della share destra
1	6	36	24
2	8	64	64
3	12	144	192
4	17	289	544
5	24	576	1536
6	33	1089	4224
7	47	2209	12032
8	67	4489	34304

Tabella 2.2: Lato minimo t dell'unità di share in funzione della profondità d , con probabilità di errore tollerata $\varepsilon = 0,0002$. L'ultima colonna riporta, in pixel, la larghezza della share del filo destro al livello più basso, pari a $2^{d+1}t$. Le due crescite si sommano: l'area dell'unità va all'incirca come $2^{d+1} \ln(1/\varepsilon)$, e la concatenazione aggiunge un altro fattore 2^{d+1} .

La tabella dice più di quanto dica la formula. A profondità 2 una share sta in un quadrato di poche decine di pixel per lato ed è maneggiabile; a profondità 8 l'unità è già 67×67 e la share del filo destro al livello più basso supera i trentamila pixel di larghezza. Con una risoluzione di stampa ragionevole si tratta di trasparenze lunghe metri. È in questo senso che il protocollo va inteso come uno strumento per funzioni semplici, non come una soluzione generale.

La stima appena riportata è quella degli autori, e conta le sovrapposizioni lungo un solo cammino. Le misure della presente implementazione indicano che, nei circuiti ramificati esaminati, il rumore proveniente dai due sottoalberi si accumula e l'esponente osservato risulta compatibile con il numero di porte del circuito. La Sezione 3.9.2 sviluppa questo modello alternativo e lo confronta con i dati; i valori della Tabella 2.2 vanno letti alla luce di tale confronto.

Per il pointer bit, come già osservato, la situazione è più comoda. Quel bit è locale alla singola porta e non viene ricondiviso a ogni livello, quindi non contribuisce di suo alla crescita esponenziale. E poiché è condiviso con uno schema deterministico,

sulla sua ricostruzione non pesa alcuna probabilità di errore: un solo pixel letto male manderebbe Bob a ritagliare la metà sbagliata della share destra, e tutto il resto della valutazione con lui.

2.5 Un esempio completo

D’Arco e De Prisco illustrano il protocollo su una funzione a sei ingressi,

$$f((x_1, x_2, x_3), (y_1, y_2, y_3)) = (x_1 \vee y_1) \wedge ((x_2 \wedge y_2) \wedge (x_3 \vee y_3)),$$

dove (x_1, x_2, x_3) è l’input privato di Alice e (y_1, y_2, y_3) quello di Bob. Il circuito corrispondente è nella Figura 2.7. Le porte sono numerate come in un albero binario completo, con la porta di uscita in posizione 1 e i figli della porta k in posizione $2k$ e $2k + 1$: per questo mancano i numeri 4 e 5, che spetterebbero ai figli di G_2 , che invece ha due foglie. Il circuito ha cinque porte disposte su tre livelli, quindi $\ell = 5$ e $d = 2$.

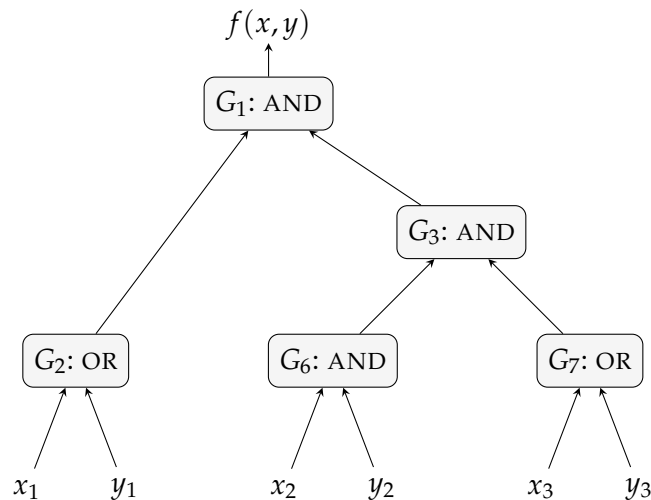


Figura 2.7: Il circuito della funzione dell’esempio. Il figlio sinistro di G_1 è G_2 , quello destro è G_3 ; per G_3 i figli sono G_6 a sinistra e G_7 a destra. In ciascuna coppia di foglie il filo di Alice è quello sinistro.

Alice comincia dal filo di uscita, a cui associa le due immagini piene I_0 e I_1 di lato t , e scende. Per G_1 costruisce il VGESS con lo Scheme-2-MVCS: essendo G_1 una porta AND, tre delle quattro combinazioni ricostruiscono I_0 e solo $(1, 1)$ ricostruisce I_1 . Passa poi a G_2 e G_3 , prendendo come segreti le share appena prodotte, e infine a G_6 e G_7 . Al

termine le immagini intermedie non servono più: contano solo le dodici trasparenze dei sei fili di ingresso, due per filo.

Il conto delle dimensioni si legge bene su questo esempio. Con $t = 8$, come scelto dagli autori, la larghezza delle share dei fili di ingresso va da 12 a 64 pixel, a fronte di un'altezza di 8 per tutte. Si vede all'opera sia il raddoppio del filo destro, sia l'accumulo dei pointer bit: x_1 , filo sinistro della porta più a sinistra, porta due pointer bit e un'immagine e sta in 12 pixel; y_3 , filo destro della porta più a destra, ha subito due raddoppi e ne occupa 64. La Figura 2.8 le riproduce tutte nella stessa scala, così che le proporzioni si possano confrontare.

Si passa al calcolo. Con l'input $x = (0, 1, 1)$ e $y = (1, 1, 0)$ Alice scarta, per ciascuno dei propri fili, la trasparenza che non le serve e consegna a Bob le altre tre; per i fili di Bob le due parti eseguono tre volte l'oblivious transfer con le buste. Bob si ritrova le sei trasparenze della Figura 2.8 e valuta dal basso: prima G_2 , G_6 e G_7 , che dipendono solo dagli ingressi, poi G_3 , infine G_1 . A ogni porta legge il pointer bit, taglia la share destra e sovrappone. La Figura 2.9 mostra i cinque risultati intermedi. Poiché $G_2 = 0 \vee 1 = 1$, $G_6 = 1 \wedge 1 = 1$ e $G_7 = 1 \vee 0 = 1$, si ha $G_3 = 1$ e quindi $f = 1$: l'immagine finale è interamente nera, come deve essere.

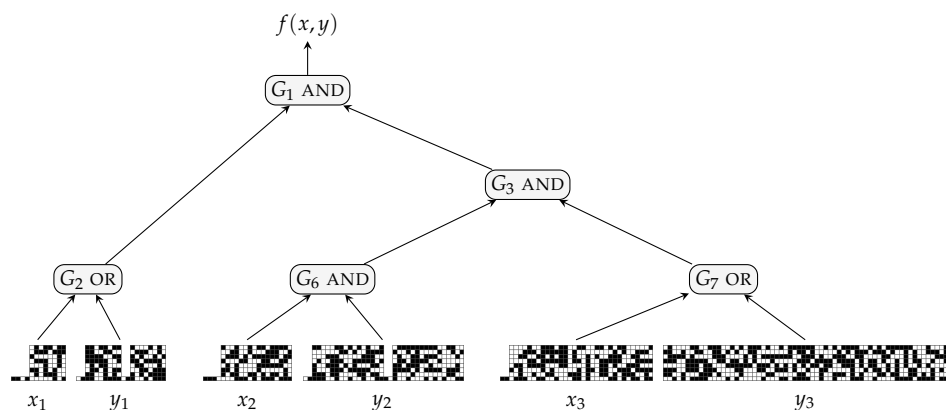


Figura 2.8: Le sei trasparenze in possesso di Bob dopo la consegna e i tre oblivious transfer, disposte sotto la porta a cui appartengono. L'input è $x = (0, 1, 1)$, $y = (1, 1, 0)$. Sono riprodotte tutte nella stessa scala: si vede crescere la larghezza da sinistra a destra, dai 12 pixel di x_1 ai 64 di y_3 . I blocchi 1×2 in basso rappresentano i pointer bit delle porte correnti e le share dei pointer bit, protetti mediante lo schema deterministico $(2, 2)$, che saranno necessari nelle valutazioni successive.

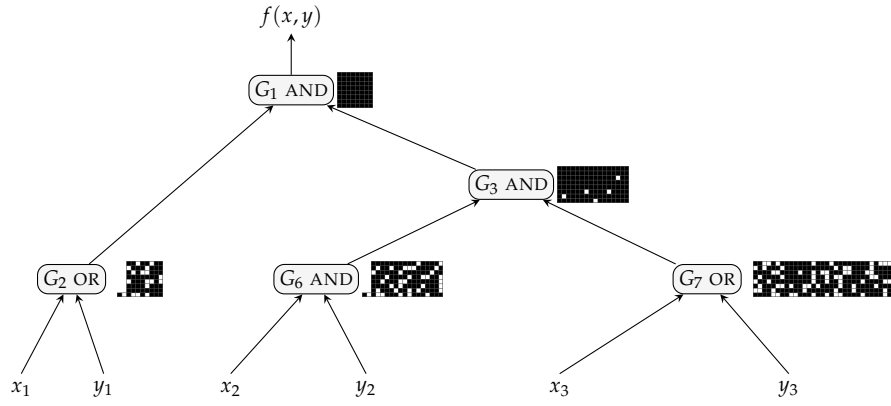


Figura 2.9: La trasparenza che Bob ottiene a ciascuna porta, accanto alla porta che la produce.

Ogni sovrapposizione consuma il pointer della porta corrente e propaga quelli ancora necessari; la larghezza delle cinque uscite è rispettivamente 10, 18, 32, 16 e 8 pixel. Sul filo di uscita resta la sola immagine 8×8 , qui interamente nera perché $f(x, y) = 1$.

2.6 Sicurezza del protocollo

La sicurezza del V2PC si dimostra nel modello semi-onesto e statico introdotto all’inizio del capitolo, con la strategia modulare consueta nel calcolo sicuro: prima si prova il protocollo assumendo che l’oblivious transfer sia dato per buono, poi si sostituisce l’assunzione con la realizzazione vera.

Il primo passo si svolge nel cosiddetto modello *ibrido*, in cui le due parti hanno accesso a un oracolo che realizza f_{OT} in modo ideale: sottopongono i propri input e ricevono il risultato, senza che nulla trapeli. In questo modello si mostra che il V2PC calcola f in modo perfettamente sicuro.

La correttezza si prova per induzione sulla profondità del circuito. Per un circuito con una sola porta l’affermazione coincide con la correttezza del VGESS che realizza quella porta: l’immagine ricostruita rappresenta il risultato della porta sui valori associati alle immagini dei fili di ingresso. Supponiamo allora che il protocollo sia corretto su tutti i circuiti di profondità $d - 1$ e consideriamo un circuito di profondità d . Le immagini che arrivano ai fili di ingresso della porta al livello d rappresentano, per ipotesi induttiva, i valori corretti dei fili sottostanti; applicando a quella porta la correttezza del suo VGESS si ottiene che anche l’immagine finale rappresenta il

valore corretto dell'uscita del circuito.

La riservatezza si prova esibendo, per ciascuna delle due parti, un simulatore. La view di Alice si esaurisce nel proprio input, in tutte le share che ha costruito e nel risultato che Bob le comunica: sono tutte cose che Alice ha generato da sé o che le spettano, quindi il simulatore le riproduce senza difficoltà. Il caso di Bob è quello interessante. La sua view contiene il proprio input, le trasparenze ricevute per i $2n$ fili di ingresso e tutte le immagini che compaiono durante la valutazione, fino a quella finale. Il simulatore Sim_{Bob} riceve soltanto y e l'immagine del risultato, $I_{f(x,y)}$, e procede dall'alto verso il basso: per la porta di uscita invoca il simulatore del VGESS su $I_{f(x,y)}$, ottenendo una coppia di share distribuita come quella vera; assegna quelle share ai fili di ingresso della porta e ripete, scendendo di livello in livello, fino ai fili di ingresso del circuito. Poiché ogni invocazione produce share con la distribuzione giusta, e poiché il pointer bit in chiaro è mascherato dal bit di permutazione, la view simulata coincide con quella reale. È lo stesso argomento con cui Kolesnikov prova la sicurezza della propria costruzione, con i simulatori dei GESS sostituiti da quelli dei VGESS e con l'aggiunta delle share dei pointer bit [25, 1].

Il secondo passo sostituisce l'oracolo ideale con l'oblivious transfer fisico della Sezione 2.3.4. Il teorema di composizione modulare garantisce che, se un protocollo è sicuro nel modello ibrido e si rimpiazzano le chiamate all'oracolo con esecuzioni di un protocollo che realizza in modo sicuro la stessa funzionalità, la sicurezza si conserva [22]. Poiché l'OT con le buste è perfettamente sicuro, il risultato complessivo è il seguente: dato un oblivious transfer con sicurezza incondizionata, il V2PC calcola f in modo perfettamente sicuro contro un avversario statico e semi-onesto. La dimostrazione completa, con i simulatori scritti per esteso, è in [1].

Il risultato è forte, e vale la pena dire in che senso. La garanzia è teorico-informativa: non poggia su alcun problema computazionale difficile e resiste quindi anche a un avversario con risorse illimitate. È la stessa garanzia della crittografia visuale discussa nel capitolo precedente, e nasce dallo stesso meccanismo: ogni singola share, presa da sola, ha un aspetto statisticamente indipendente dal valore che rappresenta, esattamente come nel One-Time Pad il testo cifrato è indipendente dal messaggio. La privacy dello Scheme-2-MVCS assicura che una share isolata non riveli nulla, e la scelta uniforme del bit di permutazione fa sì che neppure il pointer bit tradisca il

valore di un filo.

Un'ultima nota riguarda la composizione. L'analisi appena descritta considera una singola esecuzione isolata, quello che si chiama modello *stand-alone*. Gli autori osservano però che il simulatore usato è *straight-line* e *black-box*, cioè non torna indietro sui propri passi e non ispeziona il codice dell'avversario, e che per protocolli con questa proprietà la sicurezza perfetta nel modello stand-alone implica la sicurezza sotto composizione concorrente generale [1]. Il V2PC si può quindi eseguire in parallelo ad altri protocolli, o come passo interno di una procedura più ampia, senza che le sue garanzie decadano.

2.7 Proprietà e limiti

Il pregio più evidente del V2PC è che la fase di valutazione non richiede alcun calcolatore. Il computer serve soltanto ad Alice, e una sola volta, per generare le trasparenze; da quel momento in poi Alice e Bob calcolano la funzione con forbici e fogli trasparenti, senza chiavi crittografiche, senza algoritmi di cifratura da eseguire durante la fase di valutazione. La ricostruzione avviene mediante la sovrapposizione delle trasparenze e non richiede conoscenze crittografiche. La sicurezza è teorico-informativa nel modello statico e semi-onesto, sotto le ipotesi fisiche richieste dall'oblivious transfer con le buste. Sono proprietà che rendono il protocollo interessante proprio là dove un computer non può essere usato, o dove le persone non si fidano di affidargli i propri dati. Gli autori osservano anche che per casi molto piccoli il computer si potrebbe evitare del tutto, preparando le trasparenze a mano.

I limiti derivano dalla stessa struttura che rende possibile il protocollo, e conviene metterli in fila.

Il più serio è la dimensione delle share. A ogni livello la share del filo destro raddoppia, quindi cresce esponenzialmente nella profondità del circuito, e a questa crescita si somma quella dell'unità di share, che deve essere abbastanza grande da rendere trascurabile l'errore di lettura e che cresce anch'essa con la profondità. Il protocollo resta praticabile su circuiti di profondità ridotta, ed è in questo intervallo che va collocato.

Segue il numero di trasparenze da maneggiare, che cresce con il numero di fili di

ingresso, e con esso la difficoltà pratica di allinearle, ritagliarle e sovrapporle senza sbagliare. È un limite che non compare in nessuna formula ma che si sente appena si prova a eseguire il protocollo davvero.

C'è poi l'asimmetria della lettura. Il valore 1 si riconosce con certezza, perché l'immagine è nera senza eccezioni; il valore 0 si riconosce solo per la presenza di qualche pixel bianco, e con probabilità che dipende dalla dimensione scelta. Un errore di lettura, quando capita, è sempre nella stessa direzione: si legge 1 dove il risultato era 0, mai il contrario.

La riservatezza, infine, non riposa soltanto sulla costruzione matematica ma anche su gesti fisici che vanno compiuti davvero: le buste devono essere opache, la busta scartata deve essere distrutta in modo irreversibile, Alice non deve osservare la ricostruzione e Bob deve limitarsi a comunicare il bit del risultato. Nessuno di questi passaggi è verificabile a posteriori, e tutti dipendono dalla buona fede delle parti, il che si accorda con il modello semi-onesto ma non andrebbe oltre.

Proprio il modello di avversario è l'ultimo limite. La costruzione è dimostrata sicura contro avversari semi-onesti e statici; l'estensione ad avversari *malicious* o adattivi resta un problema aperto, e gli autori la indicano come non banale. Aperte restano anche la generalizzazione a più di due parti, che sembra praticabile nel caso semi-onesto e statico, e la ricerca di compromessi fra sicurezza e dimensione delle share che allarghino il raggio d'azione a circuiti più grandi. Le funzioni con più bit in uscita si trattano applicando la costruzione a ciascun bit separatamente, ma è plausibile che si possa fare di meglio.

Nel complesso il V2PC è uno strumento adatto a funzioni semplici e circuiti poco profondi, più che una soluzione generale per il calcolo sicuro su larga scala. Detto questo, il risultato che dimostra non è la praticità: è che la crittografia visuale, nata per nascondere immagini, può essere usata come strumento di calcolo. È proprio la volontà di misurare da vicino questi limiti, trasformando la costruzione in codice eseguibile, a motivare l'implementazione descritta nel prossimo capitolo.

Capitolo 3

Implementazione in Python

Il protocollo del capitolo precedente si presta male all'intuizione. Le share crescono lungo il circuito in un modo che sulla carta si descrive con una formula, i *pointer bit* si accumulano su ogni filo sinistro, e la leggibilità del risultato dipende da un parametro che nessun disegno mostra. L'implementazione descritta in questo capitolo rende osservabile tutto questo: il programma genera le share degli ingressi prima di conoscerne i valori, simula la distribuzione alle due parti e ricostruisce il risultato con le sole share selezionate. Un solo nucleo di codice, due interfacce: una a riga di comando e una demo web che disegna il circuito e prepara il materiale da stampare per la sovrapposizione fisica.

3.1 Obiettivi e ambito dell'implementazione

Il software è scritto da zero sulla descrizione di D'Arco e De Prisco [6], senza riusare il codice della precedente implementazione in C++ di De Angelis e Del Monte [7], che ha però fatto da riferimento comportamentale per la numerazione delle porte, la disposizione delle share e la rappresentazione del *pointer bit*. Sorgenti e istruzioni di esecuzione stanno nel repository pubblico della tesi.¹

La scelta che ha guidato tutta la struttura è tenere separate le operazioni del protocollo. La costruzione produce entrambe le alternative, per i valori 0 e 1, di ogni occorrenza di input, e non riceve alcun assegnamento. La distribuzione ne sceglie una sola per

¹<https://github.com/gennarocarmine/bachelor-thesis-tozza.git>

ingresso: le share di Alice per consegna diretta, quelle di Bob attraverso la selezione che nel protocollo originale passa per un *oblivious transfer*. La ricostruzione vede soltanto il materiale selezionato, mai i valori degli input o le alternative scartate.

Su quell'*oblivious transfer* va detta subito una cosa, perché condiziona la lettura di tutto il capitolo. Il protocollo fisico basato sulle buste non è implementato, e nessun protocollo crittografico di rete lo sostituisce: la selezione di Bob avviene in locale, nello stesso processo che ha già letto l'intero assegnamento. Alice, in altre parole, potrebbe sapere cosa ha scelto Bob. Il programma riproduce il flusso del protocollo e l'informazione disponibile alla fine di ogni fase, non le sue garanzie di sicurezza. È uno strumento per generare share, studiarne la struttura e riprodurre la ricostruzione visuale.

3.2 Architettura del software

La divisione in moduli segue la divisione del protocollo, non l'ordine in cui il codice è stato scritto. La Tabella 3.1 ne riassume le responsabilità. Il confine che conta è quello fra il nucleo e le interfacce: `cli` e `web` passano entrambe per le stesse funzioni di `protocol`, quindi usano la stessa versione attiva delle primitive.

Le immagini sono matrici binarie NumPy, con 1 per il pixel nero e 0 per il pixel trasparente. Sovrapporre due share significa allora applicare un OR logico elemento per elemento, e questa corrispondenza fra l'operazione software e la sovrapposizione fisica dei lucidi è esatta, non approssimata. Pillow produce i PNG e i documenti da stampare, Flask espone l'interfaccia web.

Modulo	Responsabilità principale
<code>circuit</code>	Analisi della formula, costruzione dell'albero e valutazione booleana ordinaria.
<code>visual</code>	Primitive sulle immagini binarie, random grid, Scheme-2-MVCS e schema deterministico per i pointer.
<code>gate</code>	Costruzione delle alternative visuali associate a una singola porta.
<code>protocol</code>	Costruzione completa, distribuzione simulata e ricostruzione.
<code>artifacts</code>	Salvataggio e caricamento dei pacchetti di costruzione e trasferimento.
<code>render, printkit</code>	Esportazione delle immagini e preparazione delle share stampabili.
<code>cli, web</code>	Interfaccia a riga di comando e demo web.

Tabella 3.1: Organizzazione dei moduli dell'implementazione Python.

3.3 Rappresentazione della funzione booleana

La funzione si scrive come formula booleana, con $\&$ per AND, $|$ per OR, $\wedge$ per XOR e $\sim$ per NOT; le parentesi fissano l'ordine delle operazioni, e per NAND, NOR e XNOR basta negare le tre porte corrispondenti. La funzione dell'esempio completo del Capitolo 2 diventa

$$(x_1 | y_1) \& ((x_2 \& y_2) \& (x_3 | y_3)),$$

dove gli indici finiscono dentro il nome della variabile: x_1 , y_1 e così via. Il parsing passa per l'albero sintattico astratto della libreria standard di Python, accetta solo nomi di variabile e i quattro operatori previsti, e rifiuta chiamate di funzione, accessi ad attributi e ogni altra espressione. L'albero che ne esce ha tre tipi di nodo: ingresso, porta binaria, negazione. Accanto alla valutazione visuale ne resta attiva una booleana

ordinaria, che fa da riferimento nei test e nel comando di valutazione immediata.

La rappresentazione è quella di una formula, quindi ogni filo alimenta al più una porta. Una variabile può però comparire più volte: ogni occorrenza diventa una foglia distinta con share proprie, pur leggendo lo stesso bit dell'assegnamento. Anche le funzioni non *read-once* restano così trattabili senza introdurre nel protocollo un fan-out maggiore di uno, al prezzo di più porte e di share complessivamente più grandi.

Le variabili che iniziano per x sono input di Alice, quelle che iniziano per y input di Bob, come nella notazione del paper; con altri prefissi la libreria accetta la formula ma non attribuisce l'ingresso a una delle due parti. Le porte sono numerate come nodi di un albero binario: la radice è G_1 , i figli di G_i sono G_{2i} e G_{2i+1} . L'esempio del paper contiene perciò G_1, G_2, G_3, G_6 e G_7 , e salta gli indici 4 e 5. Un nodo è figlio sinistro se e solo se il suo indice è pari, e più avanti si vedrà che la ricostruzione sfrutta proprio questa proprietà.

3.4 Primitive visuali

I due valori logici sono immagini quadrate di lato s , la stessa quantità che il Capitolo 2 indica con t quando discute la dimensione dell'unità di share: I_0 tutta trasparente, I_1 tutta nera. Da qui in avanti useremo sempre s . L'utente lo sceglie, e in mancanza di indicazioni vale 32. Le funzioni di base controllano che ogni matrice contenga soltanto zeri e uno e che le share da sovrapporre abbiano la stessa forma. Sono controlli noiosi ma necessari: senza, un'immagine non valida o due ritagli disallineati produrrebbero in silenzio un risultato privo di significato.

3.4.1 Random grid e Scheme-2-MVCS

Lo schema random grid (2,2) della Sezione 1.4.1 parte da una matrice casuale R e calcola la seconda share come $R \oplus I$, con I immagine segreta. La sovrapposizione in OR dà nero pieno quando $I = I_1$; quando invece $I = I_0$ le due share coincidono e sopravvive una frazione statistica di pixel trasparenti. La funzione resta disponibile come primitiva a sé, ed è quella con cui sono state generate le figure del Capitolo 1;

sul percorso del protocollo, però, non viene invocata, perché lo schema che segue estrae direttamente la propria griglia comune.

Lo Scheme-2-MVCS della Sezione 2.3.1 porta lo stesso principio su due immagini I e J con una share in comune. L'implementazione segue esplicitamente i passaggi del paper: per ogni pixel seleziona una matrice di distribuzione dello schema $(2,2)$ -KK per ottenere sh_1 e sh_2 , poi costruisce sh_3 copiando sh_1 dove J è bianca e complementandola dove J è nera. La funzione `multi_secret_share_direct` usa due cicli sulle coordinate dei pixel, estrae un bit casuale a ogni iterazione e applica le condizioni dello pseudocodice in modo esplicito. NumPy conserva le matrici e fornisce il generatore pseudocasuale. Indicando con R la griglia casuale comune, la stessa costruzione si esprime in forma compatta come

$$sh_1 = R, \quad sh_2 = R \oplus I, \quad sh_3 = R \oplus J,$$

da cui $sh_1 \vee sh_2$ ricostruisce I e $sh_1 \vee sh_3$ ricostruisce J , sempre nel senso visuale appena descritto. Su questa primitiva poggia la costruzione della singola porta.

3.4.2 Costruzione di una porta

La funzione riceve l'operazione booleana, le due immagini dei possibili valori del filo di uscita e il materiale dei pointer che arriva dall'alto. Come nel GenGESS della Sezione 2.3.2, nascono due istanze indipendenti dello Scheme-2-MVCS: la prima copre le righe della tabella di verità con ingresso sinistro 0, la seconda quelle con ingresso sinistro 1. Le due share comuni diventano le alternative del filo sinistro; le quattro rimanenti si concatenano a coppie e formano quelle del filo destro.

L'ordine delle due metà dipende da un bit di permutazione casuale b : con $b = 0$ l'istanza del valore sinistro 0 occupa la prima metà, con $b = 1$ l'ordine si inverte. Chi valuta deve quindi leggere $p = v_1 \oplus b$, dove v_1 è il valore trasportato dal filo sinistro. Niente di tutto questo dipende dai valori che il protocollo sceglierà più tardi: la costruzione guarda soltanto la tabella di verità della porta e il materiale che le arriva dalla porta soprastante.

3.4.3 Rappresentazione e propagazione del pointer bit

Il software tiene distinti due oggetti che il paper chiama con lo stesso nome. Il pointer della porta corrente deve essere leggibile subito, perché decide quale metà della share destra ritagliare, e sta in testa alla share sinistra nella forma ricostruita:

$$0 \mapsto [\blacksquare \square], \quad 1 \mapsto [\blacksquare \blacksquare].$$

I pointer delle porte superiori non possono invece restare in chiaro. Ognuno è condiviso una volta sola con lo schema deterministico (2,2)-NS: una share porta, per ogni bit, un pixel nero e uno trasparente, l'altra la ripete per il segreto 0 e la complementa per il segreto 1. Sovrapponendole si ottiene sempre un blocco di peso uno oppure di peso due, senza errore.

Le due convenzioni non sono la stessa cosa, ed è opportuno distinguerle. Il pointer della porta corrente viene scritto direttamente nella forma qui sopra, con il pixel nero sempre in prima posizione. Quello di una porta superiore viene invece *ricostruito*: la share sinistra prodotta dallo schema (2,2)-NS ha un pixel nero e uno trasparente in ordine casuale, e per il valore 0 la sovrapposizione restituisce ora $[\blacksquare \square]$, ora $[\square \blacksquare]$, con la stessa probabilità. La lettura si fa quindi sul peso e non sulla posizione. Non è un dettaglio di comodo: è proprio l'ordine casuale a impedire che la sola share sinistra riveli il bit, cioè la condizione di sicurezza richiesta nella Sezione 2.3.3.

Le share deterministiche già prodotte viaggiano poi lungo il circuito senza essere ricondivise: sul filo sinistro in coda al pointer locale, sul filo destro replicate nelle due metà selezionabili, così che un solo taglio valga per l'immagine e per il canale dei pointer insieme. La proprietà che rende possibile questo trasporto è controllata a ogni porta durante la costruzione: se le share dei pointer che arrivano dall'alto differissero fra le due alternative del filo, la procedura si fermerebbe con un errore anziché produrre share che rivelano il valore del filo che le trasporta.

Il filo destro merita una parola in più. Quando il filo di uscita di una porta alimenta l'ingresso destro di quella soprastante, non c'è nessun pointer locale da condividere: il materiale che arriva dall'alto viene copiato per intero nelle due metà selezionabili, e sul filo sinistro prende posto una striscia trasparente della stessa lunghezza. Poiché il pixel trasparente è l'elemento neutro dell'OR, la sovrapposizione restituisce la metà selezionata inalterata, e la ricostruzione può applicare a ogni porta la stessa operazio-

ne senza distinguere il ruolo del filo.

In ricostruzione il pointer locale si legge prima di scegliere la metà. Dopo il taglio, le share deterministiche dei due fili si sovrappongono e ricostruiscono il pointer della porta successiva, che risulta quindi recuperato prima di essere usato. L'ingombro dei pointer cresce linearmente con il numero di porte attraversate.

3.5 Le fasi dell'implementazione

Nel paper le fasi sono due, costruzione e calcolo. Nel software il calcolo si spezza ancora in distribuzione e ricostruzione, e la ragione è la verificabilità: con tre operazioni distinte e tre pacchetti su disco si può controllare che la costruzione non dipenda dagli input e che la ricostruzione non raggiunga le alternative scartate.

3.5.1 Costruzione di tutte le alternative

La costruzione parte dalle immagini I_0 e I_1 del filo di uscita e scende ricorsivamente dalla radice verso le foglie. A ogni porta nascono le due alternative del filo sinistro e le due del filo destro, che diventano a loro volta i segreti da condividere nella porta sottostante. Arrivata a una foglia, la discesa conserva entrambe le alternative dell'occorrenza con le rispettive share dei pointer. Una negazione non richiede una porta visuale: il nodo NOT scambia le alternative 0 e 1 ricevute dall'alto, immagini e share dei pointer insieme, e prosegue senza aggiungere una sovrapposizione alla valutazione. Scambiare le sole immagini non basterebbe, perché il pointer premesso a ciascuna share resterebbe appaiato all'alternativa sbagliata.

La funzione di costruzione non ha nemmeno un parametro per l'assegnamento, e genera sempre le alternative di tutte le foglie. La casualità viene da un generatore inizializzato con un seme di 128 bit scelto dal sistema; fissandolo si riproduce un esperimento, altrimenti due costruzioni della stessa formula danno share diverse. Un limite configurabile sul numero totale di pixel ferma la procedura prima che una formula troppo grande esaurisca la memoria.

3.5.2 Distribuzione delle share

La distribuzione riceve una costruzione completa e un assegnamento binario, e di ogni occorrenza tiene la sola share del valore scelto. Le share degli input x portano l'etichetta di consegna diretta, quelle degli input y l'etichetta di OT simulato, e quelle delle variabili non attribuite a nessuna delle due parti l'etichetta di selezione locale. Ne esce un pacchetto con una sola immagine e un solo canale di pointer per foglia. Dentro non ci sono l'assegnamento, il seme della costruzione o le alternative non selezionate, e la destinazione deve essere nuova oppure vuota, altrimenti i file di una costruzione precedente lascerebbero disponibili proprio le share che il pacchetto dovrebbe escludere. La separazione è quindi materiale, non solo logica: quello che il programma consegna sta in una cartella diversa da quello che era disponibile all'inizio.

3.5.3 Ricostruzione

La ricostruzione riceve il solo pacchetto trasferito e percorre il circuito dalle foglie verso la radice. A ogni porta legge il pointer sul filo sinistro: se vale 0 prende la prima metà della share destra, se vale 1 la seconda. La metà scelta si sovrappone in OR alla share sinistra e produce la share del filo di uscita.

Lo stesso taglio vale per il canale dei pointer. Dal filo sinistro cade il blocco locale già letto, la parte restante si sovrappone alla metà selezionata del canale destro, e ne esce il pointer che accompagna l'immagine verso la porta superiore. Di ogni passaggio il programma registra indice della porta, operazione, metà selezionata, immagine prodotta e i due pointer, tenuti distinti: quello letto sul filo sinistro, che ha deciso il taglio, e quello ricostruito per la porta superiore. Il secondo viene interpretato soltanto quando serve davvero, cioè quando il filo di uscita alimenta l'ingresso sinistro della porta soprastante: con la numerazione ad albero questo accade esattamente quando l'indice della porta è pari, e il programma usa quel criterio. Sono gli stessi dati che alimentano l'output della CLI e il disegno della demo web.

Alla radice l'immagine si interpreta con la convenzione del protocollo: nero pieno significa 1, un solo pixel trasparente significa 0. Questa lettura automatica prende il posto dell'osservazione a occhio dell'esperimento fisico. La funzione non conosce

il valore booleano atteso, e il confronto con la valutazione ordinaria della formula spetta al solo comando di valutazione immediata, che esegue in locale tutte e tre le fasi.

3.6 Persistenza ed esportazione delle share

Costruzione e trasferimento si salvano su disco, ed è per questo che le tre fasi possono girare in momenti diversi. Una costruzione contiene la formula, la dimensione delle immagini, il seme effettivo, le informazioni sulle foglie e le matrici di entrambe le alternative; un trasferimento usa un formato separato e tiene le sole matrici selezionate. I metadati stanno in un piccolo file JSON interno, le matrici in archivi NumPy compressi che il programma carica con la serializzazione di oggetti disabilitata.

Per la visualizzazione e la stampa ogni matrice diventa un PNG senza interpolazione, così che i singoli pixel restino netti, con i blocchi pointer in testa all'immagine principale. In una share destra ogni pointer precede la metà a cui appartiene: pointer sinistro, immagine sinistra, pointer destro, immagine destra, come nelle figure del paper.

Il kit di stampa delle share trasferite contiene un PDF A4 a 300 dpi, i PNG individuali, l'immagine di riferimento dell'uscita e le istruzioni operative. Le pagine condividono la stessa scala e portano crocini per taglio e allineamento. Le share sono disposte fino a tre per pagina, con testi esterni al bordo di taglio e distanziati dai crocini. L'intestazione compare solo sul primo foglio; scala e numero di pagina restano nel piè di pagina di ciascun foglio. Le istruzioni incluse nell'archivio sono in italiano e in inglese. Un reticolo sottile rende riconoscibili i pixel logici come nelle tavole finali del paper, e la sequenza delle sovrapposizioni segue la numerazione ad albero delle porte. Il materiale per l'esperimento fisico c'è tutto. La qualità del risultato, però, dipende anche da stampante, opacità del supporto e precisione dell'allineamento, che nessun controllo automatico può verificare.

3.7 Interfaccia a riga di comando

L'eseguibile `v2pc` espone le tre fasi una per comando:


```
v2pc construct "(x1|y1)&((x2&y2)&(x3|y3))" \
  --output costruzione --side 32
```

```
v2pc transfer --source costruzione \
  --input "x1=0,y1=1,x2=1,y2=1,x3=1,y3=0" \
  --output trasferimento
```

```
v2pc reconstruct --source trasferimento
```

Nel primo comando l'assegnamento non compare, nel terzo nemmeno: è la separazione delle fasi resa visibile dalla riga di comando. Il comando `evaluate` accorcia il giro per gli esperimenti locali, costruendo, selezionando e ricostruendo in un colpo solo, e mostra anche il confronto con il valore booleano; `serve` avvia la demo web. Le opzioni scelgono il lato delle immagini e, dove entra nuova casualità, un seme riproducibile.

Anche nell'output testuale i pointer compaiono accanto alle rispettive share, e la CLI distingue quello della porta corrente, disponibile in chiaro, dalle share dei pointer che serviranno più in alto. Terminale e demo web mostrano lo stesso modello del protocollo, compreso l'avviso sulla natura simulata della selezione di Bob.

3.8 Demo web

La demo gira in locale su Flask e usa le stesse funzioni della CLI. La pagina chiede formula, dimensione delle immagini e un seme opzionale; le variabili si riconoscono mentre la formula viene digitata e per ciascuna compaiono i due selettori, come nella Figura 3.1. Il pulsante *Simula il trasferimento e ricostruisci* esegue le tre fasi in locale senza avviare download. Il pulsante separato *Scarica tutte le alternative* genera ed esporta la costruzione senza usare i valori scelti. Il risultato segue il flusso del protocollo: quante alternative iniziali, quali share di Alice consegnate direttamente, quali share di Bob selezionate, e infine la ricostruzione. La pagina non espone un'API JSON, così i formati interni restano separati dalla presentazione. Se il seme è lasciato vuoto, il download delle alternative restituisce al browser il seme effettivamente

usato e la pagina lo conserva, insieme alla formula e alla dimensione delle immagini. La simulazione può così riprodurre la stessa costruzione casuale; il seme conservato viene ignorato se nel frattempo cambiano formula o dimensione.

Visual Two-Party Computation

Costruisci le share, segui il pointer bit e osserva la sovrapposizione trasformarsi nel risultato della funzione.

FUNZIONE BOOLEANA

$(x1|y1) \& ((x2\&y2) \& (x3|y3))$

Operatori: $\&$ AND, $|$ OR, $\wedge$ XOR, $\sim$ NOT. Le variabili che iniziano per x sono di Alice, quelle per y di Bob.

VALORI DELLE VARIABILI

Selezione 0 oppure 1

x1: 0 1 y1: 0 1 x2: 0 1 y2: 0 1 x3: 0 1 y3: 0 1

LATO IMMAGINE: 32

SEME (OPZIONALE): casuale

[Simula il trasferimento e ricostruisce](#) [Scarica tutte le alternative](#)

Figura 3.1: Il modulo di configurazione. La formula si scrive con la sintassi della Sezione 3.3, e per ogni variabile riconosciuta compare un selettore binario. I due pulsanti tengono separata la costruzione, che ignora i valori scelti, dalla distribuzione simulata.

Dopo la simulazione la pagina mostra le share effettivamente consegnate, una per occorrenza, con i blocchi pointer ingranditi accanto a ciascuna (Figura 3.2). L’etichetta sotto ogni share ricorda da quale canale è arrivata, consegna diretta per gli input di Alice e OT simulato per quelli di Bob. Le larghezze diverse rendono visibile il raddoppio sul filo destro senza bisogno di misurarle.

Share trasferite

Una sola alternativa per ogni occorrenza di input.



Ogni pointer è un blocco 1x2 anteposto alla parte a cui appartiene. Sul filo destro ogni pointer precede immediatamente la propria metà.

Figura 3.2: Le sei share selezionate per l'assegnamento $x = (0, 1, 1)$, $y = (1, 1, 0)$. Accanto a ciascuna compaiono ingranditi il pointer bit in chiaro della porta corrente e le share dei pointer destinati alle porte superiori. Il filo y_3 , che non fa da ingresso sinistro a nessuna porta, non porta alcun pointer.

La ricostruzione si legge porta per porta (Figura 3.3): per ciascuna il programma riporta l'operazione, quali share sono state sovrapposte, il pointer recuperato, la metà che quel pointer ha selezionato e l'immagine prodotta. È la traccia che nella versione fisica corrisponde alla sequenza di tagli e sovrapposizioni, e serve soprattutto a controllare che il pointer venga recuperato prima del momento in cui va usato.



Figura 3.3: La ricostruzione dell'esempio del paper, porta per porta. Le immagini si restringono salendo da G_2 verso G_1 , e le prime quattro porte producono uscite lette come 0: solo la radice arriva al nero pieno.

Il pannello del risultato (Figura 3.4) affianca all'immagine di uscita il valore letto, il confronto con la valutazione booleana ordinaria e il promemoria sulla separazione delle tre fasi, compreso l'avviso che l'oblivious transfer non è implementato.

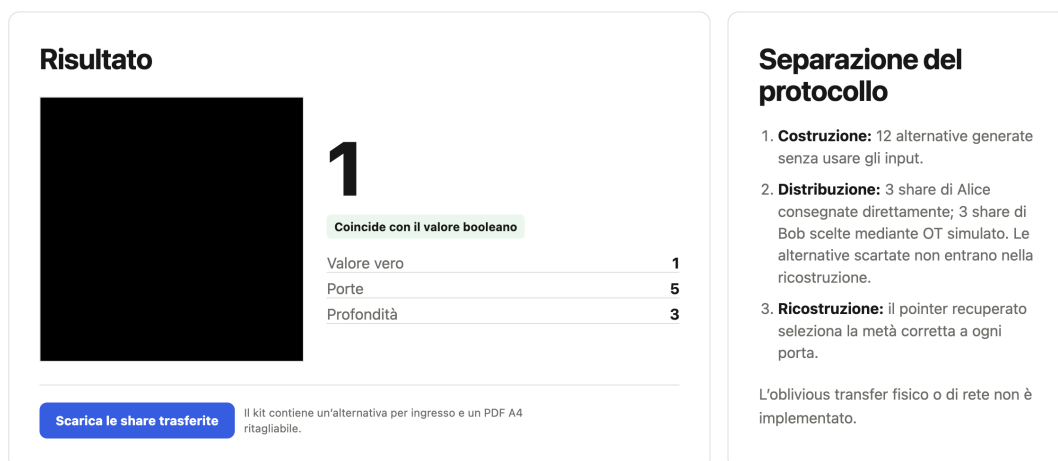


Figura 3.4: Il pannello del risultato. L'immagine di uscita è interamente nera e viene letta come 1, in accordo con la valutazione booleana della formula sull'assegnamento scelto. Il riquadro a destra riassume che cosa ciascuna delle tre fasi ha visto.

Dopo la ricostruzione compare il disegno dell'intero circuito (Figura 3.5), che si legge dagli ingressi verso l'uscita e mostra per ogni nodo la porta, la share che lo attraversa e il risultato intermedio. Accanto alle share di ingresso stanno, disegnati come blocchi di pixel, il pointer bit della porta corrente e le eventuali share dei pointer superiori. Il diagramma si ridimensiona da solo per stare nello spazio disponibile, e un pulsante lo cattura in un PNG ad alta risoluzione con la stessa disposizione mostrata dal browser.

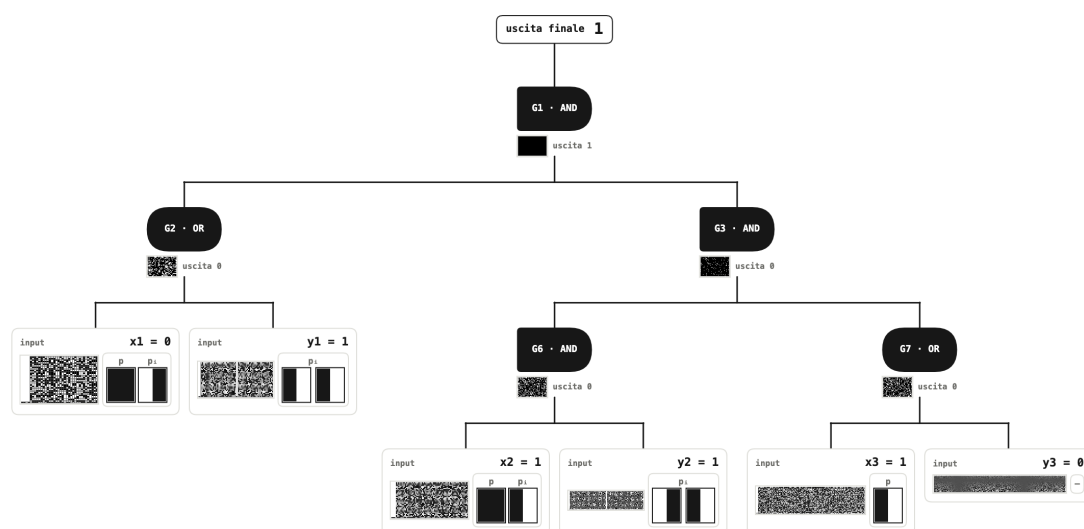


Figura 3.5: Il circuito disegnato dalla demo. Ogni porta riporta l'uscita letta e la share che la attraversa; alle foglie compaiono la share di ingresso, il pointer in chiaro p e le share dei pointer associati alle porte superiori p_i . Le larghezze crescono scendendo di livello, come previsto dalla concatenazione sul filo destro.

Gli archivi scaricabili sono due, e la differenza fra loro riproduce la differenza fra le fasi. Il primo contiene tutte le alternative, un PDF A4 a scala comune e il piano di distribuzione da preparare prima di conoscere gli input: le etichette che distinguono i valori 0 e 1 restano fuori dal bordo di taglio e vanno rimosse prima dell'OT fisico. Il secondo contiene le sole share trasferite e un altro PDF per verificare la sovrapposizione sull'assegnamento scelto. In entrambi più share condividono lo stesso foglio, con crocini di allineamento e reticolo dei pixel logici, e nelle share destre ogni blocco pointer precede immediatamente la metà di immagine a cui appartiene.

3.9 Misure

Il Capitolo 2 prevede due cose sul comportamento del protocollo: che le immagini delle share raddoppino sui fili destri, e che la probabilità di leggere male un'uscita nulla dipenda dalla profondità del circuito. La prima previsione la misura conferma. Per la seconda, invece, i risultati ottenuti sui circuiti ramificati differiscono dalla stima basata sulla sola profondità e motivano il confronto sviluppato di seguito. I comandi che riproducono ciascuna misura, con gli assegnamenti e i semi usati, stanno nell'Appendice B.5: tutti i valori che seguono vengono da lì, con semi consecutivi a partire da zero.

3.9.1 Crescita delle share

Sull'esempio del paper le larghezze delle immagini di ingresso raddoppiano regolarmente da sinistra verso destra, da s fino a $8s$: il raddoppio sui fili destri descritto nella Sezione 2.4 si osserva direttamente sulle share prodotte. I canali dei pointer si comportano in modo diverso, perché ogni bit occupa due pixel qualunque sia il lato: non dipendono affatto da s e crescono soltanto con il numero di porte attraversate. Raddoppiando s l'ingombro complessivo della costruzione si moltiplica dunque per quattro, a meno dei pointer che restano identici. La dipendenza da s è quadratica, quella dalla profondità ha un limite superiore esponenziale, come previsto.

Il confronto fra formule diverse, a parità di lato, separa i due fattori. La larghezza dell'immagine principale di una share di ingresso dipende dal numero di fili destri lungo il cammino dalla radice alla foglia: ogni filo destro ne raddoppia la larghezza, mentre un filo sinistro la lascia invariata. La profondità determina quindi un limite superiore, non da sola la larghezza effettiva. L'esempio del paper e la formula bilanciata a sette porte arrivano alla stessa larghezza massima; l'ingombro complessivo dipende invece dalle larghezze di tutte le share e dai pointer. Nel confronto considerato, la formula a sette porte ha otto foglie anziché sei e un ingombro maggiore. Con $s = 8$ l'intera costruzione dell'esempio del paper occupa 2724 pixel, il valore che stampa il primo script dell'Appendice B.5; lì si trovano anche le larghezze e i totali delle altre formule.

3.9.2 Probabilità di falso positivo

Il canale dei pointer è deterministico; le immagini principali si portano dietro la componente probabilistica del random grid. Un risultato nero si legge sempre come 1, mentre un risultato che dovrebbe essere bianco può annerirsi del tutto dopo troppe sovrapposizioni. L'errore è quindi unilaterale: falsi negativi non ne esistono, falsi positivi sì.

Detto ℓ il numero di porte del circuito e s il lato dell'immagine, la stima discussa nella Sezione 2.4 lega quell'errore alla sola profondità d :

$$P_{\text{paper}} = (1 - 2^{-(d+1)})^{s^2},$$

dove $d + 1$ conta le sovrapposizioni che una share subisce lungo il cammino più lungo. Le esecuzioni della presente implementazione mostrano tuttavia una frequenza diversa. Sull'esempio del paper, con un assegnamento per cui $f(x, y) = 0$, su tremila costruzioni con semi distinti e $s = 8$ si contano 391 letture sbagliate, cioè il 13,0%, mentre la formula ne prevede lo 0,02%. In base a quest'ultima stima il numero atteso di errori in tremila prove sarebbe inferiore a uno, perciò la discrepanza osservata non è riconducibile alla variabilità campionaria.

Il motivo si vede tornando a che cosa fa una porta in ricostruzione. Chi valuta sovrappone la share sinistra, che è la griglia casuale R generata da quella porta, alla metà destra selezionata, che vale $R \oplus S$ con S il segreto condiviso dalla porta. Il risultato è

$$R \vee (R \oplus S) = R \vee S,$$

cioè il segreto voluto con sopra una griglia casuale fresca. Ogni porta ne aggiunge una, e poiché l'OR non può togliere pixel neri, nessuna sovrapposizione successiva la rimuove.

Il conto si ripete allora salendo verso la radice. All'ingresso sinistro di una porta arriva l'immagine prodotta dal sottoalbero sinistro, che si porta dietro le griglie di tutte le porte che lo compongono; all'ingresso destro arriva quella del sottoalbero destro, con le proprie. Sovrapponendole le due si sommano, e a esse si aggiunge la griglia della porta corrente; le griglie restano indipendenti, perché ogni porta usa casualità fresca. L'immagine finale è quindi $I_{f(x,y)}$ coperta da tante griglie quante

sono le porte del circuito, non quante sono quelle di un singolo cammino: il rumore che sale dal sottoalbero sinistro e quello che sale dal destro si accumulano sulla stessa area.

Se ne ricava, per la costruzione implementata, il modello seguente. Un pixel dell'area bianca sopravvive solo quando vale 0 in tutte le ℓ griglie, e trattando i pixel di una stessa immagine come indipendenti, con la stessa semplificazione adottata dagli autori,

$$\Pr[\text{un pixel dell'area bianca sopravvive}] = 2^{-\ell}, \quad P_{\text{porte}} = (1 - 2^{-\ell})^{s^2}.$$

Sull'esempio del paper questa formula prevede il 13,1%. Il valore misurato è il 13,0%: su tremila prove la deviazione standard della frequenza, secondo il modello binomiale, vale circa 0,006. Lo scarto è inferiore a 0,2 deviazioni standard ed è compatibile con la previsione.

Per escludere che si tratti di una coincidenza sul singolo circuito, l'esponente si può ricavare dai dati invertendo la formula come $k = -\log_2(1 - P^{1/s^2})$. La Tabella 3.2 lo riporta per quattro formule con numero di porte e profondità diversi, e in ogni caso k risulta vicino al numero di porte e non a quello dei livelli. Le ultime due righe sono le più indicative, perché hanno la stessa profondità e danno esponenti diversi.

Formula	porte	livelli	s	errore	k
$x1 \& y1$	1	1	2	0,0655	1,02
$(x1 \& y1) \& (x2 \& y2)$	3	2	3	0,2890	2,96
$(x1 y1) \& ((x2 \& y2) \& (x3 y3))$	5	3	4	0,6035	5,01
$((x1 \& y1) \& (x2 \& y2)) \& ((x3 \& y3) \& (x4 \& y4))$	7	3	4	0,8840	7,03

Tabella 3.2: Esponente k ricavato dall'errore misurato su quattromila costruzioni per riga, con lati s scelti piccoli perché l'errore resti misurabile. Ogni riga viene dal secondo script della Sezione B.5, eseguito con semi consecutivi a partire da zero: la colonna dell'errore è la frequenza osservata, quella di k si ottiene invertendo la formula come indicato qui sopra.

Le due stime coincidono quando il circuito è una catena, cioè quando ogni porta ha per figli un'altra porta e una foglia: in questo caso porte e livelli sono in numero

uguale. Sulle due catene riportate nell'Appendice B.5 l'esponente misurato vale 2,99 e 4,06, in accordo con entrambe. Sui circuiti ramificati, invece, la stima basata sulla profondità sottostima l'errore osservato.

Adottando il modello suggerito dalle misure e imponendo $P_{\text{porte}} < \varepsilon$ si ottiene

$$s \geq \sqrt{\log_{1-2^{-\ell}} \varepsilon},$$

che sostituisce ℓ a $d + 1$ nella disuguaglianza della Sezione 2.4. Sull'esempio del paper, con la stessa probabilità di errore tollerata scelta dagli autori, il lato necessario passa da 8 a 17. Su circuiti bilanciati lo scarto si allarga in fretta, perché in un albero binario completo con L livelli le porte sono $2^L - 1$: la Tabella 3.3 mette a confronto i due dimensionamenti.

livelli	porte	s per profondità	s per numero di porte
2	3	6	8
3	7	8	33
4	15	12	529

Tabella 3.3: Lato minimo dell'unità di share per un albero binario completo, con la probabilità di errore tollerata della Sezione 2.4, secondo i due criteri messi a confronto. In un albero completo con L livelli le porte sono $2^L - 1$, quindi lo scarto fra le due colonne si allarga a ogni livello.

Lo scarto ha un riscontro immediato nel programma, che rifiuta lati superiori a 512 pixel: già a quattro livelli il criterio basato sul numero di porte chiede un'immagine che l'implementazione non accetta, mentre quello per profondità ne chiederebbe una di dodici. Oltre quel punto il confronto perde interesse pratico, perché entrambe le grandezze escono dal campo in cui il protocollo è eseguibile.

Il risultato non modifica la logica funzionale né le proprietà di sicurezza del protocollo, ma incide sulla garanzia quantitativa di correttezza per una dimensione fissata delle immagini e, quindi, sulla soglia oltre la quale il protocollo è utilizzabile. La conclusione degli autori, che il V2PC serva per circuiti poco profondi, ne esce rafforzata più che smentita; le misure forniscono inoltre un criterio più prudente per

il dimensionamento. Il programma non lo calcola e non lo impone: *s* resta una scelta dell'utente, ed è il primo controllo automatico che varrebbe la pena aggiungere.

3.10 Limiti

Il limite di sicurezza è già stato dichiarato all'inizio del capitolo, ma torna qui perché è il vincolo che pesa di più: senza un oblivious transfer, fisico o crittografico, fra due soggetti realmente separati, il software da solo non è un protocollo sicuro a due parti. Anche assumendo di aggiungerlo, le proprietà del paper continuerebbero a poggiare sul modello semi-onesto e sulle modalità fisiche descritte nel Capitolo 2.

Gli altri limiti non nascono dall'implementazione ma dall'ipotesi su cui il protocollo poggia. Il circuito è una formula a fan-out uno, quindi un risultato intermedio non può alimentare più porte e il sotto-circuito va duplicato; per lo stesso motivo ogni formula ha una sola uscita, e le funzioni a più bit richiedono costruzioni indipendenti che l'interfaccia non raccoglie insieme. La duplicazione poi si paga due volte, perché le concatenazioni sul filo destro fanno crescere in fretta le share con la profondità, e alla luce della Sezione 3.9.2 si paga anche una terza volta: ogni porta aggiunta, comprese quelle nate da una duplicazione, alza il lato minimo delle immagini secondo il modello emerso dalle misure. Il limite sul numero complessivo di pixel evita che la memoria finisca, ma restringe l'uso pratico a formule piccole, e il criterio basato sul numero di porte restringe quell'intervallo più di quanto suggerisca la stima per profondità.

Resta un ultimo scarto, ed è quello fra la lettura del programma e la lettura di una persona. Il criterio software è esatto: un solo pixel trasparente basta a determinare il valore 0. Un osservatore davanti a una stampa poco opaca, o a due lucidi appena disallineati, può vedere altro.

Capitolo 4

Conclusioni

Questa tesi è partita da un'idea che la crittografia visuale rende quasi tangibile: un segreto si può dividere in immagini che, sovrapposte, lo rivelano all'occhio senza alcun calcolo. Il Capitolo 1 ha ricostruito quell'idea e i suoi schemi, dal secret sharing di Shamir e Blakley fino al random grid senza espansione dei pixel. Il Capitolo 2 l'ha portata fino al V2PC di D'Arco e De Prisco, dove la sovrapposizione smette di limitarsi a nascondere e comincia a calcolare, e dove perfino l'oblivious transfer si esegue con due buste e un tritacarte. Il Capitolo 3 l'ha tradotta in codice. Il filo che tiene insieme i tre capitoli è sempre lo stesso: due persone che calcolano una funzione scambiandosi lucidi invece che bit.

L'implementazione in Python è scritta da zero sulla descrizione degli autori, senza riusare il codice del software C++ che citano, adottato però come riferimento comportamentale per la numerazione delle porte e la disposizione delle share. La scelta che ne ha determinato la struttura è tenere separate le tre operazioni del protocollo: la costruzione genera entrambe le alternative di ogni ingresso senza ricevere alcun assegnamento, la distribuzione ne sceglie una sola, la ricostruzione vede soltanto il materiale selezionato. Sono tre comandi e tre pacchetti su disco, e quella separazione non è una comodità di ingegneria: è ciò che permette di controllare, guardando i file, che la costruzione non dipenda dagli input e che la ricostruzione non raggiunga le alternative scartate. Il linguaggio delle formule copre tutte le porte binarie non costanti, e non soltanto le AND e le OR dell'esempio del paper, e accetta anche le formule in cui una variabile si ripete, trattando ogni occorrenza come una foglia distinta. Le

due interfacce, riga di comando e demo web, permettono di usare il protocollo senza scrivere codice e, soprattutto, di vedere le share invece di immaginarle.

Eseguire il protocollo ha reso preciso un punto che la teoria enuncia e lascia in secondo piano, cioè il legame fra correttezza e dimensione delle immagini. Poiché la ricostruzione del bianco è probabilistica, un'uscita che vale zero può essere letta come uno quando le immagini sono troppo piccole, e la stima degli autori lega quel rischio alla profondità del circuito. Nei circuiti ramificati esaminati, tuttavia, le misure mostrano una frequenza di errore diversa. Ogni porta, in ricostruzione, sovrappone una griglia casuale nuova, e le griglie che salgono dai due sottoalberi si accumulano entrambe sulla stessa area: nei casi studiati l'esponente osservato risulta compatibile con il numero di porte, più che con il numero di livelli. Le due quantità coincidono sui circuiti a catena. Sull'esempio degli autori la differenza è concreta: con il lato che propongono l'errore misurato è del 13,0% invece dello 0,02% previsto, e per rientrare nella soglia considerata il modello suggerito dalle misure richiede di portare il lato da otto a diciassette. Il risultato non modifica la logica funzionale né le proprietà di sicurezza del protocollo, ma incide sulla garanzia quantitativa di correttezza per una dimensione fissata delle immagini e sulla soglia di praticabilità. Conferma così, con maggiore prudenza, la conclusione degli autori che il V2PC sia adatto soprattutto a circuiti poco profondi.

Restano dei limiti, ed è giusto dichiararli. Il primo è che l'oblivious transfer non è implementato: la selezione di Bob avviene in locale, nello stesso processo che ha già letto l'assegnamento, quindi il software da solo non è un protocollo sicuro a due parti e non pretende di esserlo. Il secondo è il fan-out uno: il circuito è una formula, un risultato intermedio non può alimentare più porte e dividerlo richiede di duplicare il sotto-circuito, con share che crescono di conseguenza. Per lo stesso motivo ogni formula ha una sola uscita, e una funzione a più bit richiede una costruzione indipendente per ciascuno, che l'interfaccia non raccoglie insieme. Il terzo è la crescita delle share, che raddoppiano sul filo destro a ogni livello e che, secondo il modello emerso dalle misure, si paga anche in dimensione dell'unità, perché ogni porta aggiunta alza il lato minimo. C'è infine il modello di avversario: la costruzione è dimostrata sicura contro avversari semi-onesti e statici, e una parte che devia dai passi previsti esce dalle garanzie del V2PC.

Su che cosa serva davvero un protocollo del genere conviene essere sobri. Non compete con il calcolo sicuro tradizionale: i circuiti di dimensioni reali sono fuori portata per il V2PC. Quello che offre è di natura diversa: un calcolo che si può eseguire senza calcolatore e, cosa che conta almeno quanto, la cui fase di valutazione si può osservare mentre accade. Il processo consiste nel ritagliare un lucido e appoggiarlo su un altro, e le garanzie continuano a valere finché reggono le ipotesi di sicurezza e di corretta preparazione previste dal protocollo. Sono qualità che valgono nella didattica e nella divulgazione, dove il protocollo rende visibile una nozione altrimenti astratta, e nei contesti a bassa tecnologia in cui un computer non è disponibile oppure non è desiderato.

I lavori possibili si vedono bene da qui. Il primo è chiudere la parte mancante, cioè un oblivious transfer vero fra due processi separati, che renderebbe il software un protocollo eseguibile e non soltanto una simulazione del suo flusso. Il secondo è più modesto ma immediato: far calcolare al programma il lato minimo delle immagini a partire dal numero di porte, e avvisare quando la scelta dell'utente è troppo piccola. Restano poi le direzioni che gli autori stessi indicano come aperte, l'estensione ad avversari *malicious* o adattivi e la generalizzazione a più di due parti, insieme alla ricerca di compromessi fra sicurezza e dimensione delle share che allarghino il raggio d'azione a circuiti più grandi.

Vederlo funzionare, alla fine, è la parte che convince più di ogni dimostrazione. Si stampa una funzione su sei fogli trasparenti, si sceglie un input decidendo quale foglio consegnare, si taglia, si sovrappone, e il risultato compare. Per un protocollo nato per calcolare senza calcolatori, è la verifica più fedele che si possa chiedere.

Appendice A

Implementazione dello schema (2,2)

Si riporta il codice Python che implementa lo schema di crittografia visuale (2,2) descritto nella Sezione 1.3.1. Lo script binarizza l'immagine segreta, genera le due *share* estraendo per ogni pixel una matrice dalla collezione corrispondente al colore del pixel, e ricostruisce il segreto sovrapponendo le *share* con l'operatore OR. Le immagini della Figura 1.4 sono state prodotte con questo codice.

```
1 import numpy as np
2 from PIL import Image
3
4 # Le collezioni C[I][r] dello pseudocodice.
5 # - I = valore del pixel segreto: 0 = bianco, 1 = nero
6 # - r = bit scelto a caso in {0, 1}
7 # Ogni matrice ha 2 righe (riga 0 -> share 1, riga 1 -> share 2)
8 # e 2 colonne (i 2 sottopixel). 1 = nero, 0 = bianco.
9 C = {
10     0: [ # pixel BIANCO -> le due righe sono UGUALI
11         np.array([[1, 0],
12                  [1, 0]], dtype=np.uint8), # r = 0
13         np.array([[0, 1],
14                  [0, 1]], dtype=np.uint8), # r = 1
15     ],
16     1: [ # pixel NERO -> le due righe sono OPPOSTE
17         np.array([[1, 0],
18                  [0, 1]], dtype=np.uint8), # r = 0
19         np.array([[0, 1],
20                  [1, 0]], dtype=np.uint8), # r = 1
21     ],
22 }
23
24
```

```

25 def carica_segreto(percorso, soglia=128):
26     """Apri un'immagine e la rende bianco/nero. Ritorna 1 = nero, 0 =
    ↪ bianco."""
27     img = Image.open(percorso).convert("L") # scala di grigi
28     pixel = np.asarray(img)
29     return (pixel < soglia).astype(np.uint8) # scuro -> 1 (nero)
30
31
32 def Shr(I):
33     """Shr(I): divide il segreto I nelle due share sh1, sh2."""
34     n_righe, n_colonne = I.shape
35
36     # Ogni pixel diventa 2 sottopixel: le share sono larghe il doppio.
37     sh1 = np.zeros((n_righe, 2 * n_colonne), dtype=np.uint8)
38     sh2 = np.zeros((n_righe, 2 * n_colonne), dtype=np.uint8)
39
40     for i in range(n_righe):
41         for j in range(n_colonne):
42             r = np.random.randint(2) # r_{i,j} a caso in {0, 1}
43             M = C[I[i, j]][r] # scelgo la matrice C_{I, r}
44             sh1[i, 2*j:2*j+2] = M[0] # riga 0 -> share 1
45             sh2[i, 2*j:2*j+2] = M[1] # riga 1 -> share 2
46
47     return sh1, sh2
48
49
50 def Sup(sh1, sh2):
51     """Sup: sovrapposizione delle share. Nero se nero in almeno una share
    ↪ (OR)."""
52     return sh1 | sh2
53
54
55 def Rec(sh1, sh2):
56     """Rec(sh1, sh2): ricostruisce il segreto sovrapponendo le share."""
57     return Sup(sh1, sh2)

```

A.1 Implementazione dello schema (2,2) con Random Grid

```

1 import numpy as np
2 from PIL import Image
3
4 def carica_segreto(percorso, soglia=128):
5     """Apri un'immagine e la rende bianco/nero. Ritorna 1 = nero, 0 =
6     ↪ bianco."""
7     img = Image.open(percorso).convert("L") # scala di grigi
8     pixel = np.asarray(img)
9     return (pixel < soglia).astype(np.uint8) # scuro -> 1 (nero)
10
11 def ShrRG(I):
12     """ShrRG(I): divide il segreto I nelle due griglie R1, R2 (stessa
13     ↪ dimensione)."""
14     n_righe, n_colonne = I.shape
15     # Le griglie hanno la STESSA dimensione del segreto (nessuna
16     # ↪ espansione).
17     R1 = np.zeros((n_righe, n_colonne), dtype=np.uint8)
18     R2 = np.zeros((n_righe, n_colonne), dtype=np.uint8)
19
20     for i in range(n_righe):
21         for j in range(n_colonne):
22             R1[i, j] = np.random.randint(2) # bit casuale in {0, 1}
23             if I[i, j] == 0:
24                 # pixel BIANCO -> R2 uguale a R1
25                 R2[i, j] = R1[i, j]
26             else:
27                 # pixel NERO -> R2 opposto a R1
28                 R2[i, j] = 1 - R1[i, j]
29
30     return R1, R2
31
32 def Sup(R1, R2):
33     """Sup: sovrapposizione. Nero se nero in almeno una griglia (OR)."""
34     return R1 | R2
35
36
37 def Rec(R1, R2):
38     """Rec(R1, R2): ricostruisce il segreto sovrapponendo le griglie."""
39     return Sup(R1, R2)

```


Appendice B

Riproduzione del software V2PC

Questa appendice serve a rimettere in piedi l’implementazione descritta nel Capitolo 3 su un’altra macchina, e a rieseguire le misure della Sezione 3.9. Il codice completo sta nel repository pubblico della tesi,¹ quindi queste pagine si limitano alla primitiva visuale su cui poggia tutto il resto, al procedimento minimo per riprodurre l’esempio del paper e agli script che generano i numeri del capitolo.

B.1 La primitiva Scheme-2-MVCS

Immagini e share sono matrici binarie NumPy, con 1 per il pixel nero e 0 per il pixel trasparente; la sovrapposizione fisica diventa un OR. Lo Scheme-2-MVCS genera una share casuale comune e due ulteriori share, ciascuna associata a uno dei due segreti: la coppia (sh_1, sh_2) ricostruisce I_0 , la coppia (sh_1, sh_3) ricostruisce I_1 . Il frammento seguente riprende la funzione del modulo `visual`, usata da CLI e demo, omettendo le annotazioni di tipo e la documentazione. La funzione ausiliaria `_binary` verifica la forma bidimensionale e i valori binari delle matrici usate dalla primitiva.

¹<https://github.com/gennarocarmin/bachelor-thesis-tozza.git>

```

1 def multi_secret_share_direct(first_secret, second_secret, rng):
2     I0 = _binary(first_secret, dimensions=2)
3     I1 = _binary(second_secret, dimensions=2)
4     if I0.shape != I1.shape:
5         raise ValueError("I due segreti devono avere la stessa forma.")
6
7     # C[colore del segreto, indice r, riga della matrice]
8     C = np.array(
9         [[0, 0], [1, 1]], [[0, 1], [1, 0]], dtype=np.uint8
10    )
11    sh1 = np.empty_like(I0)
12    sh2 = np.empty_like(I0)
13    sh3 = np.empty_like(I0)
14    height, width = I0.shape
15    for i in range(height):
16        for j in range(width):
17            r = rng.integers(0, 2)
18            matrix = C[I0[i, j], r]
19            sh1[i, j] = matrix[0]
20            sh2[i, j] = matrix[1]
21            if I1[i, j] == 1:
22                if sh1[i, j] == 0:
23                    sh3[i, j] = 1
24                elif sh1[i, j] == 1:
25                    sh3[i, j] = 0
26            elif I1[i, j] == 0:
27                if sh1[i, j] == 1:
28                    sh3[i, j] = 1
29                elif sh1[i, j] == 0:
30                    sh3[i, j] = 0
31    return sh1, sh2, sh3

```

Chiamando sh_1 , sh_2 e sh_3 le tre share restituite,

$$\text{Sup}(sh_1, sh_2) = I_0, \quad \text{Sup}(sh_1, sh_3) = I_1$$

nel senso visuale del random grid: i pixel neri del segreto tornano esatti, nelle zone bianche resta una trama casuale. Da questa costruzione nascono le share di ogni porta. Il canale dei pointer bit fa eccezione e usa lo schema deterministico (2,2) della Sezione 2.3.3.

B.2 Preparazione dell'ambiente

Serve Python 3.9 o successivo. Conviene creare un ambiente virtuale dalla radice del repository, così che NumPy, Pillow e Flask non interferiscano con altre installazioni presenti sul sistema. Su macOS e Linux:

```
1 python3 -m venv .venv
2 source .venv/bin/activate
```

Su Windows l'attivazione è diversa:

```
1 py -m venv .venv
2 .venv\Scripts\activate
```

Con l'ambiente attivo si installano dipendenze e programma:

```
1 python -m pip install --upgrade pip setuptools wheel
2 python -m pip install -r requirements.txt
3 python -m pip install --no-deps -e ./code/v2pc-python
```

L'ultima riga installa in modalità modificabile e collega il comando `v2pc` al codice locale; `v2pc -help` conferma che l'interfaccia risponde. La cartella `.venv` appartiene al computer su cui nasce, non contiene risultati e resta fuori dal repository.

B.3 Riproduzione dell'esempio completo

L'esempio del Capitolo 2 calcola

$$f(x, y) = (x_1 \vee y_1) \wedge ((x_2 \wedge y_2) \wedge (x_3 \vee y_3)),$$

che nella sintassi della CLI si scrive con `&` per AND e `|` per OR. Eseguire le tre fasi con tre comandi separati le rende osservabili una per una.

Costruzione

Il primo comando genera entrambe le alternative di ogni occorrenza di input, e non riceve i valori di x e y :

```
1 v2pc construct "(x1|y1)&((x2&y2)&(x3|y3))" \
2   --output costruzione --side 32 --seed 7
```

La cartella `costruzione` raccoglie il circuito, le matrici delle due alternative e le rispettive immagini. Il seme rende riproducibile la casualità; omettendolo, il programma ne sceglie uno nuovo.

Distribuzione simulata

Il secondo comando tiene una sola alternativa per ingresso:

```
1 v2pc transfer --source costruzione \
2   --input "x1=0,y1=1,x2=1,y2=1,x3=1,y3=0" \
3   --output trasferimento
```

Il pacchetto risultante non conserva l'assegnamento né le alternative scartate. Per gli ingressi di Alice la selezione vale come consegna diretta; per quelli di Bob è un oblivious transfer simulato in locale, con i limiti discussi nel Capitolo 3.

Ricostruzione

L'ultima fase riceve soltanto il pacchetto trasferito:

```
1 v2pc reconstruct --source trasferimento
```

Porta per porta la procedura recupera il pointer bit che le serve, seleziona la metà giusta della share destra e la sovrappone alla share sinistra. Con l'assegnamento qui sopra l'immagine finale è tutta nera e si legge come 1, in accordo con la valutazione booleana della formula. Share ricevute, risultati intermedi e uscita finiscono in `trasferimento/reconstruction`.

B.4 Demo web ed esportazione fisica

La stessa implementazione si avvia nel browser con `v2pc serve`, che mette in ascolto `http://127.0.0.1:5000`. L'interfaccia è descritta nella Sezione 3.8; quello che interessa qui è il materiale che produce. Gli archivi scaricabili sono due: il primo, con tutte le alternative, serve alla preparazione che precede la conoscenza degli input; il secondo, con le sole share selezionate, serve a verificare la sovrapposizione. In ciascuno c'è un unico file `LEGGIMI.txt`, invece di istruzioni sparse in più documenti, e la Tabella B.1 riassume il contenuto.

Elemento	Funzione
PDF A4	Dispone le share a una scala fisica comune, con reticolo, bordi di taglio e crocini di allineamento.
PNG individuali	Forniscono una copia digitale di ogni share con il pointer bit nella posizione prevista dal paper.
<code>LEGGIMI.txt</code>	Riunisce contenuto dell'archivio, impostazioni di stampa, distribuzione, taglio e sequenza di sovrapposizione.
Immagine di uscita	Nel kit delle share selezionate costituisce il riferimento con cui confrontare la ricostruzione fisica.

Tabella B.1: Materiale prodotto dalla demo web per l'esperimento fisico.

Due avvertenze contano più delle altre. La stampa va fatta al 100%, con l'adattamento automatico alla pagina disattivato, altrimenti le share non si sovrappongono. E le etichette che rivelano il valore dell'alternativa servono solo in preparazione: vanno rimosse dal materiale consegnato nell'esperimento fisico, perché altrimenti rivelano l'input. Un esemplare di quel PDF, prodotto per l'esempio del paper, chiude l'appendice nella Sezione B.7.

B.5 Riproduzione delle misure

I due script che seguono producono i numeri e le tabelle della Sezione 3.9. Vanno eseguiti con l'ambiente virtuale attivo, così che `import v2pc` funzioni da qualunque cartella. Nessuno dei due usa file intermedi: costruiscono, distribuiscono e ricostruiscono in memoria, e stampano il risultato su terminale.

Crescita delle share

Il primo misura larghezze e ingombro di una costruzione. Le larghezze sono quelle delle immagini associate alle occorrenze di input, il totale comprende entrambe le alternative di ogni foglia e i canali dei pointer.

```

1 from v2pc.circuit import parse_expression
2 from v2pc.protocol import build
3
4 FORMULE = (
5     "x1&y1",
6     "(x1&y1) & (x2&y2) ",
7     "(x1|y1) & ( (x2&y2) & (x3|y3) ) ",
8     "( (x1&y1) & (x2&y2) ) & ( (x3&y3) & (x4&y4) ) ",
9 )
10
11 for formula in FORMULE:
12     costruzione = build(parse_expression(formula), side=32, seed=1)
13     larghezze = sorted({f.images[0].shape[1] for f in
14         ↪ costruzione.leaves})
15     print(costruzione.circuit.gate_count, larghezze,
16         ↪ costruzione.total_pixels)

```

Cambiando `side` in 8 e tenendo la sola formula del paper si ottengono le larghezze 8, 16, 32 e 64 e il totale di 2724 pixel citati nel capitolo.

Errore di lettura ed esponente

Il secondo conta quante volte un'uscita che vale 0 viene letta come 1. Per ogni formula serve un assegnamento che annulli la funzione, altrimenti non c'è area bianca da perdere. Il ciclo ricostruisce da capo a ogni giro, con un seme diverso, così

che le costruzioni siano indipendenti fra loro. Dall'errore si ricava poi l'esponente invertendo $P = (1 - 2^{-k})^{s^2}$.

```

1 import math
2
3 from v2pc.circuit import parse_expression
4 from v2pc.protocol import build, evaluate
5
6 CASI = (
7     ("x1&y1", {"x1": 0, "y1": 0}, 2),
8     ("(x1&y1) & (x2&y2)", {"x1": 0, "y1": 0, "x2": 0, "y2": 0},
9     ↪ 3),
10    ("(x1|y1) & ((x2&y2) & (x3|y3))", {"x1": 0, "y1": 0, "x2": 1,
11    ↪ "y2": 1, "x3": 1, "y3": 1}, 4),
12    ("((x1&y1) & (x2&y2)) & ((x3&y3) & (x4&y4))",
13    ↪ {"f"{parte}{i}": 0 for i in (1, 2, 3, 4) for parte in ("x", "y")},
14    ↪ 4),
15 )
16 PROVE = 4000
17
18 for formula, assegnamento, lato in CASI:
19     circuito = parse_expression(formula)
20     sbagliate = sum(
21         1
22         for seme in range(PROVE)
23         if evaluate(build(circuito, side=lato, seed=seme),
24             ↪ assegnamento).value != 0
25     )
26     errore = sbagliate / PROVE
27     k = -math.log2(1 - errore ** (1 / lato**2))
28     print(f"{circuito.gate_count} porte s={lato} "
29         ↪ f"errore={errore:.4f} k={k:.2f}")

```

L'uscita riempie la Tabella 3.2:

1	1 porte	s=2	errore=0.0655	k=1.02
2	3 porte	s=3	errore=0.2890	k=2.96
3	5 porte	s=4	errore=0.6035	k=5.01
4	7 porte	s=4	errore=0.8840	k=7.03

I lati sono piccoli di proposito. Con s grande l'errore scende sotto la risoluzione di quattromila prove e l'esponente non si può più stimare, mentre con questi valori resta fra il 6% e il 90% e si legge bene. Aggiungendo a CASI le due catene

$((x1 \& y1) \& y2) \& y3$ con $s = 3$ e $((x1 \& y1) \& y2) \& y3 \& y4$ con $s = 4$, entrambe con tutte le variabili a zero, si ottengono $k = 2,99$ e $k = 4,06$: sono i circuiti in cui porte e livelli coincidono e le due stime danno la stessa risposta.

Il numero di apertura della sottosezione, cioè il 13,0% di errore con il lato $s = 8$ scelto dagli autori, viene dallo stesso ciclo ristretto alla formula del paper:

```

1 circuito = parse_expression("(x1|y1)&((x2&y2)&(x3|y3))")
2 assegnamento = {"x1": 0, "y1": 0, "x2": 1, "y2": 1, "x3": 1, "y3": 1}
3
4 sbagliate = sum(
5     1
6     for seme in range(3000)
7     if evaluate(build(circuito, side=8, seed=seme), assegnamento).value
8     ↪ != 0
9 )
10 print(sbagliate, sbagliate / 3000)

```

Il conteggio è 391 su tremila. Il tempo di esecuzione dipende dalla macchina, quasi tutto speso nella costruzione delle share e non nella ricostruzione.

B.6 Riproducibilità e limiti dell'esecuzione

Due parametri decidono cosa si ottiene rieseguendo il programma. `-seed` rigenera le stesse matrici a parità di versione dello schema, generatore e ambiente NumPy, senza toccare la funzione calcolata: non è una chiave crittografica. `-side` fissa il lato dell'immagine di base, e qui c'è un compromesso da scegliere, perché valori più alti abbassano la probabilità che una ricostruzione bianca venga letta per nera, ma costano memoria, spazio su disco e tempo. La Sezione 3.9.2 propone, sulla base delle misure, il criterio $s \geq \sqrt{\log_{1-2^{-\ell}} \varepsilon}$ con ℓ il numero di porte della formula. Il programma non lo calcola, quindi il controllo resta a chi esegue.

I valori riportati nel Capitolo 3 vengono da semi consecutivi a partire da zero, come negli script della Sezione B.5; ripetendoli con altri semi le frequenze possono cambiare per variabilità campionaria. La compatibilità con il modello va valutata rispetto al numero di prove, non all'identità delle ultime cifre.

Quanto ai limiti, quello che conta è ricordare cosa queste istruzioni riproducono

davvero: le relazioni fra le share, la protezione dei pointer bit e la sequenza di taglio e sovrapposizione. Non l’oblivious transfer, che qui resta una selezione locale. Chi ripete l’esperimento sta studiando il protocollo, non eseguendo un calcolo sicuro fra due soggetti.

B.7 Tavole di taglio delle share

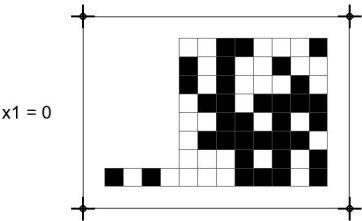
Le due pagine seguenti contengono le sei share selezionate per $x = (0, 1, 1)$ e $y = (1, 1, 0)$. Le ha generate il programma con immagini di lato $s = 8$, seme 4 e scala comune: non sono ricostruzioni grafiche fatte a parte. Ogni rettangolo delimitato dai crocini è un elemento da ritagliare, il blocco 1×2 del pointer bit sporge in basso a sinistra come nel paper, e nelle share destre la guida centrale separa le due metà lasciando ciascun pointer unito alla propria immagine. Vale anche qui la regola della stampa al 100%, o a “dimensioni reali”.

V2PC - share selezionate

Kit di verifica dopo la selezione degli input; le alternative scartate non sono incluse.
Stampare al 100% / dimensioni reali. Disattivare 'Adatta alla pagina'.

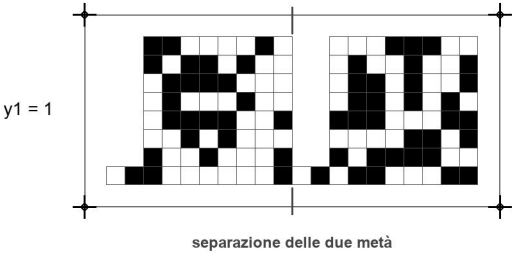
S01 - x1

Alice · consegna diretta



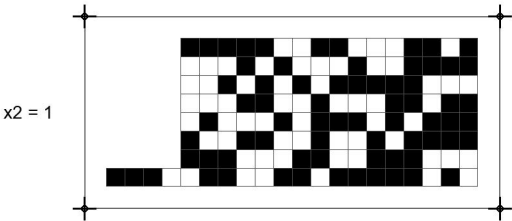
S02 - y1

Bob · OT simulato



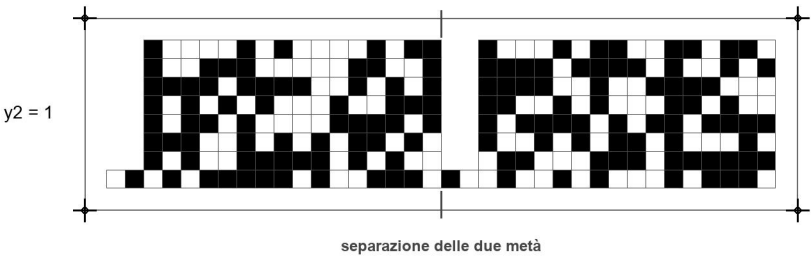
S03 - x2

Alice · consegna diretta



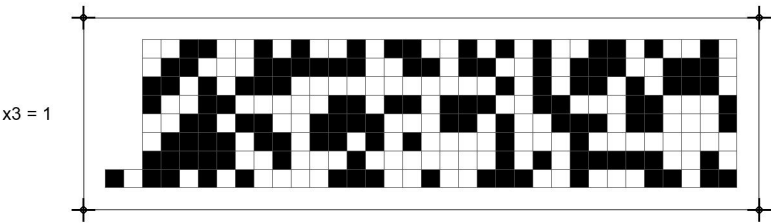
S04 - y2

Bob · OT simulato



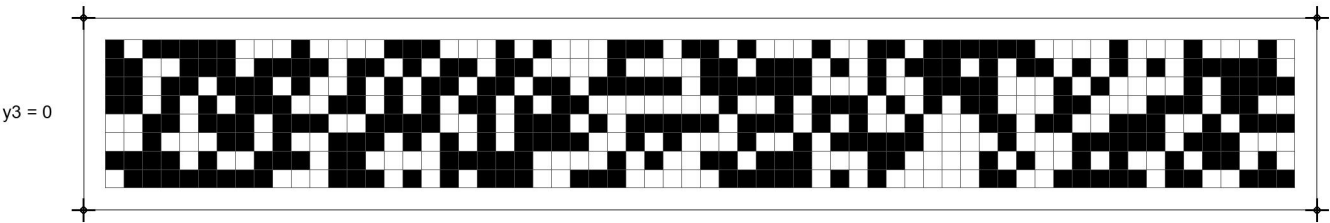
S05 - x3

Alice · consegna diretta



S06 - y3

Bob · OT simulato



Bibliografia

- [1] P. D'Arco and R. De Prisco, "Secure two-party computation: a visual way," in *Information Theoretic Security (ICITS 2013)*, ser. Lecture Notes in Computer Science, vol. 8317. Springer, 2014, pp. 18–38. (Citato alle pagine vi, 2, 22, 28, 32, 36, 37, 50 e 51)
- [2] A. C. Yao, "Protocols for secure computations," in *23rd Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 1982, pp. 160–164. (Citato alle pagine 1 e 22)
- [3] A. C.-C. Yao, "How to generate and exchange secrets," in *27th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 1986, pp. 162–167. (Citato alle pagine 1 e 24)
- [4] Y. Lindell, "Secure multiparty computation," *Communications of the ACM*, vol. 64, no. 1, pp. 86–96, 2021. (Citato alle pagine 1 e 23)
- [5] M. Naor and A. Shamir, "Visual cryptography," in *Workshop on the Theory and Application of Cryptographic Techniques*. Springer, 1994, pp. 1–12. (Citato alle pagine 1, 6 e 9)
- [6] P. D'Arco and R. De Prisco, "Secure computation without computers," *Theoretical Computer Science*, vol. 651, pp. 11–36, 2016. (Citato alle pagine 2, 22, 45 e 53)
- [7] A. De Angelis and B. Del Monte, "VisualTwoPartyComputation: implementazione in C++ del protocollo v2pc," <https://github.com/VenturaDelMonte/>

- VisualTwoPartyComputation, 2016, software sviluppato come lavoro di tesi triennale. (Citato alle pagine 2 e 53)
- [8] C. Liu, *Introduction to combinatorial mathematics*. New York, McGraw-Hill, 1968. (Citato a pagina 5)
- [9] A. Shamir, "How to share a secret," *Commun. ACM*, vol. 22, no. 11, p. 612–613, 1979. [Online]. Available: <https://doi.org/10.1145/359168.359176> (Citato a pagina 5)
- [10] G. R. Blakley, "Safeguarding cryptographic keys," in 1979 *International workshop on managing requirements knowledge*. IEEE Computer Society, 1979, pp. 313–318. (Citato a pagina 6)
- [11] C. E. Shannon, "Communication theory of secrecy systems," *The Bell system technical journal*, vol. 28, no. 4, pp. 656–715, 1949. (Citato a pagina 7)
- [12] P. D'Arco and R. De Prisco, "Visual cryptography: models, issues, applications and new directions," in *International Conference for Information Technology and Communications*. Springer, 2016, pp. 20–39. (Citato alle pagine 8, 9, 16 e 18)
- [13] G. Ateniese, C. Blundo, A. De Santis, and D. R. Stinson, "Visual cryptography for general access structures," *Information and Computation*, vol. 129, no. 2, pp. 86–106, 1996. (Citato a pagina 8)
- [14] O. Kafri and E. Keren, "Encryption of pictures and shapes by random grids," *Optics Letters*, vol. 12, no. 6, pp. 377–379, 1987. (Citato alle pagine 15 e 16)
- [15] R. De Prisco and A. De Santis, "On the relation of random grid and deterministic visual cryptography," *IEEE Transactions on Information Forensics and Security*, vol. 9, no. 4, pp. 653–665, 2014. (Citato alle pagine 16 e 18)
- [16] C.-N. Yang, "New visual secret sharing schemes using probabilistic method," *Pattern Recognition Letters*, vol. 25, no. 4, pp. 481–494, 2004. (Citato a pagina 17)
- [17] S. Cimato, R. De Prisco, and A. De Santis, "Probabilistic visual cryptography schemes," *The Computer Journal*, vol. 49, no. 1, pp. 97–107, 2006. (Citato a pagina 18)

-
- [18] T. Hofmeister, M. Krause, and H. U. Simon, "Contrast-optimal k out of n secret sharing schemes in visual cryptography," in *Computing and Combinatorics (COCOON 1997)*, ser. Lecture Notes in Computer Science, vol. 1276. Springer, 1997, pp. 176–185. (Citato a pagina 20)
- [19] C. Blundo, A. De Santis, and D. R. Stinson, "On the contrast in visual cryptography schemes," *Journal of Cryptology*, vol. 12, no. 4, pp. 261–289, 1999. (Citato a pagina 20)
- [20] C. Blundo, A. De Bonis, and A. De Santis, "Improved schemes for visual cryptography," *Designs, Codes and Cryptography*, vol. 24, no. 3, pp. 255–278, 2001. (Citato a pagina 20)
- [21] A. De Bonis and A. De Santis, "Randomness in secret sharing and visual cryptography schemes," *Theoretical Computer Science*, vol. 314, no. 3, pp. 351–374, 2004. (Citato a pagina 21)
- [22] O. Goldreich, *Foundations of Cryptography, Volume II: Basic Applications*. Cambridge University Press, 2004. (Citato alle pagine 23 e 50)
- [23] Y. Lindell and B. Pinkas, "A proof of security of yao's protocol for two-party computation," *Journal of Cryptology*, vol. 22, no. 2, pp. 161–188, 2009. (Citato alle pagine 24 e 25)
- [24] S. Even, O. Goldreich, and A. Lempel, "A randomized protocol for signing contracts," *Communications of the ACM*, vol. 28, no. 6, pp. 637–647, 1985. (Citato a pagina 25)
- [25] V. Kolesnikov, "Gate evaluation secret sharing and secure one-round two-party computation," in *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 2005, pp. 136–155. (Citato alle pagine 29, 44 e 50)
- [26] D. Chaum, "Secret-ballot receipts: True voter-verifiable elections," *IEEE security & privacy*, vol. 2, no. 1, pp. 38–47, 2004. (Citato a pagina 38)